

Monitoring of Cuff Pressure in Intubated Patients



AALBORG UNIVERSITY
DENMARK

GROUP HST-24-ST-6-6406

BIOMEDICAL ENGINEERING AND INFORMATICS 6. SEMESTER 2024

AALBORG UNIVERSITY



AALBORG UNIVERSITY
DENMARK

Department of Health Science and Technology

Biomedical Engineering and Informatics

Selma Lagerløfs Vej 249

9220 Aalborg Øst

<http://www.hst.aau.dk/>

Title:

Monitoring of cuff pressure in intubated patients

Project:

Semesterproject 6th semester

Project period:

1 February 2024 - 30 May 2024

Projectgroup:

Hst-24-st-6-6406

Participants:

Kirstine Riisager Madsen

Muna Mohamed Ali Abdi Guled

Natasja Toftgaard Garner

Signe Kirstine Rosager

Signe Meiners Lau

Supervisor: Dan Stieper Karbing

Page numbers ex. Appendix: 114

Page numbers incl. Appendix: 152

Finished: 24-05-2024

Abstract:

The following report presents the process of designing a system, the CPATool, meant to support cuff pressure monitoring in the ICU. Cuff pressure monitoring is essential in optimal endotracheal intubation practice, as over- or under inflation of the cuff can lead to a variety of complications. The aim of this project was to develop the system to monitor cuff pressure over time, thereby enable an analysis to facilitate quality control in the ICU, inferred from the preceding problem analysis. Based on user requirements, formed in collaboration with chief physician Nilanjan Dey at Operation and Intensive, Region Hospital Gødstrup, and developers of the TrachFlush device, AW Technologies, a system design will be presented through various analysis and design methods, ultimately leading to an overview of the implementation of the system as a whole. Lastly, the system and its functionality are examined through a series of tests, to ensure compliance with the established requirements. In conclusion, the system visualizes cuff pressure as intended, allowing the user to perform quality management procedures, based on the findings of the cuff pressure data recorded for this project.



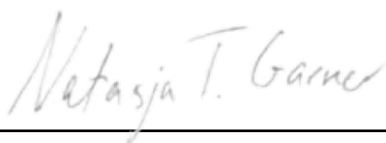
Kirstine Riisager Madsen

krma21@student.aau.dk



**Muna Mohamed Ali Abdi
Guled**

mguled21@student.aau.dk



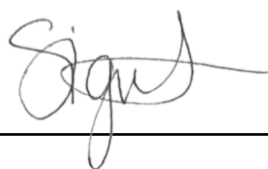
Natasja Toftgaard Garner

ngarne21@student.aau.dk



Signe Kirstine Rosager

srosag21@student.aau.dk



Signe Meiners Lau

silau21@student.aau.dk

Preface

The project is written by group hst-24-st-6-6406 in the 6th semester of the bachelor in Biomedical Engineering and Informatics at Aalborg University, with the theme *Clinical Information and Technology Development*. The project was written from February 1st to May 30th and has been developed under guidance from supervisor Dan Stieper Karbing, Assistant Professor, Department of Health Science and Technology at Aalborg University. We would like to express our gratitude for his guidance. Additionally, we would like to thank Nilanjan Dey from Operation and Intensive, Region Hospital Gødstrup, and the developers at AW Technologies for their collaboration.

Reference method

The reference method used in the project is the Vancouver method. The reference will appear as a number in square brackets [x]. The number in the square brackets will correspond to the source reference in the bibliography, which is arranged chronologically. If the reference is positioned before the end of a sentence, it means that the reference belongs to that specific sentence. If the reference is positioned after the period, it indicates that the reference pertains to the entire paragraph.

Contents

I	Problem analysis	1
1	Introduction	2
2	Introduction to Intubation	3
2.1	Mechanical Ventilation	3
2.2	Endotracheal Tube	3
2.3	Endotracheal Suction	4
3	Measurements Methods	5
3.1	Conventional Measurement Methods	5
3.2	Cuff Pressure Complications	6
3.3	Direct Intracuff Pressure Measurement Technique	7
4	Cuff Technologies	9
4.1	Cuff Measurements and Regulating Technologies	9
5	Problem Delimitation	11
II	Solution	12
6	Method	13
6.1	Verification and Validation Model	13
7	Requirements Analysis	15
7.1	System definition	15
7.2	Use Scenario	16
7.3	User Requirements	16
7.4	Use Case Diagram	17
7.5	Use Cases	19
8	European Union Requirements	22
8.1	Medical Device Regulation	22
8.2	Technical Documentation	23
8.3	International Organization for Standardization	24

8.4	European Conformity Marking	25
8.5	Intended Purpose	26
8.6	Risk Classification	27
8.7	Risk Management	28
IIIDesign		35
9	System Specification	36
9.1	System Requirements	36
9.2	Database Requirements	42
10	High Level Design	46
10.1	Prototype	46
10.2	Activity Diagram	51
11	Low Level Design	54
11.1	Class Diagram	54
11.2	Entity Relationship Diagram	56
11.3	Design of Algorithms	58
IVImplementation		62
12	Implementation	63
12.1	Block 1: Database Communication	64
12.2	Block 2: Screen Login	66
12.3	Block 3: Screen Layout	68
12.4	Block 4: Add New Treatment ID	69
12.5	Block 5: Calculate Events and Statistics	69
12.6	Block 6: Dashboard 1	72
12.7	Block 7: Dashboard 2	73
12.8	Implemented Activity Diagram	74
V Test		75
13	Software Testing	76
13.1	Test Cases	76
13.2	Black Box Testing	77

13.3 Test Results	88
VI Discussion and Conclusion	92
14 Discussion	93
14.1 The Addressed Problem	93
14.2 Results	93
14.3 Problem Solving	94
15 Future perspective	98
16 Conclusion	100
Bibliography	101
VII Appendix	108
A Risk Management Plan	109
A.1 References	109
A.2 Scope	109
A.3 Methodology	109
A.4 Assignment of Responsibilities and Authors	110
A.5 Risk Acceptability Criteria	110
A.6 Detailed Descriptions of Steps in the Risk Management Plan	110
B Pilot Studies	115
B.1 Baseline test	115
B.2 Flush test	118
B.3 Pressure test	121
B.4 Cuff leakage test	125
C Test Case	129
C.1 Unit Test Case	129
C.2 Block Testing	131

Part I

Problem analysis

1. Introduction

Patients hospitalized in the intensive care unit (ICU) are critically ill and typically have present organ dysfunction, necessitating intensive monitoring and treatment. Some of these patients require mechanical ventilation to assist with respiration. To maintain open airways during mechanical ventilation, an endotracheal tube is often used. In the United States, the incidence of endotracheal intubation was estimated to be 15 million annually in 2010. To prevent aspiration into the lungs, an inflatable cuff is placed on the endotracheal tube. It is crucial to regulate the cuff pressure correctly to avoid complications, which will be described further in the problem analysis.[1] The cuff pressure related to the endotracheal tube must be regulated correctly to avoid complications.[2] A complication that arises during endotracheal intubation is ventilator-associated pneumonia (VAP). The reported incidence of VAP varies significantly from 5% to 40% [3], and despite various preventative measures, it remains high [4]. The range of incidence is partially due to different hospital and country conditions as well as VAP diagnosis and prevention methods. [4]. Furthermore, a report from 2014 in the United States shows that approximately 10% of patients requiring mechanical ventilation in 2011 developed VAP [5]. Additionally, it has been documented that VAP is associated with mortality rates as high as 50%, a factor influenced by the variance among patients on mechanical ventilation.[6]

Besides VAP other complications occur under endotracheal intubation, therefore the following initial thesis will be researched:

What positive and negative effects does cuff pressure have on patients during endotracheal intubation, and which systems are used to control cuff pressure?

2. Introduction to Intubation

To comprehend the effects of cuff pressure on patients, it is relevant to understand the process of intubating patients. This chapter will describe the purpose of mechanical ventilation and the functionality of endotracheal intubation.

2.1 Mechanical Ventilation

Mechanical ventilation is used in the ICU where the patients are critically ill [7]. The purpose of mechanical ventilation is to support breathing, under surgery or in acute situations when the patient is unable to breathe on their own or needs support for respiration. [8] Spontaneous breathing, also known as normal breathing, occurs as the respiratory muscles facilitate the movement of air in and out of the lungs. In contrast, mechanical ventilation operates by applying positive pressure to the airway, thus promoting the inflow of oxygen and the removal of carbon dioxide from the lungs [9, 10].

For critically ill patients, there are two types of mechanical ventilation using positive pressure: non-invasive and invasive ventilation. In the non-invasive method, the patient is wearing a face mask connected to the ventilator. By invasive mechanical ventilation, the patient has a tube placed in the airway connected to a ventilator.[11]

There are two methods of intubation with invasive mechanical ventilation: Tracheostomy and endotracheal, which both facilitate the exchange of gases in the lungs. A tracheostomy is inserted in the neck into the airway with a surgical procedure to create an opening for the tube.[12, 13] Endotracheal intubation is when the tube is inserted through the mouth or nose. [14]

Endotracheal intubation serves as a temporary measure that can be easily inserted and removed, whereas tracheostomy represents a more permanent intervention. Therefore endotracheal intubation is used as the primary intubation before tracheostomy. [14]

2.2 Endotracheal Tube

An endotracheal tube (ETT) is a tube made of polyvinyl chloride (PVC). It is inserted between the vocal cords into the trachea to maintain open airways ensuring ventilation while protecting the lungs against potential contamination such as gastric fluids or blood.[1]

The gas flow is delivered to the ETT through a connector attached to the mechanical ventilator tube. The diameter of the tube impacts the ease of gas flow, with narrower tubes presenting greater resistance. Therefore, selecting the largest appropriate tube for the patient is crucial, particularly for spontaneously breathing patients who must overcome increased resistance.[1] Ideally, the distal tip of the ETT should be positioned approximately 4 cm above the carina, in adult patients.[1] The placement is visualized in figure 2.1.

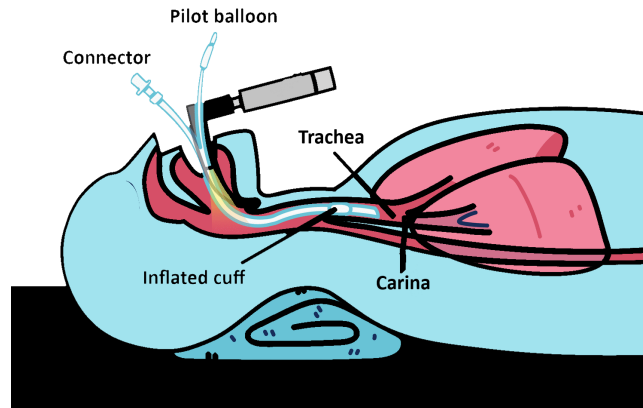


Figure 2.1: Illustration of the placement of the endotracheal tube. Modified from [15].

Most ETTs feature an inflatable balloon, known as a cuff, at its distal end. Upon inflation, the cuff forms a seal against the tracheal wall. This prevents the entry of gastric contents into the trachea and creates positive pressure ventilation. [1]. The ideal cuff pressure is within the range of 20-30 cm H_2O , to prevent pressure damage while ensuring an optimal seal. [2, 16]. The inflation of the cuff can be achieved by using a syringe attached to the pilot balloon. Subsequently, the syringe must be removed to prevent air from escaping and deflating the cuff. [1]

2.3 Endotracheal Suction

The intubation under mechanical ventilator treatment stimulates the mucous membranes in the airways to increase the production of secretion. The intubated patient is not able to keep their airway clear of secretion, due to their decreased cough force and inability to transport secretion up above the cuff. To compensate for this, endotracheal suction (ETS) is used to clean the airways, thereby decreasing the risk of VAP.[17]

The ETS will often result in discomfort for the patient. Almost 50% of patients associate ETS with anxiety, stress, and pain. Furthermore, complications are also related to the ETS, such as hypoxia which is experienced as a feeling of suffocation and trauma to the tracheal mucosa, leading to the risk of infections and bleeding. [18, 17]

3. Measurements Methods

To ensure optimal cuff pressure, measurement is essential. Various measurement methods can be employed for this purpose. The following chapter will describe different measurement methods and which complications that may arise from unregulated cuff pressure.

3.1 Conventional Measurement Methods

To inflate the cuff to the desired pressure, of 20-30 cm H_2O [2], different techniques are used. It has been estimated that 98% of ETT cuff inflations are done using 'estimation techniques', even though these techniques have been proven to be consistently inaccurate [19]. The following four techniques describe how cuff inflation is done using these estimations.

The *minimal occlusive volume technique* works by inflating the endotracheal cuff with the smallest amount of air needed to prevent air leakage at end-inspiration [20]. A study shows that 2 out of 15 patients were in the ideal range of cuff pressure when using the minimal occlusive volume technique [21].

The *minimum leak technique* is used by injecting air into the cuff, to the point where a small amount of air leakage is auscultated at end-inspiration. It is important to note that the *small* amount is not defined, and therefore variation in results is to be expected. [21] A study shows that when using the minimum leak technique, 20 out of 45 were in the ideal range of cuff pressure [22].

The *predetermined volume technique* works by injecting a predetermined volume of air into the cuff. This method of cuff inflation does not factor in components such as tracheal diameter, thoracic pressure, and the type of anesthetic agent used. [21] A study shows that while using this method, only 3 out of 20 patients were in the ideal range of cuff pressure [21].

The most commonly used technique is the *palpation technique*. With this technique, the endotracheal cuff is inflated with air, while the intracuff pressure is estimated by the palpation of the pilot balloon. [21] A study shows that 3 out of 9 patients were in the ideal range of cuff pressure, using this method [21].

As the studies indicate, the majority of the time, when employing these conventional measurement techniques, the cuff pressure did not remain within the ideal range. Typically, baseline cuff pressure

is checked every 8-12 hours, but it varies from hospital to hospital. There can also be reasons to check it more frequently. The reasons for checking more often could be due to clinical findings such as air leakage or ventilator alarms indicating pressure changes. If the pressure becomes either too high or too low, it could lead to various complications.[23]

3.2 Cuff Pressure Complications

The cuff pressure must be high enough to prevent microaspiration while also being low enough to allow some air leakage, ensuring proper ventilation [24]. A study from 2012 shows that 51.7% of pressure values were out of range as set between 20-30 cm H_2O [2].

3.2.1 Low Cuff Pressure Complications

Multiple factors, such as the efficacy of the seal and occurrences of cuff leaks can cause low cuff pressure complications. [25, 26]

The probability of patients undergoing endotracheal intubation developing VAP increases if the cuff does not achieve a proper seal, due to the pressure of the cuff being too low, defined as under 20 cm H_2O [25, 27]. VAP is a type of pneumonia that can occur more than 48 hours after the patient has been intubated [27]. VAP frequently occurs due to the aspiration of bacteria from the oral cavity into the lower respiratory tract. Since intubated patients are unable to cough up the bacteria, secretions, and mucus, which may lead to a change in the bacterial flora of the oral cavity, thereby resulting in the development of VAP. [28]

Cuff Leak

Cuff leaks are a problem among mechanically ventilated patients. Studies show that up to 11% of intubations do not have a proper airway seal. Cuff leak can potentially result in life-threatening respiratory compromise, if not properly managed. [19]

As mentioned in section 3.2.1, underinflated cuffs can lead to leakage around the endotracheal tube, although leakage can also occur due to structural cuff and/or inflation system defects. These defects can be due to either unintentional damage or manufacturing defects. [26]

Damage or defects can occur in different parts of the intubation system, such as the cuff inflation valve, the pilot balloon, or the pilot tubing. The most common structural cause of leakage is cuff defects. While cuff defects may be the result of a manufacturing defect, more often than not, cuff defects are caused by trauma to the cuff, from either teeth, during insertion, or surgical instruments, during neck surgery. [26]

3.2.2 High Cuff Pressure Complications

Various external factors can influence the cuff pressure, including position changes which can lead to different high cuff pressure complications.[29, 24]

The primary cause of tracheal injury is the high pressure applied to the tracheal wall [24]. Post-intubation sore throat is a frequently experienced side effect of the cuff being overly inflated. This over-inflation can cause ischemia in the tracheal- or oropharyngeal mucosa. This is frequently observed when cuff pressures exceed 30 cm H_2O . In contrast, if the pressure surpasses 50 cm H_2O , it can result in total obstruction of the tracheal blood flow, which in rare instances may lead to acute complications such as tracheal bleeding or rupture.[30]

A study from 2016 was conducted to compare instrumental methods against conventional methods for measuring endotracheal tube cuff pressure. In the study, the mentioned side effects were observed, and it was found that the cuff pressure was often measured too high using conventional measurement methods. It was noted that 69% of the measured cuff pressures were too high. Therefore, an advantage is seen in utilizing continuous cuff pressure management, as the conventional method is deemed unreliable. [24]

The Effect of Position Changes

As mentioned in section 3.2.2, factors such as changes in body position during the placement of the endotracheal tube in the mouth can affect the cuff pressure. The basic position for patients is lying on their back with the head elevated to 30 degrees.[29]

A study from 2022 has examined the impact of position changes from the basic position on cuff pressure. Three different positions were investigated, and the pressure was monitored over 0-90 minutes. It was observed that cuff pressure increased within 0 minutes after changing positions, while after 90 minutes, the pressure had decreased, almost back to the baseline, which was set at 25 cm H_2O . Thus, the study found that position changes can have an impact on cuff pressure.[29]

3.3 Direct Intracuff Pressure Measurement Technique

As explained in section 3.1, too low or too high cuff pressure can often be a result of the use of conventional measurement methods. It is estimated that 60-80% of cuff pressures are out of the recommended range [19]. To decrease this, instrumental measurements can be used. This enables the maintenance of the cuff pressure to prevent it from becoming too low or too high. [31]

The manometer is recommended to monitor cuff pressure[32]. The manometer is a simple pressure

gauge, which can be connected directly to the intubation tube, after inflation, via the pilot balloon, and thus shows the pressure of the endotracheal cuff [33].

While the manometer provides a more precise reading of the cuff pressure, studies show that the cuff pressure decreases every time the manometer is connected and disconnected to the ETT [34]. Disconnection of the manometer in the first and fifth minutes showed a decrease in cuff pressure from 25 cm H_2O to 19.11 cm H_2O . Meanwhile, the control, in which the manometer was not disconnected during monitoring, showed no difference in cuff pressure [34]. Therefore continuous cuff monitoring and regulation are to be desired.

4. Cuff Technologies

To help healthcare personnel maintain and monitor cuff pressure, several systems have been developed. These include IntelliCuff from Hamilton, Puritan Bennett Cuff Pressure Manager from Medtronic, and TrachFlush from AW Technologies which can be seen in figure 4.1. [35, 36, 37] The cuff technologies can be used for endotracheal intubation.



(a) IntelliCuff.



(b) Puritan Bennett cuff pressure manager.



(c) TrachFlush.

Figure 4.1: Different cuff pressure monitors: (a) IntelliCuff from Hamilton [35], (b) Puritan Bennett from Medtronic [38], (c) TrachFlush from AW Technologies [37].

4.1 Cuff Measurements and Regulating Technologies

Hamilton- IntelliCuff

The IntelliCuff system by Hamilton, as seen in figure 4.1a, enables the healthcare personnel to set the desired cuff pressure, following the system will automatically maintain the cuff pressure through its built-in manometer. Additionally, it can detect leakage accumulated around the cuff, and feature separated buttons on the controller to control inflation and deflation of the cuff. The Intellicuff can also be set to give alarms depending on the measured cuff pressure. The IntelliCuff is designed for use on adult, pediatric, and neonatal patients. [35]

Medtronic - Puritan Bennett Cuff Pressure Manager

The Puritan Bennett Cuff Pressure Manager by Medtronic, which can be seen in figure 4.1b, offers functionalities similar to the IntelliCuff, including cuff pressure control, leakage detection, and cuff inflation and deflation. [36] It is suitable for use with both adult and pediatric patients. [38]

AW Technologies - TrachFlush

The TrachFlush from AW Technologies consists of two primary functions, a Cuff Control- and a Flush

Control function, the device can be seen in figure 4.1c. The Cuff Control function is used for regulating cuff pressure and shares the most important functions with the IntelliCuff and the Puritan Bennett Cuff Pressure Manager. Additionally, the TrachFlush device is also connected to the ventilator output. This enables the TrachFlush to utilize the airway pressure to set a more precise cuff pressure. The Flush Control function aims to remove tracheal secretion from below and above the cuff. An in vivo study from 2022 has shown that the flush function may decrease the use of endotracheal suctioning and its complications. Given the flush function's ability to reduce the frequency of endotracheal suctioning, it is expected that this could also positively impact the incidence of VAP. [39]

The Flush Control function is activated with a button called the Deflate/Inflate button. When pushed the TrachFlush deflates the cuff to 0 cm H_2O . It utilizes then the ventilator air pressure to flush the secretion upward into the oral cavity. Subsequently, the cuff reinflates to the set target pressure all during an inspiratory cycle. The TrachFlush can only be used on adults. [37]

4.1.1 Limitations of Current Technologies

None of the aforementioned technologies incorporate the functionality of continuous monitoring over an extended period. The incorporation of this functionality presents considerable potential to get a better understanding of the parameters that affect cuff pressure regulation over time [40].

5. Problem Delimitation

Based on the problem analysis, it is assessed that wrongful measurements as well as external factors can affect cuff pressure under endotracheal intubation. High and low cuff pressure can lead to complications such as tracheal injuries and VAP. VAP especially is of significant concern, due to its wide range of incidence, despite various preventative measures, and its high mortality rate. Conventional measurement methods, as described in section 3.1, are used to regulate cuff pressure, however, this has been proven to be inaccurate. In recent times new technologies have been developed to measure and support the regulation of cuff pressure, due to the need for more precise monitoring.

As mentioned in chapter 4, IntelliCuff from Hamilton, Puritan Bennett Cuff Pressure Manager from Medtronic, and TrachFlush from AW Technologies all possess these functions. However, these technologies lack the ability to save measured data over time. The purpose of saving data is to detect variations in cuff pressure including wear of the cuff, changes in treatment, as well as a quality control tool in the ICU.

AW Technologies, developers of the TrachFlush states that the ability to save and later analyze cuff pressure data will add significant value to the quality control of the device. Furthermore, a study performed by Nilanjan Dey et al, at Operation and Intensive, Region Hospital Gødstrup[41], emphasizes how important the monitoring of cuff pressure is. They state that a proper seal is important to mitigate the risks of VAP. Therefore, continuous cuff pressure monitoring could be of significance.

Therefore it is relevant to solve the following problem statement:

How can a system including a database be developed to monitor cuff pressure over time and thereby enable an analysis to facilitate quality control in the ICU related to maintaining cuff pressure?

Part II

Solution

6. Method

The following chapter will include a description of the Verification and Validation Model (V-model), which helps to create structure throughout the project, thus illustrating the design process that the project will adhere to. Additionally, the Object-Oriented Method (OOM) will be described, providing insight into how the system analysis will be conducted, how the system will be designed, and how the programming of the system will take place based on the analysis and design.

6.1 Verification and Validation Model

The V-model is used, to provide a systematic approach to system development and validation. This means that the V-model follows a sequential design process, where each phase must be completed before the next phase can begin, as shown in figure 6.1. Simultaneously, in each phase, strategies are made regarding the testing of the system. By employing the V-model, it will help prevent shortcomings and ensure quality throughout the development process. [42] The V-model is visualized in figure 6.1.

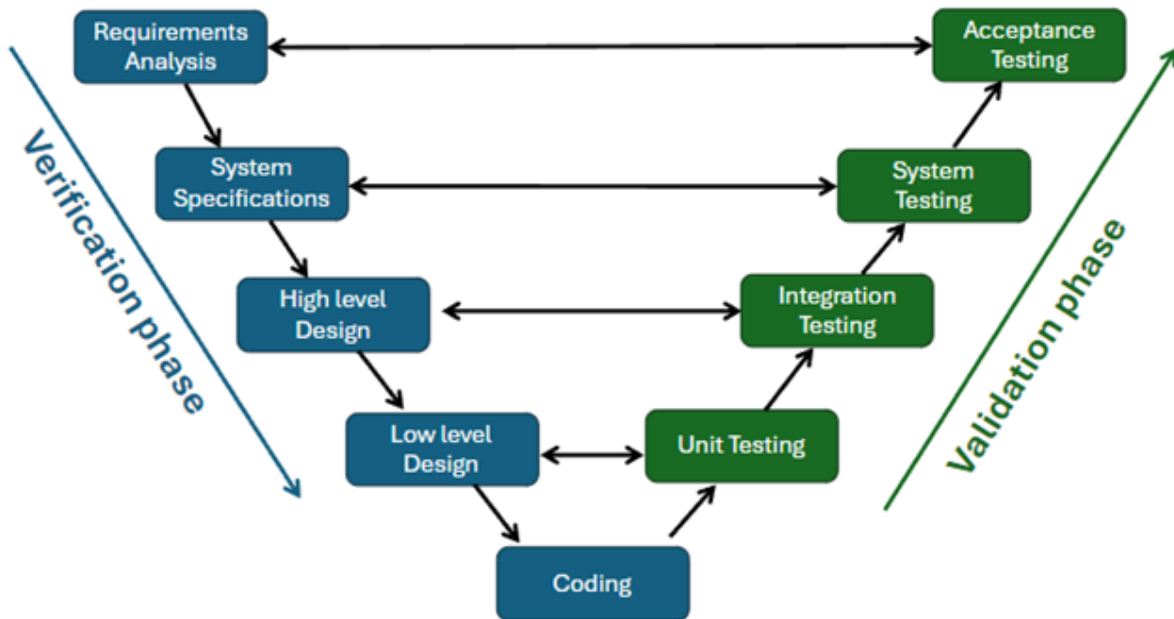


Figure 6.1: Visualizing the steps of both the verification and validation phase, modified from [43].

The initial phase of the project is the verification phase. During this phase, the system is constructed according to the specified requirements. The primary objective of the verification phase is to identify if all the requirements are met, to ensure that the system obtains the specified requirements. The first step in the verification phase is to collect the users' requirements, and from these, the system

requirements can be developed. Through Object-Oriented Analysis (OOA) the desired functionalities of the system are described, thereby establishing the structure of the system and delineating the interaction between its objects.[44] From the system requirements, a high-level design of the final system is described, to understand overall functionality. The design can then be divided into parts, described as a low-level design to get an understanding of how to develop the system. Object-Oriented Design (OOD) is applied during these phases. OOD defines object behaviors, identities, and states, alongside their relationships. By utilizing Unified Modeling Language (UML), these concepts are visually represented.[44] The UML diagrams used in this project are: Use Case, Activity-, Class-, and Entity-Relationship diagrams, which will be further elaborated in the project.

The verification phase ends before the coding phase begins which is where the system is being programmed [42]. During this phase, Object-Oriented Programming (OOP) is utilized to instantiate the classes, objects, and methods defined in the earlier phases.[44]

After the verification phase, the validation phase will follow, where testing of the requirements and the final system begins. The purpose of the validation phase is to validate whether the final system meets the requirements. The tests start with unit testing to assess individual components compared to the low-level design of the system and to eliminate errors at the coding level. After unit testing, larger functions or parts of the system are tested in integration testing. This is to ensure that the system works as it was described to do based on the high-level design. Testing the system in parts should be followed by a comprehensive system test of the program before it is finally tested by the intended user at the end by the use of acceptance testing. [42]

7. Requirements Analysis

The following chapter describes the initial phase of the V-model, the requirement analysis. Within the analysis of the solution, a definition of the framework for the system will be outlined, afterwards a use scenario is designed to illustrate how the system is intended to be used. Additionally, in connection with OOA, user requirements have been developed, leading to a use case diagram with corresponding use cases.

7.1 System definition

For this project, a software system with the name Cuff Pressure Analysis Tool (CPATool) will be developed. CPATool will be a system on a computer within an office, as seen in figure 7.1. This arrangement allows the chief physician to use the CPATool as a quality management tool. Additionally, the CPATool can be used in the daily medical practice during for example morning meetings regarding potential workflow adjustments for a specific patient with an endotracheal tube in the ICU.

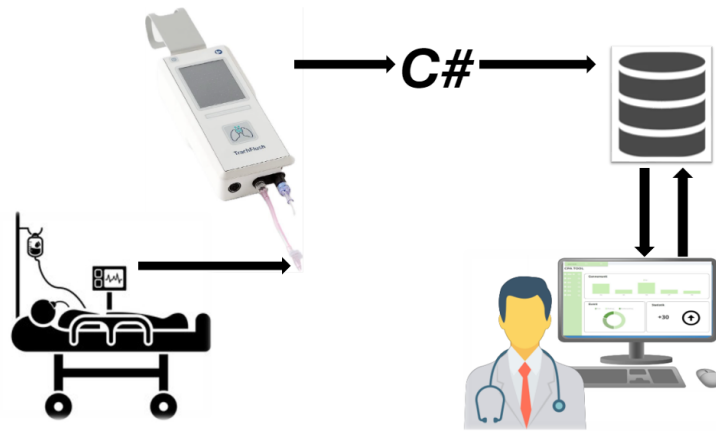


Figure 7.1: The configuration of the CPATool system and the necessary hardware. The chief physician operates the system which is connected to a database that retrieves data from one or multiple devices connected to patients. The system is also able to send values to the database.

To facilitate data recording from patients connected to a TrachFlush device, a C# code originally developed by AW Technologies has been modified. This modification includes the integration of a database connection, ensuring that data can be saved in designated tables. Furthermore, the database is linked to the CPATool system, accessible on the chief physician's office computer.

7.2 Use Scenario

The CPATool is utilized by the chief physician of the ICU. To access the CPATool, the chief physician must log in using their personal information. While monitoring intubated patients, the chief physician will use the CPATool to monitor changes in cuff pressure and airway pressure over time. As more patients are intubated, the chief physician can link additional patients to the CPATool using a Treatment ID, enabling the identification of which patient, via their CPR number, is associated with each TrachFlush device.

In this scenario, the TrachFlush device will monitor parameters from the mechanical ventilator via a respiratory circuit. As the TrachFlush device monitors the cuff pressure and airway pressure, this data will be collected and saved by the CPATool in a database, for further visualization and analysis. The purpose of the visualization is to help the user observe patterns in cuff pressure, as well as how much it has been adjusted during the period in which the TrachFlush device has been connected. The data presents the user with potential events that have occurred during the monitoring period of the cuff pressure. Irregular fluctuations in the data will typically indicate events. Examples of these events include flushes made with the TrachFlush and changes in head or body positions. Abnormal changes can be due to potential defects in the cuff.

7.3 User Requirements

In this project, all requirements are prioritized by the MoSCoW model, which is seen in table 7.1. The requirements are categorized into four levels: Must, Should, Could, and Won't have requirements. The Must-have requirements represent the highest priority, followed by Should-have, Could-have, and Won't-have. This prioritization ensures that the most important critical features and functionalities are developed first. [45]

Abbreviation	Term	Meaning
Mo	Must have	Includes all the necessary requirements for successfully completing the project.
S	Should have	Requirements that are important to the project completion, but not necessary.
Co	Could have	Desirable requirements but unimportant. Small impact on the project if left out.
W	Will not have	Requirements that are not prioritized and important in the project's time frame.

Table 7.1: The MoSCoW Model that contains the abbreviations and the descriptions of use.[45]

Based on the use scenario, the user requirements are determined accordingly, ensuring alignment with the prioritization structure established in the MoSCoW model. Additionally, the requirements are also assigned an ID, ensuring optimal traceability. The requirements are developed with input from

AW Technologies and chief physician Nilanjan Dey from Operation and Intensive, Region Hospital Gødstrup.

The user requirements are listed in table 7.2.

ID	User requirement	MoSCoW
UR1	The user must be able to graphically review cuff pressure over time from TrachFlush data.	Mo
UR2	The user must be able to graphically review airway pressure over time from TrachFlush data.	Mo
UR3	The user must be able to graphically review events that have occurred in relation to the cuff pressure over time.	Mo
UR4	The user must be able to add new patients with a TrachFlush device to the system.	Mo
UR5	The user must be able to access relevant data from TrachFlush device.	Mo
UR6	The user must be able to observe statistically relevant data, such as mean cuff pressure and the amount of events occurred across all TrachFlush devices.	Mo
UR7	The user must be able to distinguish between different patients' Trach-Flush data.	Mo
UR8	The user must be able to review the temporal nature of events.	S
UR9	The user must be able to regulate the displayed time interval for the monitored data.	S
UR10	The user must be able to detect leakage in the cuff.	Co
UR11	The user must be able to observe when an alarm from TrachFlush device has occurred.	Co
UR12	The user must be able to observe the time the TrachFlush device has been connected to the patient.	Co

Table 7.2: The table comprises user requirements, prioritized using the MoSCoW method. The ID is used to identify the requirements.

7.4 Use Case Diagram

Use cases are primarily constructed based on the use of user requirements as well as knowledge of the actor and the environment in which they work.[46] Use case modeling is used to provide detailed information about the system being developed from the user's perspective. It consists of a use case diagram, which depicts the actor's interaction, and the relationship between use cases. Use cases are more detailed descriptions of specific functionalities within the system, and provide detailed insight into the desired purpose of the system.[46]

To delineate the interaction between the system and its actor, different use cases are defined through the depiction of a use case diagram, which is illustrated in figure 7.2.

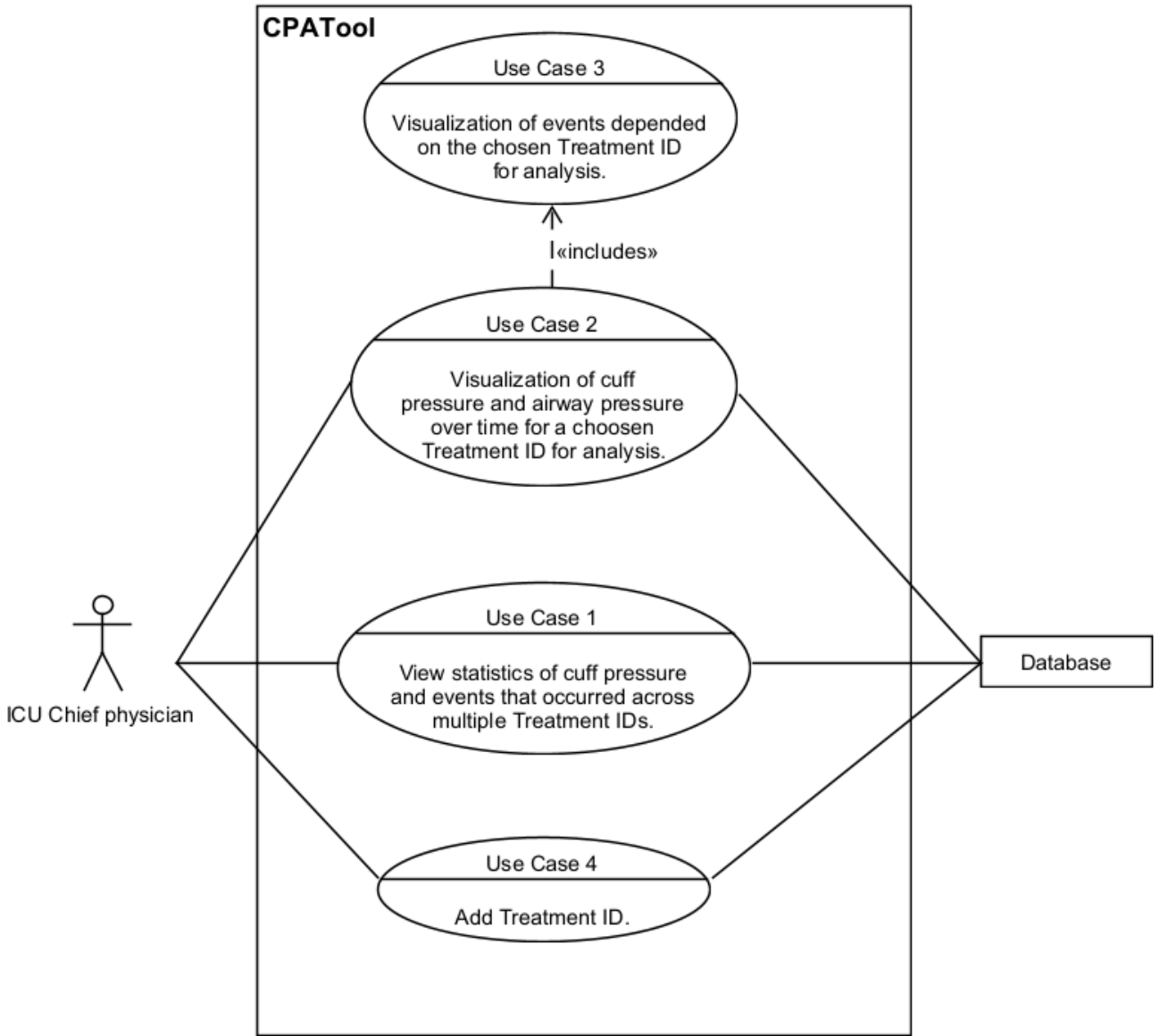


Figure 7.2: The Use Case Diagram describes the interaction between the ICU chief physician and the functions of CPATool. The Use Case Diagram is derived from the use scenario.

The use case diagram is titled the system name, *CPATool*, and contains several use cases. The actor, ICU chief physician, is connected to use case 1 which is the main use case, and to use case 2 and use case 4. The second use case is associated with the third use case through an *includes* relationship. The *includes* is represented with a dashed arrow and is used for mandatory behavior that is relevant to obtaining the main goal of the use case[47]. Three out of the four use cases are connected to a database, where the TrachFlush data is saved. In regard to UR4, a Treatment ID is established for each dataset where each Treatment ID is linked to a patient and a TrachFlush device, to ensure traceability across the datasets. The Treatment ID enables the possibility of getting a statistical overview of relevant data such as mean cuff pressure and the amount of events occurred across TrachFlush devices and across patients.

7.5 Use Cases

From the use case diagram, four use cases are derived. Each of them will be elaborated in a use case table. The use cases share the same preconditions that must be obtained for the use case to be relevant. The following preconditions apply to all use cases:

1. The user must have access to a database with data from TrachFlush devices.
2. The user must have the system open.
3. The user must log in to the system with their personal login.
4. The system automatically receives data from the database.
5. The patient must have been connected to TrachFlush device for 1 hour to ensure relevant data.

As the system requires preconditions to work, there must also be alternative courses if the preconditions are not obtained. The following alternative courses apply to all use cases.

Alternative course 1:

1. The system disconnects from the database.
2. The system sends an error message when the system is disconnected from the database.
3. The alternative course ends when the system reestablishes the connection to the database.

Alternative course 2:

1. There is no data in the database, or the dataset does not have an acceptable duration to be analyzed. An acceptable duration is defined as 1 hour.
2. A warning message to inform about this should be sent to the user.
3. The alternative course ends when there is data in the database or the dataset has an acceptable duration.

Table 7.3 represents the first use case referred to as the main use case. In this use case, the user is shown a statistics overview across multiple Treatment IDs.

Use case	Description
Name	View statistics of cuff pressure and events that occurred across multiple Treatment IDs.
ID	UC1
Purpose	The purpose is to give the actor a statistical representation across multiple treatment plans which includes data as cuff pressure and events. The statistical data is gathered from a central database.
Primary actor	ICU chief physician
Preconditions	The user must have access to a database with data from a minimum of three Treatment IDs, to ensure that the statistics are relevant.
Main course	1. The actor chooses the relevant Treatment IDs to view statistics from. 2. The system processes the data from the chosen Treatment IDs which is gathered from the central database. 3. The system calculates the statistics from the chosen Treatment IDs and visualizes it. 4. The actor has the possibility to evaluate the statistics.
Alternative course	If there are less than three Treatment IDs available the user must be informed by a warning message that the statistics do not include enough data to make a well-informed decision.
Success criteria	The system can calculate statistics based on the chosen Treatment IDs.

Table 7.3: The table represents a use case description based on the use case *View statistics of cuff pressure and events that occurred across multiple Treatment IDs* shown in use case diagram 7.2.

Table 7.4 represents the second use case. In this use case, the actor is shown a visualization of cuff pressure and airway pressure over time from a chosen Treatment ID.

Use case	Description
Name	Visualization of cuff pressure and airway pressure over time for a chosen Treatment ID for analysis.
ID	UC2
Purpose	The purpose is to give the actor a separate graphic visualization of the cuff pressure and airway pressure over time for a chosen Treatment ID.
Primary actor	ICU chief physician
Preconditions	The actor must have access to a database with data from a minimum of one Treatment ID consisting of cuff pressure and airway pressure.
Main course	1. The actor chooses which Treatment ID they want to have visualized. 2. The graphic visualization of cuff pressure and airway pressure is shown separately from the statistics. 3. The actor now has the opportunity to choose another Treatment ID for visualization of cuff pressure and airway pressure.
Alternative course	If there is no data for the chosen Treatment ID in the database, the graphic visualization is not shown. A warning message must inform the actor about the missing data.
Success criteria	A graphic visualization of the cuff pressure and airway pressure over time for the chosen Treatment ID are shown.

Table 7.4: The table represents a use case description based on the use case *Visualization of cuff pressure and airway pressure over time for a chosen Treatment ID for analysis* shown in use case diagram 7.2.

Table 7.5 represents the third use case which is *included* in the second use case. In this use case, the actor is shown a visualization of events observed for a chosen Treatment ID.

Use case	Description
Name	Visualization of events depending on the chosen Treatment ID for analysis.
ID	UC3
Purpose	The purpose is to get a visualization of the events from the chosen Treatment ID shown together with UC2.
Primary actor	ICU chief physician
Preconditions	There must be saved a minimum of one event from the chosen Treatment ID.
Main course	1. The actor chooses which Treatment ID they want to observe events from. 2. The events are shown together with visualization from UC2. 3. The actor now has the opportunity to choose another Treatment ID to observe events for.
Alternative course	If no events have occurred for the chosen Treatment ID, the events are not shown together with the visualization from UC2.
Success criteria	The program visualizes the events that have occurred for the chosen Treatment ID.

Table 7.5: The table represents a use case description based on the use case *Visualization of events depending on the chosen Treatment ID for analysis* shown in use case diagram 7.2.

Table 7.6 represents the fourth use case. In this use case, the actor has the opportunity to add a Treatment ID.

Use case	Description
Name	Add Treatment ID
ID	UC4
Purpose	The purpose is to allow the actor to add a Treatment ID, so the data from the TrachFlush device that is connected to the patient can be saved in the database.
Primary actor	ICU chief physician
Preconditions	There is a new Treatment ID to add to the program.
Main course	1. The actor has the ability to add a new Treatment ID. 2. By entering the Patient ID and the TrachFlush ID the system generates a unique Treatment ID. 3. The system sends the new Treatment ID with the belonging Patient ID and TrachFlush ID to the database. In the database, the time for the establishment of the new Treatment ID will be saved.
Alternative course	If an error occurs under adding the new Treatment ID, the actor will be informed through a warning message.
Success criteria	A new Treatment ID are added to the system.

Table 7.6: The table represents a use case description based on the use case *Add Treatment ID* shown in use case diagram 7.2.

8. European Union Requirements

The following chapter will outline the methods used in the development of a medical device. It will introduce the regulatory procedures of the European Parliament used to classify, analyze, and identify risks associated with medical devices. Additionally, the chapter will outline a specific segment of the approval process that the CPATool must undergo to gain certification as a medical device.

8.1 Medical Device Regulation

Stringent adherence to the regulatory structure delineated in the European Parliament Directive 2017-745/746, published on April 5th, 2017, is necessary when developing medical devices. The Medical Device Regulation (MDR) incorporates regulations and conditions that manufacturers must adhere to in the production of medical devices intended for use on patients within the European Union. This directive, MDR 2017/745-746, encompasses comprehensive regulations governing both general medical devices (MDR 2017/745) and in vitro diagnostic medical devices (MDR 2017/746). The primary aim of this regulation is to uphold the highest standards of medical device quality and ensure patient safety. [48]

The definition of a medical device according to MDR 2017/745 is:

‘medical device’ means any instrument, apparatus, appliance, software, implant, reagent, material or other article intended by the manufacturer to be used, alone or in combination, for human beings for one or more of the following specific medical purposes:

- *diagnosis, prevention, monitoring, prediction, prognosis, treatment or alleviation of disease,*
- *diagnosis, monitoring, treatment, alleviation of, or compensation for, an injury or disability,*
- *investigation, replacement, or modification of the anatomy or of a physiological or pathological process or state,*
- *providing information by means of in vitro examination of specimens derived from the human body, including organ, blood and tissue donations.*[48, Article 2, section 1]

This project aims to develop a software system for surveying cuff pressure in the ICU. This involves employing monitoring techniques to visualize relevant data, leading to the system being defined as a medical device. It is important to note that this definition excludes it from the in vitro diagnostic medical device category, thus subjecting it solely to compliance with the regulations outlined in MDR 2017/745.

8.2 Technical Documentation

Technical documentation refers to documents collected as part of the process to pass the conformity assessment. This assessment is necessary to obtain compliance with the MDR 2017/745-746. The developer of the medical device must provide documentation demonstrating how the development requirements for the medical device have been met. This documentation of compliance with the requirements is compiled into a technical document or technical file.[48, 49] The technical documentation may, for instance, include:

- UDI — Unique Device Identifier for identification.
- A description of the device and its intended use/user.
- The standards that will be applied.
- A description and results of the risk analysis.
- Classification of the equipment and a description of why it is classified as such.
- Information about the design and manufacture of the device.
- Verification and validation tests to demonstrate that the device meets the general requirements for safety and performance.
- Post-market surveillance (PMS) information.

The technical documentation is necessary to prove that the product meets the essential requirements, thus justifying and supporting an EU Declaration of Conformity. The EU Declaration of Conformity is a formal statement confirming the product’s compliance with the legislation, meaning a declaration asserting that the product meets the requirements to be sold on the European market. Additionally, medical devices must undergo a conformity assessment, which is a process designed to demonstrate that medical devices meet the requirements specified by the MDR 2017/745-746. [48, 49] Technical documentation will not be performed in its entirety in this project. However, some elements of the technical documentation, such as the standards that will be applied, risk classification, risk management plan, and intended use, will be addressed in the following sections.

8.3 International Organization for Standardization

In the context of ensuring compliance with the MDR, standards developed by the International Organization for Standardization (ISO) are utilized in the project. The purpose of international standards is to help maintain consistent conditions across nations. Furthermore, the purpose of developing the standards is to ensure the quality, safety, and effectiveness of products and systems. [50]

Compliance with ISO standards contributes to ensuring that medical devices meet essential safety and performance requirements and it facilitates the documentation process in accordance with the MDR.[50]

For this project, the standards ISO 13485:2016, ISO 14971:2019, and IEC 62304:2006 have been used.

ISO 13485:2016, known as Quality Management, which guarantees adherence to quality management standards specifically designed for the medical device sector. This standard outlines the requirements for medical devices to meet ISO 13485:2016 compliance.[51]

ISO 13485:2016 can help manufacturers design a Quality Management System (QMS) aimed at maintaining process efficiency. It is intended to ensure continuous improvements. The ISO standard also specifies that the quality system should be structured so that medical devices are produced under controlled conditions. The quality system helps ensure that the user requirements and expectations for the product are met. One way to implement and improve quality management is by using the method known as the Plan-Do-Check-Act cycle.[52, 51]

The Plan step involves identifying a problem, developing a strategy to solve it, and putting that strategy into action. The Do step is where the planned activities are executed and components of the strategy are implemented. In the Check step, the outcome of the plan is validated, and any areas needing improvement are identified. The final step is Act, where improvements are made to enhance performance based on the findings from the Check step. [53, 51]

The Plan step is seen in this project during the planning phase, also described as OOA and OOD. Here, the planning involves how the system should be structured based on the requirements and their analysis. A prototype illustrating how the system should look is presented, along with diagrams that form the basis for implementation. In the Do step of the method, the implementation of the requirements takes place. The Check step is observed during system testing to verify if the implementation aligns with the plan. The final step, Act, involves taking action on issues identified during the Check step. This step will be analyzed in the discussion section, where the requirements that were not implemented or failed the test will be evaluated.

In relation to managing risks within medical device development, ISO 14971:2019, titled Medical Device - Application of Risk Management to Medical Devices, is employed as a guide for documenting and maintaining a risk management process. Similarly, it aims to identify risks associated with the use of medical devices. The process outlined in ISO 14971:2019 assists manufacturers in hazard identification, risk estimation and evaluation, risk control, and assessment of control effectiveness. Therefore, ISO 14971:2019 ensures systematic risk management, addressing potential hazards throughout the device's life cycle.[54] How ISO 14971:2019 has been addressed in the project will be elaborated on later.

As a guidance document for ISO 14971:2019, ISO/TR 24971:2020 can be used to help manufacturers meet the requirements and provide guidance on how to apply ISO 14971:2019 for a wide variety of medical devices. The guidance document's purpose is to assist manufacturers in the development, implementation, and maintenance of the risk management process of a medical device.[55]

Furthermore, the international standard IEC 62304:2006, also called Medical Device Software – Software life cycle processes, published by the International Electrotechnical Commission (IEC), is used. The purpose of the standard is to ensure the establishment of guidelines and requirements for the development, maintenance, and risk management of medical software. The standard is focused on compliance with legislation, patient safety, quality assurance, and continuous improvement of software.[56]

8.4 European Conformity Marking

The Conformité Européenne (CE) mark on a medical device indicates that the medical device conforms with the current EU legislation. It also indicates that the manufacturer has developed the medical device in accordance with the ISO 13485:2016 regulation. This enables the medical device to be sold on the European market.[48, 57]

For a product to be eligible for CE marking, it must go through the process of being recognized as a medical device. Additionally, it must be accurately classified in accordance with the MDR classification standards applicable to medical devices or in vitro diagnostic medical devices. If the medical device is classified as a low-risk device, the manufacturer will have to manage the process of getting the device CE marked themselves. For devices that are at higher risk, a notified body must be involved to get approved for a CE mark. The identification number of the notified body must also be displayed alongside the CE mark.[48, 57]

Since the development of the project's system will not reach the stage of being approved for sale as a medical device, the project's system will also not reach the stage where it can be approved for a CE

mark.

8.5 Intended Purpose

In the process of developing a medical device, it is essential to define an intended purpose. This determines which regulations the system must comply with, to obtain CE marking. As described, the CE marking ensures that the system meets the EU's established requirements regarding health, safety, and environmental protection.[48]

In MDR 2017/745, the intended purpose is described as:

‘intended purpose’ means the use for which a device is intended according to the data supplied by the manufacturer on the label, in the instructions for use or in promotional or sales materials or statements and as specified by the manufacturer in the clinical evaluation;[48, Article 2, section 12].

This means that the manufacturer must declare the intended purpose which should describe the most important guidelines for the use of the system.

Intended Purpose and Indications

The Cuff Pressure Analysis Tool (CPATool) is a software designed to be used with a TrachFlush, consisting of a database integrated with a dashboard interface. Its intended purpose is to save and visualize both the cuff pressure of an endotracheal tube and the airway pressure during mechanical ventilation via endotracheal intubation, utilizing data from AW Technologies device TrachFlush.

CPATool enables the user to review TrachFlush data such as cuff pressure, airway pressure and monitors flush, alarms, and abnormal events, occurring over time. This functionality enhances the ICU's capacity for quality management of the cuff and cuff pressure. Additionally, the design of the system facilitates its integration into daily medical practice, such as discussions during morning meetings regarding potential workflow adjustments or specific actions for individual patient care necessitated by the data insights provided. CPATool also enables tracking multiple patients' progress over time and performing statistical analysis of cuff management across patients and devices, which, in conjunction with clinical evaluation studies, can emphasize the possible benefits of continuous monitoring of cuff pressure.

Intended User

The intended user of the software is a qualified trained personnel under the direction of a licensed physician. This could be an ICU chief physician. The intended field of use is the ICU, catering to critically ill adult patients, 18 years or older, requiring intubation with an endotracheal tube.

8.6 Risk Classification

The classification of medical devices is a crucial step in the regulatory process to ensure their safety and efficacy. In the EU, medical devices are classified according to the MDR 2017/745. Devices are categorized into four classes: class I devices are considered low-risk, class IIa and IIb devices are of moderate risk and class III devices pose the highest risk. The classification of software for medical use depends on the intended purpose of the software, its role in patient care, and the potential risks associated with its use. [48]

The CPATool system is designed for simultaneous use with the TrachFlush device, however, the TrachFlush device operates independently of the software as a standalone system. Consequently, the software necessitates individual classification, as outlined in MDR 745:

If the device in question is intended to be used in combination with another device, the classification rules shall apply separately to each device. Accessories for a medical device and for a product listed in Annex XVI shall be classified in their own right separately from the device with which they are used.[48, Chapter 11, section 3.2]

The CPATool is also in accordance with MDR 745 qualified as an 'Active device' due to it being a software system following:

[...] Software shall also be deemed to be an active device[48, Article 2, section 4]

Since CPATool is a software system the criteria which should be followed by the system are explicitly delineated in MDR 745:

Software intended to provide information which is used to take decisions with diagnosis or therapeutic purposes is classified as class IIa.[48, Annex VII, chapter III, section 6.3, rule 11]

The system specializes in quality management, providing information for decision-making processes. This provides an opportunity for the system to be utilized in, for example, morning meetings, where discussions can take place regarding the need for potential changes in workflows and specific patient decisions. As a result, the CPATool is classified as a class IIa device. This classification necessitates the engagement of a Notified Body for certification purposes.

As the CPATool is a software system it is required that it adheres to the Software Safety Classifications outlined following the IEC 62304:2006 standard. The classification is divided into three classes which are defined:

- Class A: No injury or damage to health is possible.

- Class B: Injury is possible, but not serious.
- Class C: Death or serious injury is possible.

[56]

In the development of medical devices, a plan is established outlining the requirements that the device must adhere to, which are determined by its classification. Documentation is then provided to demonstrate compliance with these requirements. Similarly, for the classification of software, a software development plan must be established. This plan should also outline the requirements to be met and document compliance with those requirements. Similar to the classification of medical devices, the number of requirements to be met and the amount of documentation required depends on the classification of the software. If it is classified as class A, there are not as many requirements to be met as if it is classified as class C. Likewise, this applies to documentation, as they go hand in hand, which means that not as much documentation is needed. The requirements for documentation for different classifications can be found in IEC 62304:2006.[56]

The CPATool is classified as class A, as it poses no risk of hazardous situations in case of a system error, either for the patient or the system user. This classification is based on the user's ability to resort to previously established methods in case of any system malfunction. The classification implies fewer requirements to be met and therefore less documentation needed. As a result, the system will undergo less assessment, enabling quicker market entry.

8.7 Risk Management

In order to achieve compliance with MDR 2017/745 regulation, it is essential to establish a risk management system to assess potential device-related risks. To complete this system, manufacturers must follow these steps:

Manufacturers shall establish, implement, document and maintain a risk management system. Risk management shall be understood as a continuous iterative process throughout the entire lifecycle of a device, requiring regular systematic updating. In carrying out risk management manufacturers shall:

- (a) establish and document a risk management plan for each device;*
- (b) identify and analyse the known and foreseeable hazards associated with each device;*
- (c) estimate and evaluate the risks associated with, and occurring during, the intended use and during reasonably foreseeable misuse;*
- (d) eliminate or control the risks referred to in point (c) in accordance with the requirements of Section 4;*

- (e) *evaluate the impact of information from the production phase and, in particular, from the post-market surveillance system, on hazards and the frequency of occurrence thereof, on estimates of their associated risks, as well as on the overall risk, benefit-risk ratio and risk acceptability;*
- (f) *based on the evaluation of the impact of the information*

[48, Annex 1, chapter 1, section 3]

The risk management process is developed according to the ISO 14971:2019 standard, which encompasses a comprehensive risk management plan comprising risk analysis, evaluation, control, evaluation of overall residual risk, risk management review, and production and post-production activities.[54] This plan is visualized in figure 8.1.

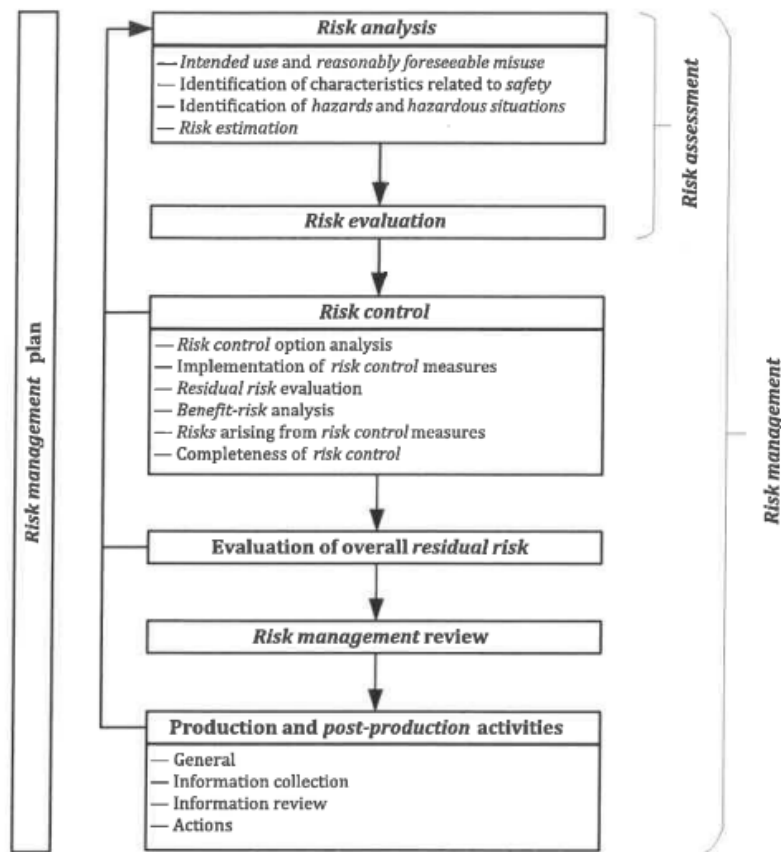


Figure 8.1: The risk management process involves risk analysis, evaluation, and control, along with evaluating the overall residual risks, risk management review, and production and post-production activities. [54]

8.7.1 Risk Analysis

The risk analysis is a part of the risk management and development process of a medical device. The following section will describe the main theories and tasks behind the risk analysis, and will further contain specific examples of the risk analysis process, done in conjunction with this project. The purpose of risk analysis is to identify hazards associated with the use of the medical device. The risk

analysis intends to minimize harm to the patient and ensure safety when using the medical device. The hazard alone cannot lead to harm but the sequence of events that lead to the hazardous situation can cause harm.[54] Figure 8.2 illustrates the risk analysis.

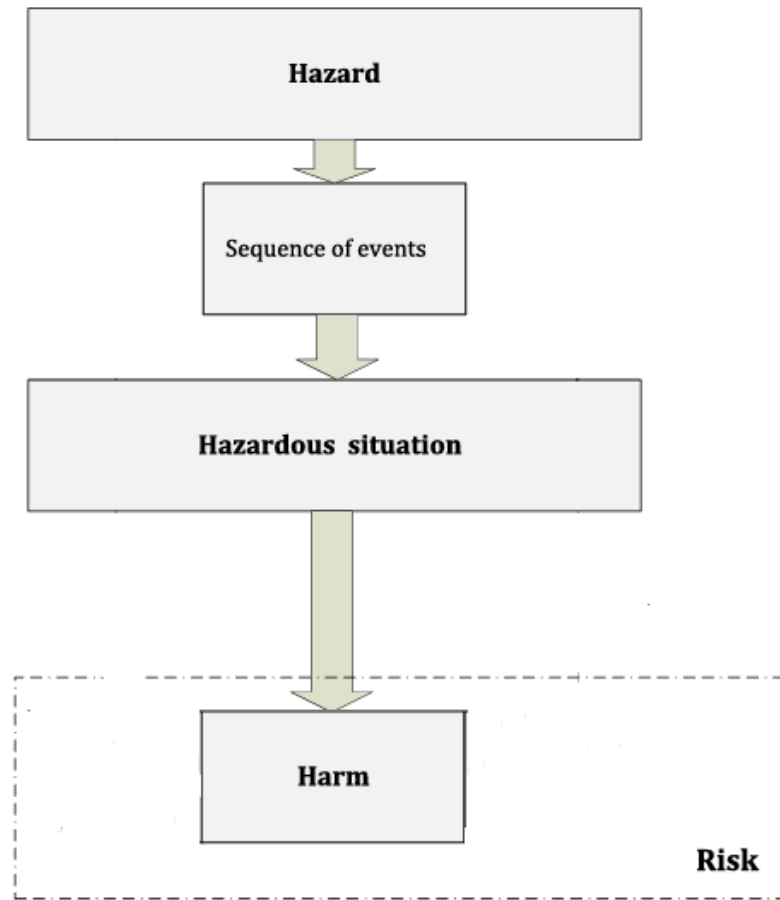


Figure 8.2: The interaction between hazard, sequence of events, hazardous situation, and harm. Modified from [54].

To support the risk analysis, different techniques to analyze possible hazards can be used. It may be necessary to use several techniques in the process of identifying hazards. Some of the techniques include the Fault Tree Analysis, the Failure Mode and Effects Analysis, the Hazard and Operability Study, or the Hazard Analysis and Critical Control Point. The technique employed to identify potential hazards associated with the use of the CPATool is the Preliminary Hazard Analysis (PHA). This technique can be used in the early development process where few design details of the medical device are known. The technique relies on current information about similar devices to make associations between the found hazards and which harms they can lead to. The purpose of PHA is to facilitate the identification of potential hazards prior to system development.[58, 55] Given that the system involves software, this approach enables reliance on already existing knowledge of potentially hazardous situations specific to software. To identify the hazards group a list of hazards from Annex C in ISO 14971:2019 is used [54].

The risk analysis includes the following parameters; ID, Hazard Group, Hazard, Cause of Hazardous Situation, Sequence of the events, Hazardous situation, and Harm. Each risk must be assigned an ID to simplify the process of tracking and ensuring that every risk has been analyzed, evaluated, and ultimately accepted. The ID also provides evidence of verification of the risks in the documentation. [54] The hazard group is defined as the identification of hazards where the hazard is the possible source of harm. The cause of a hazardous situation indicates the circumstances that form the basis of the hazard. Whereas the sequence of the events describes the progress that led to the hazard. The hazard situation is the result of the sequence of events where the harm is defined. [54] The table used in this project for the risk analysis is illustrated in table 8.1.

Risk analysis						
ID	Hazard Group	Hazard	Cause of Hazardous situation	Sequence of events	Hazard situation	Harm

Table 8.1: A template of the risk analysis table.

An example of a hazard outlined in the risk analysis table is the hazard identified as H3. This hazard is associated with limited data availability in a medical software program and, therefore belongs to the Data hazard group. The hazard stems from insufficient data for a chosen Treatment ID, specifically when less than 1 hour of data is available. This inadequate amount of data can lead to potential misinterpretation of the analysis. For example, if there is less than 1 hour of data on cuff pressure, it might be incorrectly interpreted as stable, when in fact, there could be significant fluctuations that were not captured. This misinterpretation could result in the cuff not being replaced in a timely manner, potentially overlooking a leak. The sequence of events leading to this hazard includes the user opening the program, accessing a specific tab for a chosen Treatment ID, and encountering an insufficient amount of data for analysis. This misinterpretation can result in wrongful clinical decisions, potentially leading to a deterioration of the patient's health.

All hazard found in relation to CPATool in the risk analysis is shown in figure A.2 in appendix A.

Risk Analysis						
Hazard ID	Hazard Group	Hazard	Cause of Hazardous situation	Sequence of events	Hazard situation	Harm
H3	Data	Limited data.	Not enough data available.	1. User opens the program. 2. The user accesses a tab for a chosen TreatmentID. 3. Inefficient amount of data available for the chosen TreatmentID for analysis.	Misinterpretation of the analysis.	Wrongful decisions. Deterioration of health.

Figure 8.3: The table includes the risk analysis of hazard H3.

8.7.2 Risk Evaluation

Risk evaluation is performed when the risks in the risk analysis have been identified. The purpose of risk evaluation is to evaluate the severity and the probability of the risks individually at first. This evaluation supports the decision-making process regarding risk acceptability and determines the necessity for implementing risk control. [55, 54]

In accordance with ISO/TR 24971:2020 the estimated probability and severity of harm can be divided into different levels which are decided by the manufacturer.[55] In figure 8.4a and 8.4b the levels of probability and severity of harm are listed, which are modified from ISO 14971:2019 [54]. The probability describes whether the risk will occur within 5 years of the medical device's lifespan, and the higher the level, the more likely it is that the risk will occur. The severity describes how it can affect the patient if the risk occurs.

Probability	Description
Improbable (1)	Unlikely to occur within 5 years of the medical device's lifespan.
Occasional (2)	Can occur but not frequently, it is likely to happen a few times within 5 years of the medical device's lifespan.
Frequent (3)	Likely to occur several times within 5 years of the medical device's lifespan.

(a) Probability Table

Severity	Description
Negligible (1)	Any damage to the medical equipment, excessive waiting time, and inconvenience exists.
Moderate (2)	Reversible or minor injury.
Catastrophic (3)	Loss of function or structure or death.

(b) Serverity Table

Figure 8.4: Description of the probability of harm and the severity of harm. Modified from [55].

To describe the probability and severity of the occurrence of harm, an N-by-M matrix will be used. The purpose of the matrix is to associate each possible risk found in the risk analysis with the probability and severity of the occurrence of harm. In the matrix, the Risk Priority Number (RPN) is also displayed. Each risk is given an RPN value between 1-9, with 9 being the highest. If the RPN value is greater than 7 the risk is then set to be intolerable. The RPN is calculated by multiplying the probability with the severity. Together with the probability, severity, and RPN values, the manufacturers can separate the most serious risk to establish, which risk needs to be controlled from acceptability criteria that are defined in the risk management plan, seen in appendix A. The matrix is seen in figure 8.5.[55, 59]

Probabillity of Occurrence	Serverity		
	Negligible 1	Moderate 2	Catastrophic 3
Frequent 3	3	6	9
Occasional 2	2	4	6
Improbable 1	1	2	3

Figure 8.5: Matrix of the criteria for risk acceptance. Modified from [55]

Based on Step 4 in the Risk Management Plan, seen in appendix A, the Native Risk Evaluation is developed with the defined estimation of the risk factor from figure 8.5. An example of a Native Risk Evaluation of H3 from figure 8.3 is shown in figure 8.6. Whereas the full risk evaluation is shown in figure A.2 in appendix A.

Native Risk Evaluation			
ID	Severity	Probability	RPN
H3	S2	O3	Acc(6)

Figure 8.6: The table includes the risk evaluation of hazard H3. The hazard is evaluated as acceptable with justification.

The hazard H3 exhibits severity S2, categorizing it within the moderate range, signifying a reversible or minor injury. Conversely, its probability is ranked as O3, indicating a likelihood of occurrence several times throughout the medical device’s lifetime. Consequently, the RPN is determined at level 6, falling within the range of $4 < \text{RPN} \leq 6$ in the Native Risk Evaluation, which is considered acceptable with justification.

8.7.3 Risk Control

Risk control is the process of identifying and managing risks that are deemed unacceptable by developing strategies to reduce them to an acceptable level. This involves conducting a thorough analysis of the risks at hand to determine the necessary actions and components of a solution. Subsequently, specific measures are implemented to control these risks effectively. In connection with the risk control measures of accepting risks that were not initially approved, it may be necessary to reconsider the solution, meaning making additions that will help manage some risks so that they can be accepted. This may also mean that additional system requirements need to be added to fit the new solution. This iterative process ensures that potential threats are identified, assessed, and addressed in a manner that minimizes their impact on the project. By consistently monitoring and adjusting these risk control measures, manufacturers can uphold a proactive approach to risk management, thus safeguarding against potential disruptions or losses.[54]

Residual Risk Evaluation

The implemented risk control measures undergo a verification step to assess whether they have effectively reduced the risk to an acceptable level. If the risk still surpasses the acceptability criteria outlined in the risk management plan, the manufacturer is instructed to investigate additional risk control measures. This iterative process should continue until further risk control becomes unprac-

licable, and the residual risk aligns with the acceptability criteria outlined in the risk management plan.[54]

In figure 8.7, an example of risk control of hazard H3 is shown. The hazard indicates that the user lacks sufficient data from the chosen Treatment ID for analysis. In risk control, it is necessary to implement measures to reduce the risk. In this example, risk reduction is implemented by adding a system requirement, which indicates a warning message about the time of the connection between the patient and TrachFlush. A new evaluation is prepared with a new severity and probability after the implementation of the requirement, where the RPN is determined as level 4 which is considered acceptable. Therefore, the residual risk evaluation is that the hazard is acceptable.

Risk Control					
ID	Risk Reduction measures	Severity	Probability	RPN	Residual risk evaluation
H3	Adding requiriment SR21: The system must issue a warning message if the patient has not been connected to a TrachFlush for at least one hour.	S2	O2	Acc(4)	The hazards is now acceptable.

Figure 8.7: The table includes the risk control of hazard H3. After the risk control, the hazard is evaluated as acceptable.

8.7.4 Risk Management Review

After implementation and verification of the risk of the medical device, the manufacturer has to review the execution of the risk management plan in a review which is included in the risk management file. [55] The purpose of the risk management review is to document the effectiveness and implementation of the risk management process. [55] As part of the risk management review, an overall risk table has been created to show the different hazards that could arise from the use of the CPATool. It is seen in appendix A, figure A.2.

8.7.5 Production and Post-Production Activities

Monitoring production and post-production activities is crucial for medical device manufacturers. This helps the manufacturers continuously improve risk management by collecting feedback from various sources such as users, distributors, and training personnel. Any relevant information is then used to enhance the safety of the medical device.[54, 55] Due to the limited time frame of this project, the implementation of this particular step in the risk management process will not be feasible. However, this aspect will be a relevant step to integrate upon the product market release.

Part III

Design

9. System Specification

In the following chapter the system requirements, additional requirements, and database requirements are made based on the use scenario, user requirements, and use cases, which also indicate the onset of the verification phase in the V-model.

9.1 System Requirements

For a deeper comprehension of the system requirements, an explanation of the monitoring capabilities of a TrachFlush device is provided. The TrachFlush device is able to monitor different parameters for example cuff pressure, airway pressure, alarms, and when a flush has happened. In addition, the TrachFlush device also monitors parameters from the mechanical ventilator, such as Positive End Expiratory Pressure, Respiratory Rate, and Pressure Control Ventilation. The TrachFlush device has different alarms that alert if for example, the cuff pressure is too low or high, if the cuff leaks, or if the TrachFlush device is not connected correctly.[60] Based on the user's requirements for a system that saves data from the TrachFlush device, the user aims to observe cuff pressure and airway pressure over time and when various events have occurred. Events would then include instances such as a flush being performed, if there have been noticeable changes in cuff pressure, or if the position changes have patient occurred, which has altered the cuff pressure, or cuff leakage. Events may also involve unexplained changes in cuff pressure. In the future, when referencing events, it will include the mentioned parameters.

In order to better understand TrachFlush data and the limits of the device, several pilot studies have been performed. The pilot studies are described in appendix B, and the resulting knowledge will be used to develop system requirements.

To gain a deeper understanding of how the user requirements are intended to be addressed, each requirement is described along with the corresponding system requirements as described below. Each system requirement is given an ID to identify the requirements along with the relation to the user requirement or hazard ID. The user requirements are found in 7.3 whereas hazard IDs are found in table A.2.

To obtain UR1 and UR2 the system must show the cuff pressure and the airway pressure over time in two different graphical visualizations for the chosen Treatment ID. Both UR1 and UR2 are Treatment

ID specific, and therefore it will only be shown when the user chooses a Treatment ID. The graphical visualizations must be shown in the same frame separately from the statistics across Treatment IDs, therefore there must be opened a new frame when the Treatment ID is chosen. This gives the system requirements:

SR1: The system must include a graphical visualization of the cuff pressure over the time the patient has been intubated.

SR2: The system must include a graphical visualization of the airway pressure over the time the patient has been intubated.

SR9: The system should enable the user to open a new frame with data for a selected Treatment ID.

To meet UR4, the system must incorporate a function enabling the addition of a new Treatment ID. This function should require two variables: the Patient ID and the TrachFlush ID. The purpose of this function is to generate a unique Treatment ID. The user should be prompted to input the Patient ID and the corresponding TrachFlush ID to assign a Treatment ID. Subsequently, the Patient ID, TrachFlush ID, and Treatment ID should be saved in a database. This database will facilitate future analysis of the treatment process.

SR3: The system must contain a function to create a new Treatment ID containing a Patient ID and a TrachFlush ID.

To meet UR5 the system must be able to connect to the database which consist of relevant data for UR1 to UR12 except UR5.

SR4: The system must establish a connection to the database.

SR5: The system must retrieve data from the database.

To obtain UR6, the system must graphically display cuff pressure monitored over time for multiple Treatment IDs in the same graph. This will enable the user to track how different cuff pressures have varied over time. In addition, the system must also graphically display the events that have occurred across multiple Treatment IDs. Furthermore, the system must give a graphical overview of mean cuff pressure across multiple Treatment IDs.

SR8: The system must be able to give a statistical overview of cuff pressure and events occurring across all Treatment IDs.

According to UR9, the system must allow the user to adjust the time interval in the graphical visualizations for UR1, UR2, and the statistically overview in UR6. This entails defining the range of the time interval. The time interval range should be adjustable from 2 minutes to the patient's full time of treatment where the patient has been intubated. The 2 minutes allows the user to see a minimum

of 32 breath[61] on the graph at once, which is sufficient to see if the cuff pressure behaves differently for a single breath.

SR10: The system must allow the user to adjust the time interval from 2 minutes to the full-time treatment of the graphical visualizations.

To obtain UR10, the system must have an algorithm that detects a leakage on the cuff, this algorithm must detect when the cuff pressure unexpectedly decreases and the TrachFlush device is frequently regulating the cuff pressure to the manually set cuff pressure level. This leads to the system requirement:

SR21: The system must detect continuous fluctuation in cuff pressure as cuff leakage.

To meet UR3, UR8, and UR11, the system should incorporate algorithms to identify patterns in the signal to indicate position changes and detect deviations from expected signal behavior. The events from UR3 will be categorized into two groups, alarms and events. Alarms originate from the TrachFlush device, while events are calculated in the CPATool. It will be advantageous for the system to utilize the database for saving the detected events, especially when dealing with substantial data volumes accumulated over time. By using this approach the system can minimize redundant calculations, thereby potentially reducing the system's processing time. Additionally, it must detect when a flush occurs by retrieving "FlushAvgOut" data from the database. The TrachFlush device has built-in alarms that can be retrieved from the database, and used in the program. For user clarity, the system must have the option to select and visualize these events and alarms on the graphical visualization by UR1 and UR2 from a collection of the events and alarms on the screen. This leads to the following system requirements:

SR11: The system must detect changes in body position by identifying patterns in the signal.

SR12: The system must detect flush performed by the TrachFlush device.

SR22: The system must detect abnormal fluctuation by detecting deviations from expected signal behavior.

SR15: The system must indicate graphically when an event defined in requirements SR11, SR12, SR21, and SR22 has occurred.

SR9: The system should enable the user to open a new frame with data for a selected Treatment ID.

SR17: The system must calculate when an event defined in requirements SR11, SR12, SR21, and SR22 has occurred.

SR18: The system must inform when an alarm from the TrachFlush device has occurred.

SR19: The system must give an overview of all events defined in requirements SR11, SR12, SR21, and SR22 in the specified time interval.

SR23: The system must send calculated values derived from SR17 to the database.

To obtain UR12 and UR7, information about the data that has been analyzed must be shown. This information must contain, which Patient ID, TrachFlush ID, Treatment ID, and how long the patient has been connected to the system. According to UR7, there has been implemented a Treatment ID that should ensure traceability between a TrachFlush ID and a Patient ID. This leads to the system informing the user about which TrachFlush ID and Patient ID correspond to a specific Treatment ID. The program must let the user find a patient or a TrachFlush device and thereby navigate to the Treatment ID that should be shown in the program. This gives the requirement for a frame that contains a statistical overview, and the possibility to open frames for specific Treatment ID to patient-specific analysis. The user should also be allowed to decide which treatments should be analyzed. Therefore the user must be given the availability to exclude specific Treatment IDs from the statistics. This leads to the system requirements:

SR7: The system must be divided into two dashboards to distinguish one or multiple treatments.

SR20: The system must provide information on the duration the patient's cuff has been controlled by a TrachFlush device.

SR6: The system must show the Patient ID and the TrachFlush ID when the Treatment ID is chosen.

SR13: The system must contain a search function, for TrachFlush ID, Patient ID, and Treatment ID.

SR9: The system should enable the user to open a new frame with data for a selected Treatment ID.

SR14: The system should allow the user to exclude specific Treatment IDs from the statistics.

The system requirements are listed in table 9.1.

ID	Relation ID	System Requirement	MoSCoW
SR1	UR1	The system must include a graphical visualization of the cuff pressure over the time the patient has been intubated.	Mo
SR2	UR2	The system must include a graphical visualization of the airway pressure over the time the patient has been intubated.	Mo
SR3	UR4	The system must contain a function to create a new Treatment ID containing a Patient ID and a TrachFlush ID.	Mo
SR4	UR5	The system must establish a connection to the database.	Mo
SR5	UR5	The system must retrieve data from the database.	Mo
SR6	UR7	The system must show the Patient ID and the TrachFlush ID when the Treatment ID is chosen.	Mo
SR7	UR7	The system must be divided into two dashboards to distinguish one or multiple treatments.	Mo
SR8	UR6	The system must be able to give a statistical overview of cuff pressure and events occurring across all Treatment IDs.	Mo
SR9	UR1, UR2, UR3, UR7, UR8, UR11, UR12	The system should enable the user to open a new frame with data for a selected Treatment ID.	S
SR10	UR9	The system must allow the user to adjust the time interval from 2 minutes to the full-time treatment of the graphical visualizations.	S
SR11	UR8	The system must detect changes in body position by identifying patterns in the signal.	S
SR12	UR8	The system must detect flush by the TrachFlush device.	S
SR13	UR7	The system must contain a search function, for TrachFlush ID, Patient ID, Treatment ID, and date.	S
SR14	UR7	The system should allow the user to exclude specific Treatment IDs from the statistics.	S
SR15	UR3	The system must indicate graphically when an event defined in requirements SR11, SR12, SR21, and SR22 has occurred.	S
SR16	UR3	The system must indicate graphically when an alarm has occurred.	S
SR17	UR3, UR8	The system must calculate when an event defined in requirements SR11, SR12, SR21, and SR22 has occurred.	S
SR18	UR11	The system must inform when an alarm from the TrachFlush device has occurred.	Co
SR19	UR3, UR8	The system must give an overview of all events defined in requirements SR11, SR12, SR21, and SR22 in the specified time interval.	Co
SR20	UR12	The system must provide information on the duration the patient's cuff has been controlled by a TrachFlush device.	Co
SR21	UR10	The system must detect continuous fluctuation in cuff pressure as cuff leakage.	Co
SR22	UR8	The system must detect abnormal fluctuation by detecting deviations from expected signal behavior.	Co
SR23	UR8	The system must send calculated values derived from SR17 to the database	Co
SR24	H3	The system must issue a warning message if the patient has not been connected to a TrachFlush for at least 1 hour.	Mo
SR25	H8	The system must have an automatic screen lock when the system is left unattended for 5 minutes.	Mo

Table 9.1: The table comprises system requirements for all use cases, prioritized using the MoSCoW method. The ID is used to identify the requirements along with the relation to the user requirement and hazards ID.

9.1.1 Additional System Requirements

In relation to the system requirements, additional non-user-related requirements have been developed. A detailed description of the additional requirements, ASR1 and ASR2, is provided below.

The General Data Protection Regulation (GDPR) is the EU rules for data protection and processing of personal data. The purpose of GDPR is to ensure effective and consistent protection of personal data. Data protection is divided into non-sensitive data and sensitive data. The non-sensitive data is identification information whereas sensitive is confidential information such as CPR number. The non-sensitive data can be divided into pseudonymized or anonymized.[62] Pseudonymized is defined as information that can not lead to a person without using additional data. Anonymized data ensures that data can not lead to a person and therefore, GDPR does not apply.[63] The GDPR rules are used when data can lead back to the identification of a person.[62]

To ensure compliance with GDPR, the CPATool should initiate by prompting users to log in with their personal login. Also to reinforce patient safety the data must be deleted after 5 years. This allows sufficient time for data analysis while also preventing the unnecessary storing of sensitive data. Given that the CPATool processes patient data, it is essential to implement data protection measures to adhere to GDPR, hence requirements ASR1 and ASR3:

ASR1: The system must require the user to log in with a personal login.

ASR3: The system must delete all data related to the Treatment ID after a saving period of 5 years.

To support OOM, it is necessary to utilize a programming language that is compatible with OOP. Therefore, Python is chosen for this purpose. Python is a well-known high-level programming language. It is often used for web development, software development, conducting data analysis, and data visualization. Python enables connection to database systems and can handle data and perform complex mathematics.[64] Given that the purpose of the solution is to develop a program for analyzing data collected from the TrachFlush, Python is an ideal choice as the programming language due to its robust data analysis capabilities and proficiency in executing mathematical tasks, including statistics, from which requirements ASR2 comes.

ASR2: The system must be programmed in Python.

The additional system requirements are listed in table 9.2, and are as the system requirements given an ID, to enhance the traceability between the requirements.

ID	Additional System Requirement	MoSCoW
ASR1	The system must require the user to log in with a personal login.	Mo
ASR2	The system must be programmed in Python.	Mo
ASR3	The system must delete all data related to the Treatment ID after a saving period of 5 years.	Co

Table 9.2: The table comprises requirements for all use cases, prioritized using the MoSCoW method.

9.2 Database Requirements

A database-management system (DBMS) main function is to provide a way to store and retrieve data from a database. Data management further consists of the definition of structures for storage, mechanisms for the manipulation of data within the database, as well as ensuring safety of the stored data. [65] The DBMS is used as a guideline in the preparation of database requirements as DBMS describes the functionalities, limitations, and performance characteristics of the database.

The database requirements provide the basis for the design of the database. Furthermore, the database requirements describe what is needed from the database to meet the system's requirements.

According to ASR1, the database must contain login information for personal logins, such as username and password, which leads to the requirement:

DBR11: The database must store personal logins which contain username and password.

To meet ASR3 the system must be able to contain data for a period of 5 years and delete data older than it.

DBR17: The database must be able to save data for 5 years.

DBR15: The database must delete TrachFlush data after the data has been saved for 5 years.

According to the system requirements SR1, SR2, SR11, SR12, SR16 and SR18 the system needs to be able to save data from the TrachFlush device, this means that the database must contain cuff pressure data, airway pressure data, "FlushAvgOut", alarms, system code, and timestamps for all the collected data.

DBR1: The database must save cuff pressure data.

DBR2: The database must save airway pressure data.

DBR3: The database must save timestamps for all collected TrachFlush data.

DBR9: The database must save alarms from TrachFlush.

DBR8: The database must save system codes from TrachFlush.

DBR10: The database must save "FlushAvgOut" from the TrachFlush.

DBR13: The database must save detected flush performed by the TrachFlush device for a specific Treatment ID.

To ensure compliance with requirements SR10, SR11, SR17, SR19, SR21 and SR22, it will be advantageous for the system to utilize the database for saving detected events, especially when dealing with substantial data volumes accumulated over time. This can minimize redundant calculations, thereby reducing processing time. Therefore, the database must be able to save detected leakage, changes in body position, flush, and abnormal fluctuation from CPATool.

DBR12: The database must save detected changes in body position for a specific Treatment ID.

DBR13: The database must save detected flush performed by the TrachFlush device for a specific Treatment ID.

DBR14: The database must save detected abnormal fluctuation for a specific Treatment ID.

DBR16: The database must save detected leakage of the cuff for a specific Treatment ID.

To support SR3, SR6, SR8, and SR13, the created Treatment ID with their belonging Patient ID, TrachFlush ID, and time of assignment must be saved in the database.

DBR4: The database must save TrachFlush ID.

DBR5: The database must save Patient ID.

DBR6: The database must save Treatment ID.

DBR7: The database must save time when a Treatment ID is assigned.

The database requirements are listed in table 9.3. The database requirements are also given an ID to identify the requirements along with the relation to the system- or additional system requirements.

ID	Relation ID	Database Requirement	MoSCoW
DBR1	SR1	The database must save cuff pressure data.	Mo
DBR2	SR2	The database must save airway pressure data.	Mo
DBR3	SR1, SR2, SR12	The database must save timestamps for all collected TrachFlush data.	Mo
DBR4	SR3, SR6, SR8, SR13	The database must save TrachFlush ID.	Mo
DBR5	SR3, SR6	The database must save Patient ID.	Mo
DBR6	SR3	The database must save Treatment ID.	Mo
DBR7	SR3	The database must save time when a Treatment ID is assigned.	Mo
DBR8	SR12	The database must save system codes from TrachFlush.	S
DBR9	SR16, SR18	The database must save alarms from TrachFlush.	S
DBR10	SR12	The database must save "FlushAvgOut" from the TrachFlush.	S
DBR11	ASR1	The database must store personal logins which contain username and password.	S
DBR12	SR11	The database must save detected changes in body position for a specific Treatment ID.	S
DBR13	SR12, SR17	The database must save detected flush performed by the TrachFlush device for a specific Treatment ID.	S
DBR14	SR22	The database must save detected abnormal fluctuation for a specific Treatment ID.	Co
DBR15	ASR3	The database must delete TrachFlush data after the data has been saved for 5 years.	Co
DBR16	SR19, SR21	The database must save detected leakage of the cuff for a specific Treatment ID.	Co
DBR17	ASR3	The database must be able to save data for 5 years.	Co

Table 9.3: The table comprises database requirements for all use cases, prioritized using the MoSCoW method. The ID is used to identify the requirements along with the relation to the user requirement.

9.2.1 Additional Database Requirements

In relation to the database requirements, additional non-user-related requirements have been developed.

To obtain requirement ADBR1, the system must send data to the database every 500 ms. This requirement is based on the following considerations: A patient's respiratory rate is typically 12 to 16 breaths/minute[61]. According to Nyquist sampling theorem, the sampling frequency must be at least double the highest frequency component in the signal[66]. For cuff pressure, the highest significant frequency component corresponds to the respiratory rate. Thus, the minimum sampling frequency should be 32 samples/minute, which is approximately every 2 seconds. However, by sampling every 500 ms, it is possible to obtain approximately 5 samples during inhalation and 5 samples during exhalation. This frequency is sufficient to monitor adjustments in cuff pressure and meets the Nyquist sampling criterion.

ADBR1: The database must receive data from the TrachFlush device every 500 ms.

The additional database requirements are listed, with an associated ID, in table 9.4.

ID	Additional Database Requirement	MoSCoW
ADBR1	The database must receive data from the TrachFlush device every 500 ms.	S

Table 9.4: The table comprises additional database requirements for all use cases, prioritized using the MoSCoW method.

10. High Level Design

In the following chapter, the high level design will be represented where the overall system is described. The following section will describe the development of the prototype and activity diagram as part of OOD process.

10.1 Prototype

A prototype is a working model for developing and testing design ideas. In this project, the presented prototype for the Graphical User Interface (GUI) is used to examine content, interaction, and aesthetics, from the perspectives of the intended users. Prototypes can range from low fidelity to high fidelity, the more the prototype is similar to the final product, the higher the fidelity. The media of the prototype evolves throughout the process of the project. In this project, prototypes are not categorized strictly into high or low fidelity but are approached as part of an iterative process. [67]

The prototypes have been developed based on the use scenario and system requirements. It includes an overview of the CPATool system which consists of a login page along with two different dashboards created for use on a computer. The following subsection shows the digital mock-ups of the system.

10.1.1 Mock-Up

Login

When the CPATool is opened, the user is presented with a login page, which is shown in figure 10.1. Here, they have the option to enter their username and password followed by a login button, which directs them to the initial dashboard. The system will be connected to a database, facilitating the user's utilization of their personal login information.

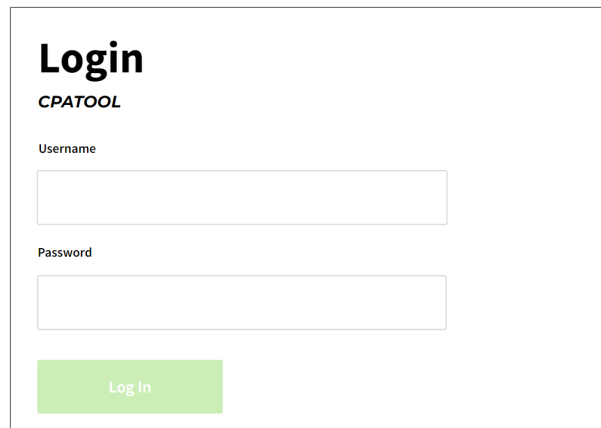
The image shows a login form for a web application. At the top, the word "Login" is written in a large, bold, black font. Below it, the text "CPATOOL" is displayed in a smaller, bold, black font. The form contains two input fields: the first is labeled "Username" and the second is labeled "Password". Both labels are in a small, black font. Below the password field, there is a green rectangular button with the text "Log In" in white. The entire form is enclosed in a thin black border.

Figure 10.1: The figure illustrates the login page of the CPATool, allowing users to input their credentials for access to the initial dashboard.

Dashboard 1

After the login the user is directed to the initial dashboard, seen in figure 10.2, where the user gains an overview of the selected Treatment IDs. To the left side, there is a list where the user can choose to view all Treatment IDs collectively or individually. The search box at the top of the list allows the user to search after a Patient ID, TrachFlush ID, Treatment ID, or date. At the bottom of the list the button 'Add New Treatment ID' enables the user to create a new connection between a patient and a TrachFlush device. Choosing a Treatment ID opens an additional tab at the top of the screen, providing more detailed descriptions about that specific Treatment ID. Conversely, opting to view all Treatment IDs displays this screen, presenting various statistics such as the average cuff pressure for all IDs in a bar chart. Additionally, a pie chart illustrates the percentage distribution of event occurrences, while a scatter plot with lines depicts the cuff pressure across all Treatment IDs. Here the user has the option to adjust the time to their preference by utilizing the date and time picker placed above the graphs.

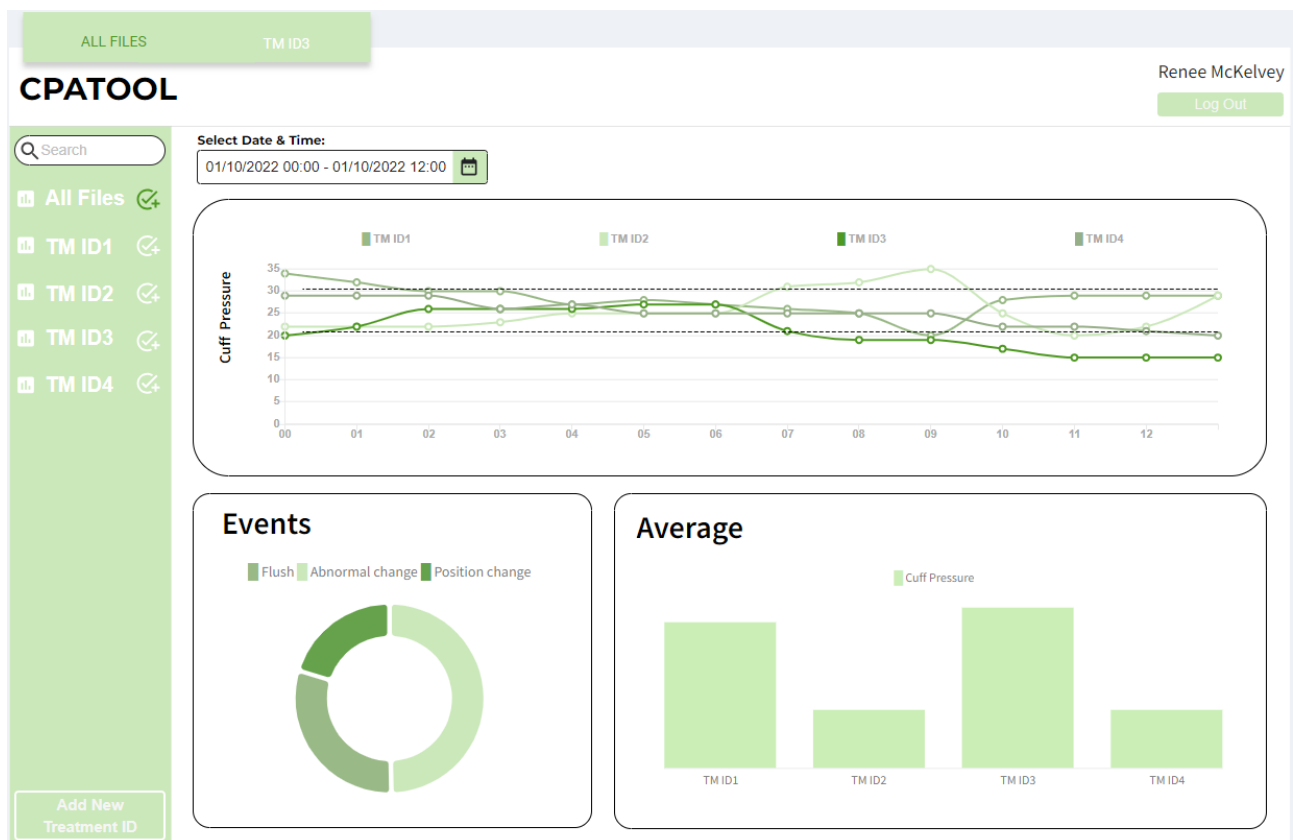


Figure 10.2: The figure shows the initial dashboard which offers features for collective viewing, search functionalities, and data visualization.

Calendar

In order to observe data over a specific time period, users can utilize the date and time function, seen in figure 10.3. This action initiates access to a calendar interface, facilitating user selection of both date and time. Consequently, this process enables real-time updating of the data displayed on the dashboards according to the user's preferences.

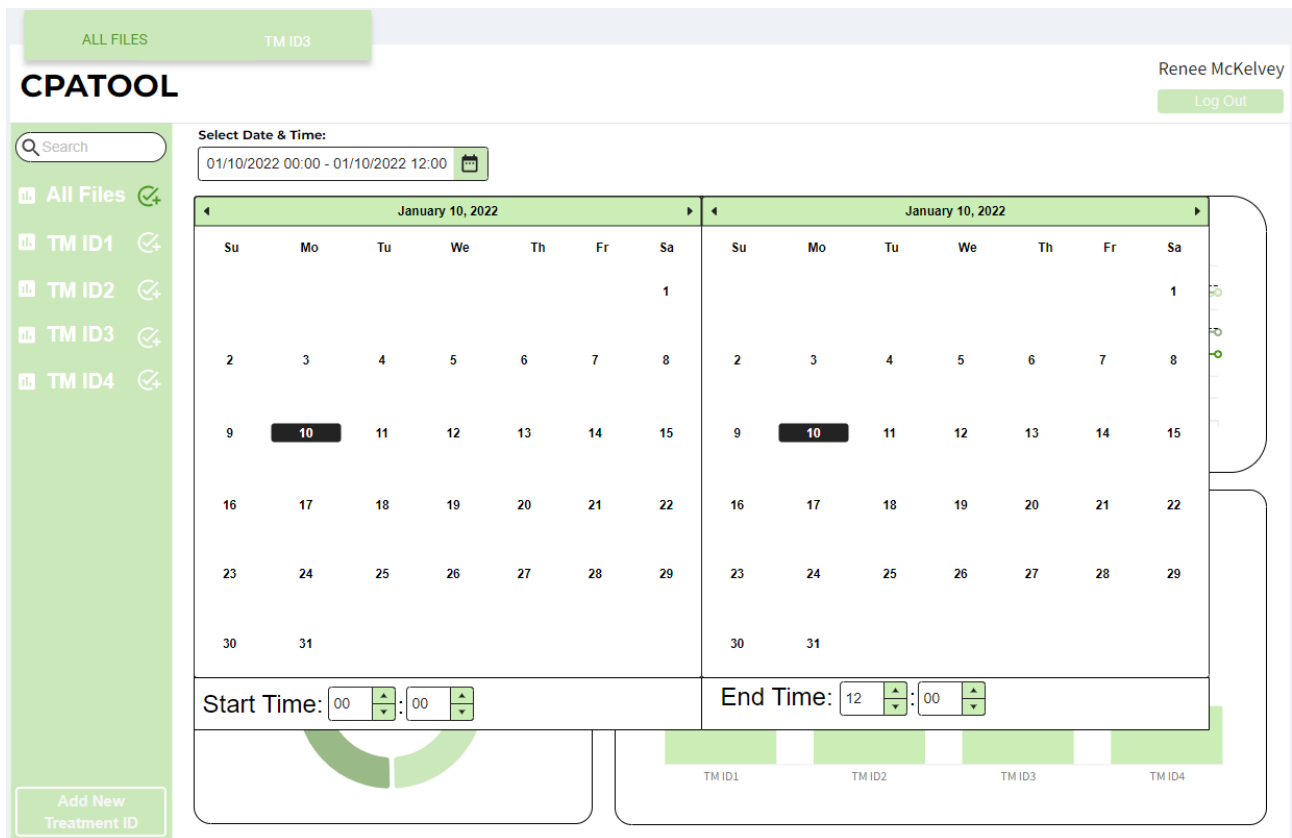


Figure 10.3: The figure shows the date and time function to observe and update data on the dashboards.

Dashboard 2

If the user prefers to inspect a single Treatment ID, the second dashboard is accessed, which is seen in figure 10.4. Here, a list on the left enables the user to switch between Treatment IDs. The dashboard provides a more detailed description, with a bar at the top presenting general information about the Treatment ID, including Patient ID and duration of intubation with a TrachFlush connected. Two graphs are displayed, one depicting cuff pressure over time and the other showing airway pressure over time. Users can adjust the time period for both simultaneously by accessing the date and time function.

On the cuff pressure graph, upper and lower limits are indicated, offering a visual indication of whether the patient's values fall within the ideal range between 20-30 cm H_2O . On the right side, two lists are presented. The top one indicates the times at which events were detected, along with the type of events. Users can select events of interest, which are then marked on the cuff pressure graph for visual representation.

The second list displays alarms from the TrachFlush device and the start time. Users can also select alarms of interest, which are visually indicated on the cuff pressure graph with two red dashed lines. The first line represents the start time of the alarm, and the second line indicates the end time.

To return to the first dashboard, the user can press 'All Files' at the top. If the user wishes to log out, they can press the 'Log Out' button located at the top right corner. This button is accessible on both the first and second dashboards.



Figure 10.4: The figure shows the second dashboard for a detailed Treatment ID description, featuring graphs of cuff pressure and airway pressure over time, event and alarm markers, and navigation options.

Treatment ID

If the user wishes to add a new Treatment ID, they can utilize the 'Add New Treatment ID' button. This action triggers a pop-up window, as seen in figure 10.5 where the user is prompted to input the Patient ID and TrachFlush ID. Subsequently, the system generates a Treatment ID linked to these inputs. The user can then save by pressing the 'Add Treatment ID' button. In case the user wishes to cancel the action, they have the option to press the close button located in the top right corner, which redirects them to the previous dashboard.

The screenshot shows the CPATool application. At the top, there's a navigation bar with 'ALL FILES' and 'TM ID3' tabs. The user 'Renee McKelvey' is logged in, with a 'Log Out' button. A sidebar on the left lists 'All Files', 'TM ID1', 'TM ID2', 'TM ID3', and 'TM ID4'. The main content area displays a modal window titled 'New Treatment ID'. This modal has three input fields: 'Patient ID', 'TrachFlush ID', and 'Treatment ID'. Below these fields is a green button labeled 'Add Treatment ID'. The modal also has a close button (X) in the top right corner.

Figure 10.5: The 'Add New Treatment ID' button prompts user input for patient and TrachFlush IDs, allowing creation and saving of a new Treatment ID.

10.2 Activity Diagram

An activity diagram is used in OOA and shows the system's flow with different activities. In an activity diagram, specific symbols are used to signify different functions. The purpose of the activity diagram is to give an overview of the system's workflow. The activity diagram is illustrated in figure 10.6. In the activity diagram, the interaction between the database and the CPATool is visualized as swimlanes, indicating what part is responsible for each activity.

The start of the activity diagram is visualized with a small filled circle with an arrow pointing to the next state. The action state is represented by a rectangle with rounded corners. the diamond illustrates a decision with paths to different activities where paths are labeled with a condition. The condition should be true before moving to the next activity. Synchronization is used to split one incoming into multiple concurrent flows or multiple concurrent into one outgoing flow which is visualized with a thicker line. Sent and received signals illustrate activities modified from outside the system. The arrows between the different symbols indicate the flow of the activity diagram. The end of a specific process flow is visualized as a circle with a cross. [68]

The activity diagram starts with the user logging in to CPATool with personal information. The login process ensures that only approved users have access to the system, and the personal information

within it.

After logging in the database sends data to the CPATool. If the condition *[if calculation events are in the database]* is obtained, the CPATool receives event data from the database where the activity *view statistics of all Treatment IDs* will occur. This ensures that event data that already has been calculated is not recalculated, each time the user opens the CPATool. At the other path *[if 1 hour data is available]* events will be calculated. As mentioned, this will be the only time events for this specific dataset will be calculated. Lastly, the path *textit[else less than 1 hour data]* is accessed if the database contains less than 1 hour of data for the selected TrachFlush device. This has been implemented to ensure, a sufficient amount of data is available before initiating monitoring and analysis. The synchronization illustrates that after calculating events two flows occur simultaneously which is *send to Database*, in which the calculated events are sent to the database and *view statistics of all Treatment IDs*. Following the flow from *view statistics of all Treatment IDs*. if the condition *[if viewing data for a specific Treatment ID]*, the user can choose a Treatment ID, open a new tab, and view a graphical visualization of the chosen Treatment ID. If the other condition *[if viewing data for multiple specific TreatmentIDs]*, the user can *deselect Treatment ID* and get a *new overview of statistics*, making the statistical overview customizable for the user. On the other path *[else]*, the user can close CPATool.

After the new overview without the deselected Treatment ID, the condition *[if viewing data for a specific treatment ID]*, the user can, once again, choose a Treatment ID, open a new tab, and view a graphical visualization of the chosen Treatment ID. On the other path *[else]*, the user can close the CPATool.

After the graphical visualization, if the condition *[if viewing data for another Treatment ID]*, the user can choose another Treatment ID for visualization. If the other condition *[if viewing statistics overview for all Treatment ID]*, the user can view the statistics overview for all Treatment IDs again. On the other path *[else]*, the user can close CPATool. Additionally, the functionality of adding a new Treatment ID is always available to the user. Furthermore, the CPATool automatically logs out if the user is inactive for more than 5 minutes, to ensure protection of sensitive data.

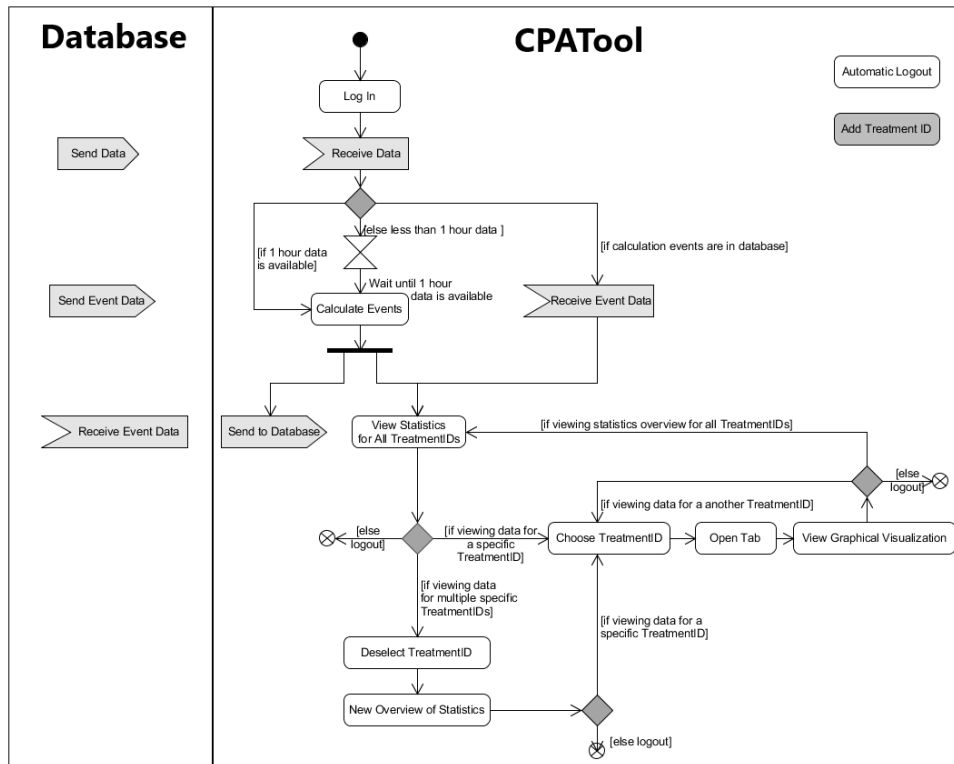


Figure 10.6: The activity diagram developed to CPATool.

11. Low Level Design

In the following chapter, the low level design process will be represented where the high level design is broken down into detailed components and algorithms. In this, the class diagram and entity relationship diagram will be developed as part of OOD process. Subsequently, the design of algorithms will be described.

11.1 Class Diagram

The class diagram is one of the fundamental models in OOD. It models the static design of a system. The class diagram is often used to get an understanding of the parts of the software, for example, what the user interface will consist of. It is used to model the objects the system will consist of, what the objects can do and to describe the relation between the objects.[69][70, Chapter 9]

The class diagram has been designed through the use of the Model-View-Controller (MVC) approach, an architectural pattern, in which an application is separated into three logical components. [71] The *Model* component is responsible for all data-related logic in the application, for example, the data being transferred between the View and Controller components. The *View* component is responsible for all User Interface (UI) logic. This includes elements such as drop-down menus, buttons, text fields, and more. Lastly, the *Controller* component is responsible for the interaction between the Model and View components. It handles processes such as incoming requests and corresponding actions or manipulation of data from the Model component. [71]

The essence of the MVC pattern is that the different classes work together as follows:

- The user interacts with the system through the view class interface.
- The input from the View class is sent to the Controller class, where it is processed.
- The processed input is then used to update the Model class, and subsequently, the View class is updated to reflect these changes.[72]

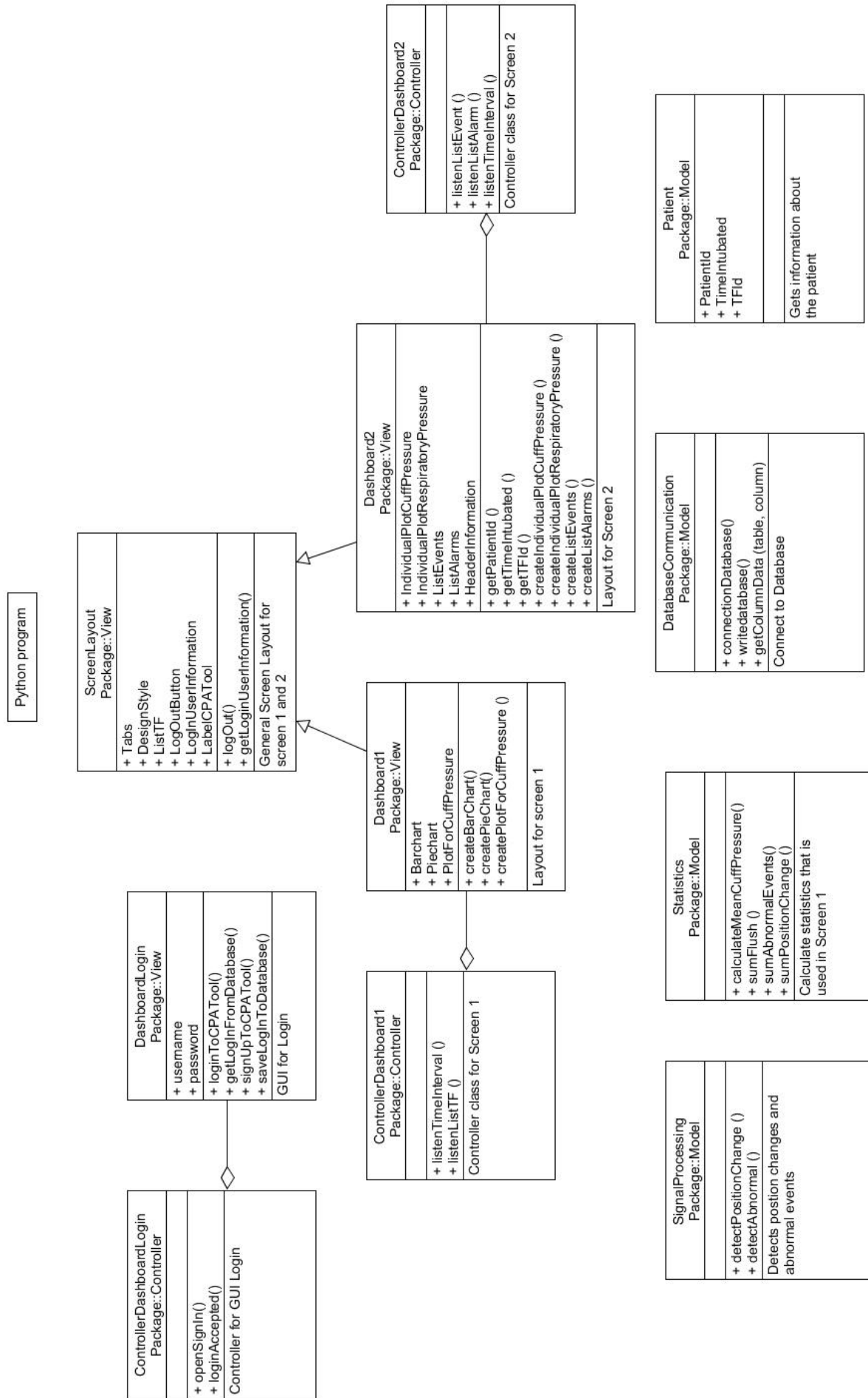


Figure 11.1: The class diagram developed for CPATool.

In figure 11.1 the class diagram designed for this project is shown. The *main* class is not included in the class diagram, as its only function is to run the code and it does not contain any additional information. The relationship between the shown classes is visualized through connections with various symbols. A connection with an unfilled arrow tip symbolizes *inheritance*. [70, Chapter 9][69] Further, a connection with an unfilled diamond-shaped tip symbolizes aggregation between classes and means the classes can exist independently of each other.

The main GUI components of the class diagram are the Controller and View classes for respectively the *DashboardLogin*, *Dashboard1*, and *Dashboard2*. These three classes make up the three main screens as seen in the prototype, see section 10.1, figure 10.1, 10.2 and 10.4. Each View and Controller class of the same screen is connected by the aggregation arrow. Furthermore, Dashboards 1 and 2 both inherit from the *ScreenLayout* class. The screen layout consists of the main elements visible in the system, while Dashboard 1 and 2, essentially acts as a second screen within the layout. That way the repeating features such as the general design style, tabs, and logout button, won't have to be implemented more than once.

The Model classes at the bottom of the diagram all contain functions used to provide data to the View class. Firstly the class *DatabaseCommunication* is responsible for sending and retrieving data to and from the database. The *Patient* class ensures that information about the patient, such as time of intubation and connected TrachFlush device, is retrievable in the system. Lastly, the two classes *SignalProcessing* and *Statistics* both are responsible for processing data and calculating all that is needed to visualize cuff pressure and more, on the dashboards.

11.2 Entity Relationship Diagram

When defining the structure of a database, a *data model* is typically used. A data model is a collection of tools used to describe data, data relationships, data semantics, and consistency constraints. [65]

In order to design and implement the database the Entity-Relationship (ER) data model is used to identify entities and the relations between them. When creating a database it is important to consider elements such as the design of the database schema, design of the programs that access and update the data, as well as necessary security measures to control access to the data. [65]

A series of design phases can be implemented to assist in the creation of the database design. First and foremost, one should characterize the data needed in the database. This will typically be done based on user needs and will result in several user requirements, as presented in table 9.3.

These requirements will be converted into a database schema based on the concepts of the ER model.

In order to fully conceptualize the database schema, one must take into account, the functional needs of the database. It should be established what functions the user will be able to implement when using the database. This could be functions such as updating or removing data, or searching for and retrieving specific data. Lastly, the database design will be implemented in the database system of one's choosing, which in this project is PhpMyAdmin. [65, p. 259-261]

The following ER diagram has been designed using the *Crow's Foot notation*. This notation technique represents entities by rectangles containing both the name of the entity and its attributes. Furthermore, Crow's notation uses distinct notations for the representation of cardinality. Relationships between entities have two indicators, one which describes the maximum number of times a relationship can exist, commonly referred to as *multiplicity*, and one which describes the minimum number of times a relationship can exist. The vertical line represents "one" and the three prongs, or crow's foot, represent "many".

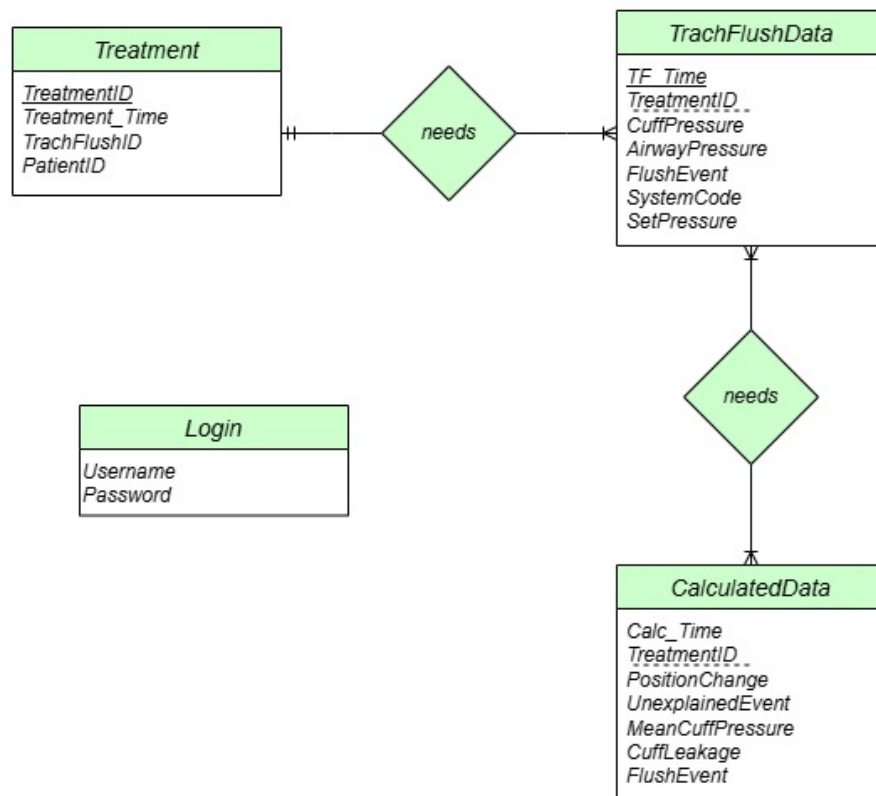


Figure 11.2: The ER-diagram for the database designed for the CPATool.

Figure 11.2 shows the database design used in conjunction with the CPATool. It consists of four entities, three of which are in relation to each other. The lone entity, *Login*, contains the attributes *Username* and *Password*, and is used to store data needed to ensure the functionality of the login page, see section 10.1.1.

The three related entities, *Treatment*, *TrachFlushData* and *CalculatedData* all consist of data used in

the CPATool. The entity *Treatment* contains the attributes *TreatmentID*, *Time*, *TrachFlushID* and *PatientID*. *TrachFlushID* and *PatientID* are both IDs used to identify devices and patients. Every unique combination of device and patient will be allocated a unique Treatment ID, which in turn will receive a timestamp, *Time*, indicating when the TrachFlush device has been connected. This ensures that no potential duplicates will occur in the database, in the event that a returning patient is paired with the same TrachFlush device multiple times.

For every unique Treatment ID, a corresponding *TrachFlushData* and *CalculatedData* table will be instantiated. *TrachFlushData* contains the attributes *Time*, *TreatmentID*, *CuffPressure*, *AirwayPressure*, *FlushEvent* and *SystemCode*. *CuffPressure*, *AirwayPressure*, and *FlushEvent* are all measured data from the TrachFlush device. All the data from the device will be stored in the database, along with a timestamp which will make it possible to visualize the data across a timeline, when programming the software. The attribute *SystemCode* contains the system code ID related to different types of alarms from the TrachFlush device. System codes will be utilized, along with the aforementioned data from the TrachFlush device, to indicate when an alarm has been activated.

The weak entity *CalculatedData* contains the attributes *Time*, *TreatmentID*, *PositionChange*, *UnexplainedEvent*, *MeanCuffPressure*, *CuffLeakage* and *FlushEvent*. Aside from *Time* and *TreatmentID*, all these attributes are calculated in the CPATool, using data from the 'TrachFlushData' table, and sent back to the database for storage, in order to eliminate the need for multiple calculations across the same data points. The calculated data is used to visualize events, both on the timeline for a single TrachFlush device, as well as a summation of all events across multiple devices.

11.3 Design of Algorithms

To provide the database with information about flush, cuff leakage, and position changes, the CPATool must contain algorithms to detect these events. These algorithms require data saved in the database to function as expected. The pilot studies in appendix B are used to develop the algorithms, which are based on the plot of cuff pressure over time. As described in appendix B, a C# code is modified and used for the recording of the pilot studies, where relevant parameters for developing the algorithms are saved in the database.

Baseline

Before developing the algorithms, the baseline test, see appendix B.1, is analyzed. In the baseline test, it was observed that the cuff pressure varied from 24 cm H_2O to 26 cm H_2O when the manually set cuff pressure was 25 cm H_2O which is visualized in figure 11.3. The baseline test forms the basis for how the algorithms for detection are developed because the baseline test represents the test without

external factors' influence. The baseline test is used as a starting point for comparison with other tests which are influenced by external factors. Based on the baseline test, the effect of the external factors can be accurately measured and assessed.

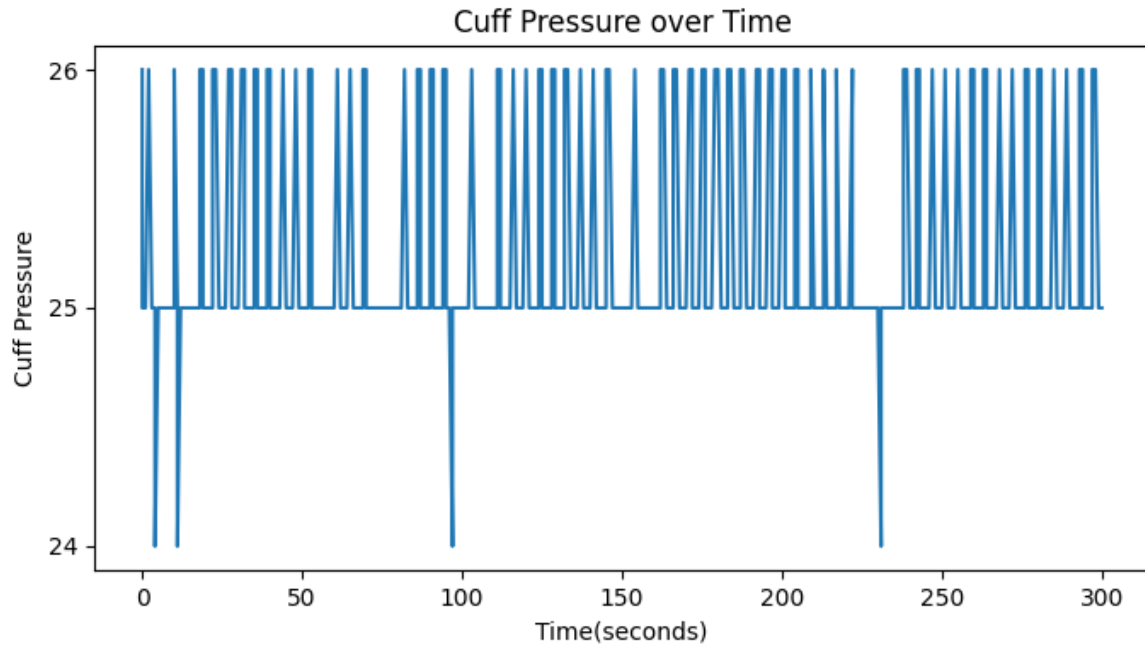


Figure 11.3: A line plot that visualizes the cuff pressure over time in seconds, during a baseline test. The manually set cuff pressure is 25 cm H_2O .

Flush Algorithm

The results obtained from testing the flush function on the TrachFlush device, see test in appendix B.2, revealed that the "FlushAvgOut" parameter changes every time the flush function was executed. The algorithm will be developed to detect and return when the "FlushAvgOut" parameter is changed and the corresponding time. The corresponding time can be used in the implementation to visualize when a flush has been executed.

Position Change Algorithm

To develop the algorithm for position change, the cuff pressure over time from the pressure test in appendix B.3 is visualized in figure 11.4.

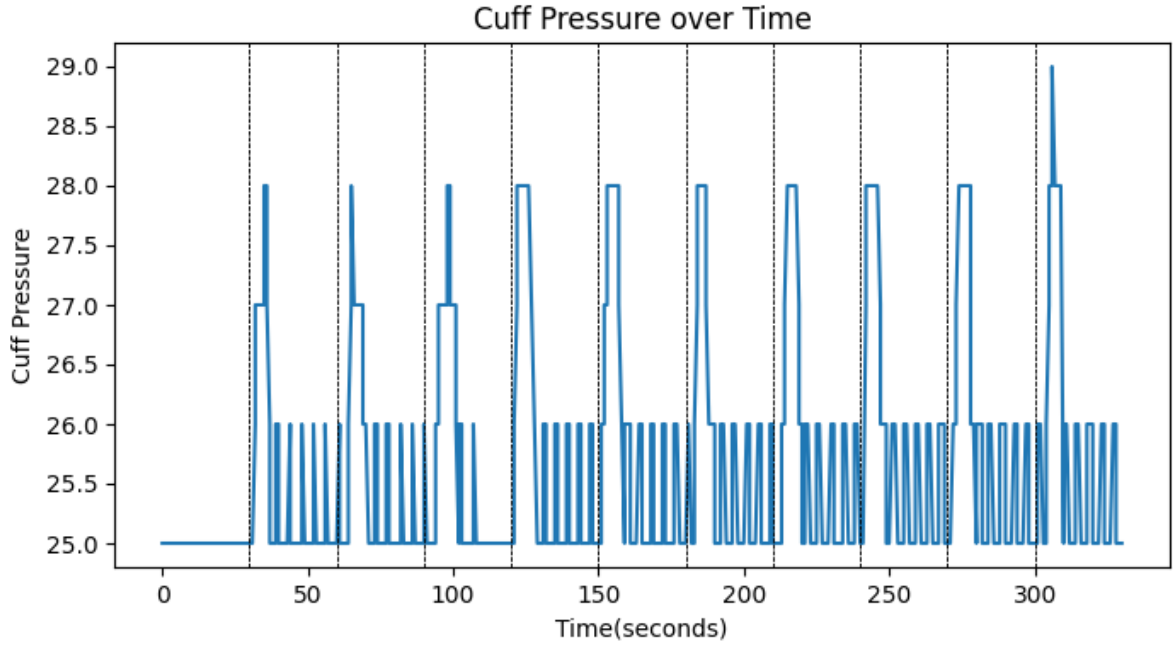


Figure 11.4: A line plot that visualizes the cuff pressure over time during a pressure test. The manually set cuff pressure is 25 cm H_2O .

In figure 11.4, the cuff pressure increases when pressure is applied to the simulated trachea every 30 seconds for 5 seconds. The algorithm must utilize this information to detect the fluctuations connected to position changes. As described in section 3.2.2, the cuff pressure increases during a position change. The purpose of the pilot study in appendix B.3 is to simulate an increase in the cuff pressure based on the data.

The algorithm is constructed by retrieving the manually set cuff pressure for the specific Treatment ID which is being collected and stored in an array list. Additionally, the cuff pressure for the specific Treatment ID is stored in another array list.

As described in the baseline test with a manually set cuff pressure of 25 H_2O , the cuff pressure varies from 24 cm H_2O to 26 cm H_2O when the recording is not affected which is visualized in figure 11.3. Based on the baseline test, the algorithm is developed to detect when the change is more than 1 cm H_2O over the manually set cuff pressure. To emphasize a position change, the algorithm must compare the monitored cuff pressure with the manually set cuff pressure at the corresponding time. If the difference is more than 1 cm H_2O , the algorithm must return an over-detection which represents a position change. The position changes are detected with a corresponding time which can be used to visualize when a position change is executed.

Cuff leakage algorithm

To develop the algorithm for leakage detection, the leakage test in appendix B.4 is used. In figure 11.5

the cuff pressure over time from the leakage test is visualized.

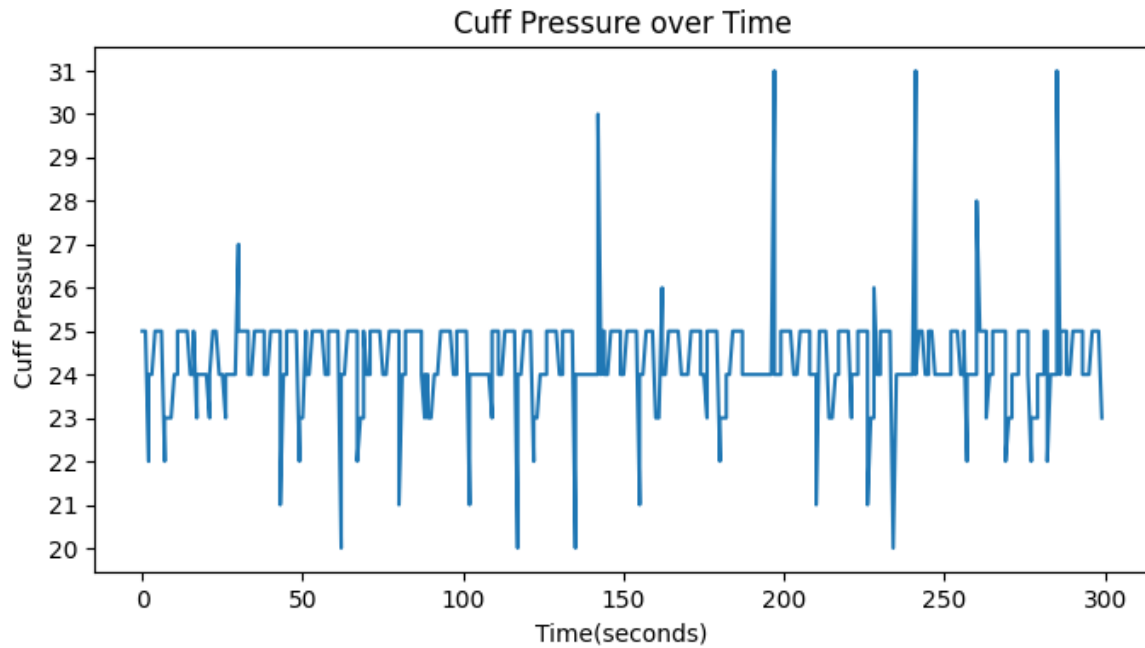


Figure 11.5: A line plot visualizing the cuff pressure over time during a test of leakage. The manually set cuff pressure is 25 cm H_2O .

In figure 11.5, the cuff pressure is primarily below the manually set cuff pressure compared with the baseline test in figure 11.3. In the leakage test described in appendix B.4, a puncture was made with a syringe in the cuff to simulate leakage. As shown in figure 11.5, leakage was observed throughout the entire recording. As described in 3.2.1, a cuff leakage can lead to an underinflated cuff, which reflects that the cuff pressure remains under the manually set cuff pressure. Therefore it will be essential to construct an algorithm to detect when the cuff pressure is below the manually set cuff pressure.

The algorithm is constructed similarly to the algorithm to detect position changes. Rather than detecting cuff pressure over set cuff pressure, the algorithm detects cuff pressure more than 1 cm H_2O under the set pressure. The algorithm must return the under-detection and the corresponding time. By utilizing the algorithm for under-detection another algorithm for potential leakage is developed. When leakage occurs, it is expected that the pressure will decrease by more than 1 cm H_2O multiple times within few minutes. Under normal treatment conditions, it is anticipated that the cuff pressure will decrease briefly but reestablish again after a few seconds. The algorithm must therefore give a warning when the cuff pressure is detected under set pressure more than 20 times in a window of 5 minutes. The warning must alert the user about a potential cuff leakage.

Part IV

Implementation

12. Implementation

In the following chapter, the implementation of CPATool is elaborated as a part of the OOP process. The chapter initiates with an updated class diagram followed by a block description outlining the requirements associated with each block. Additionally, a detailed explanation of the main functionalities within each block is presented as pseudocode. Finally, the chapter concludes with an updated activity diagram that illustrates the implementation of the intended system.

In relation to the implementation of CPATool, an updated class diagram has been developed to provide a more precise illustration of the actual code. As in the initial class diagram in section 11.1 in figure 11.1 the *main* class is not included. In the class diagram, the Model classes are indicated by a green frame and the View classes by a blue frame. Additionally, the unfilled arrow tip, as described in 11.1, shows inheritance meaning that both *Dashboard1* and *Dashboard2* inherit from *ScreenLayout*[70, Chapter 9][69]. The directed association arrow, indicated by an arrowhead, illustrates which classes have a relationship, indicating that the classes are associated in a specific way[73]. For example, it can be observed that *Login* is associated with *DatabaseCommunication*. The class diagram is illustrated in figure 12.1

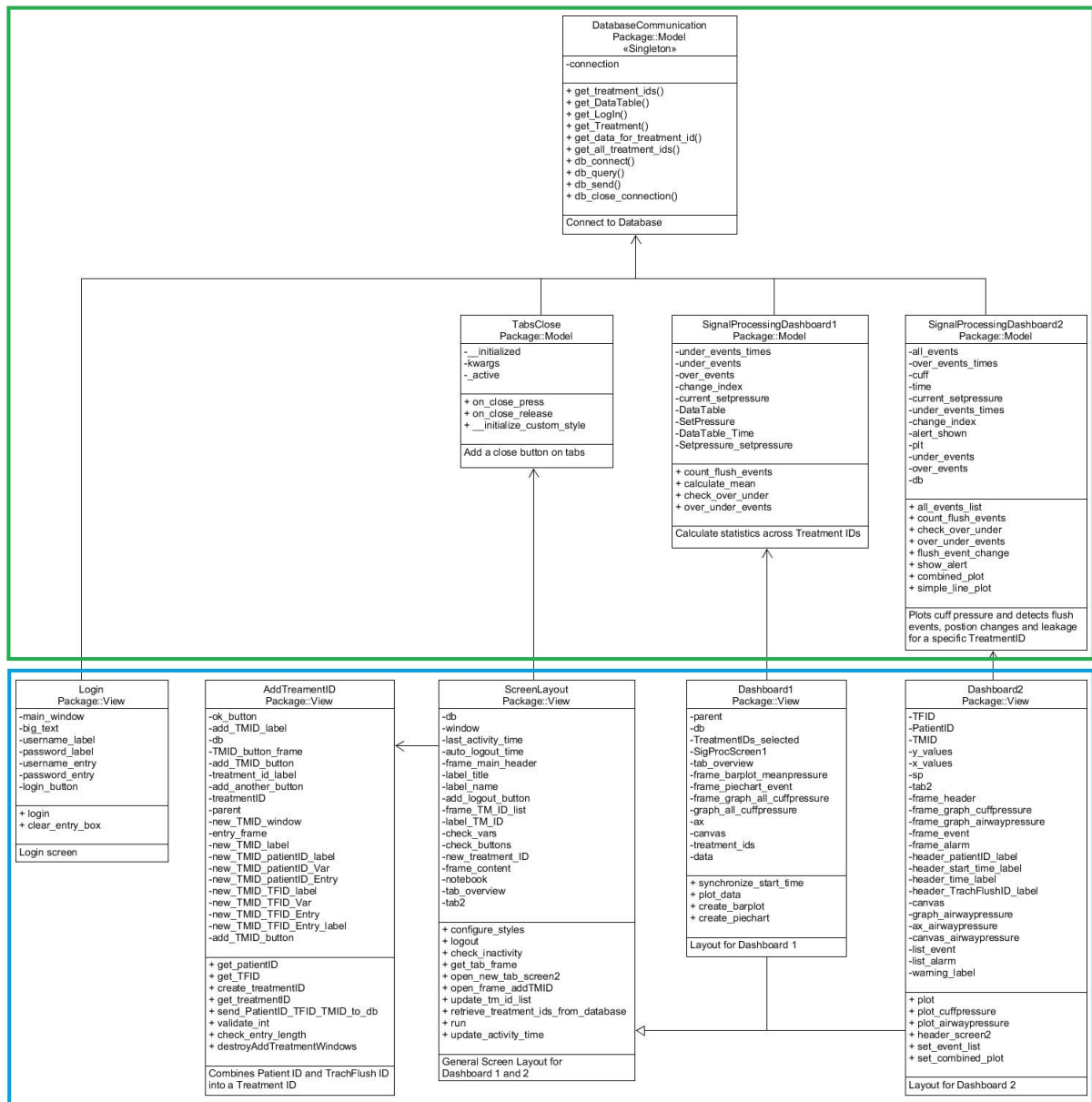


Figure 12.1: The class diagram illustrates the implementation of CPATool. The green frame indicates the Model classes and the blue frame indicates the View classes.

12.1 Block 1: Database Communication

To access all the data, CPATool must establish a connection to the phpMyAdmin server. In the code, the host, user, password, and database name are specified. The objective of this block is to fulfill the specified requirements effectively in 12.1

ID	Requirements
SR4	The system must establish a connection to the database.
SR5	The system must retrieve data from the database.
SR23	The system must send calculated values derived from SR17 to the database.
DBR1	The database must save cuff pressure data.
DBR2	The database must save airway pressure data.
DBR3	The database must save timestamps for all collected TrachFlush data.
DBR4	The database must save TrachFlush ID.
DBR5	The database must save Patient ID.
DBR6	The database must save Treatment ID.
DBR7	The database must save time when a Treatment ID is assigned.
DBR8	The database must save system codes from TrachFlush.
DBR9	The database must save alarms from TrachFlush.
DBR10	The database must save "FlushAvgOut" from the TrachFlush.
DBR11	The database must store personal logins which contain username and password.
DBR12	The database must save detected changes in body position for a specific Treatment ID.
DBR13	The database must save detected flush performed by the TrachFlush device for a specific Treatment ID.
DBR14	The database must save detected abnormal fluctuation for a specific Treatment ID.
DBR15	The database must delete TrachFlush data after the data has been saved for 5 years.
DBR16	The database must save detected leakage of the cuff for a specific Treatment ID.
DBR17	The database must be able to save data for 5 years.
ADBR1	The database must receive data from the TrachFlush every 500 ms.

Table 12.1: The table shows the requirements that needs to be met by block 1.

To illustrate the implementation of the database connection a pseudocode is displayed below.

Class DatabaseCommunication:

```
    _instance = None
```

```
Method __new__(cls, *args, **kwargs):
```

```
    If cls._instance is None:
```

```
        cls._instance = Super(DatabaseCommunication).__new__(cls, *args, **kwargs)
```

```
    Return cls._instance
```

```
Method __init__(self):
```

```
    If not hasattr(self, 'connection'):
```

```
        self.connection = self.db_connect()
```

```
Method db_connect():
```

```
    Try:
```

```
        self.connection = Database connecion with host, user, password, database name
```

```

        Print "Connection succeed."

    Return self.connection

Except Error as e:
    Print "Error"
    Return None

```

Other database methods...

The 'new' method is used to create a new instance of the class. It is employed to implement the singleton pattern by checking if an instance of the class already exists. If the instance is 'None', a new instance is created using this method. If an instance already exists, it returns the existing instance, thus preventing the creation of a new one.

The 'init' method initializes the instance attributes, a process that occurs only once due to the singleton pattern. This method checks whether the instance already possesses the connection attribute. If not, it creates it by calling the 'db connect' method.

The 'db connect' method facilitates the connection to the phpMyAdmin database. It configures connection parameters and attempts to establish a connection. Upon success, it returns the connection object, enabling SQL query execution. In case of failure, it provides an error message indicating the inability to establish a connection with the database.

12.2 Block 2: Screen Login

To access Dashboard 1 in the CPATool, users must authenticate themselves by providing a valid username and password, which are then validated in the database. Block 2 serves the purpose of fulfilling the requirement in table 12.2:

ID	Requirements
ASR1	The system must require the user to log in with a personal login.

Table 12.2: The table shows the requirements that need to be met by block 2.

To illustrate the implementation of the Login a pseudocode is displayed below.

```
Method login():
Retrieve username from username_inputbox
Retrieve password from password_inputbox
db_communication = New DatabaseCommunication()
connection = db_communication.db_connect()
If connection:
    login_data = db_communication.db_query("SELECT Username, Password FROM LogIn")
    For each row in login_data:
        If username equals row['Username'] and password equals row['Password']:
            Show messagebox "Log In Successful: Welcome to CPATool!"
            Set logged_in to True
            Close the login window (main_window.withdraw)
            Open the main page (open_screenlayout)
            Return
    Show message "Login Failed: Invalid username or password"
Else:
    Show message "Error: Failed to connect to database"
```

The function begins by establishing a connection to the database, from which the function iterates through the login data to check if the provided username and password match any entry in the database. If the username and password match the records in the database, the user will receive the message "Welcome to the CPATool." The login screen will then close, and the user will be redirected to Dashboard 1. If the user enters an incorrect username or password a message saying "Invalid username or password" is displayed. Additionally, the user will receive an error message if the connection to the database has failed.

12.3 Block 3: Screen Layout

To ensure a consistent layout across the entire system, consisting of its two distinct dashboards, a class named *ScreenLayout* has been created. This class comprises a variety of functions that are generic across the dashboard. The objective of this block is to fulfill the specified requirements effectively in 12.3

ID	Requirements
SR7	The system must be divided into two dashboards to distinguish one or multiple treatments.
SR9	The system should enable the user to open a new frame with data for a selected Treatment ID.
SR10	The system must allow the user to adjust the time interval from 2 minutes to the full-time treatment of the graphical visualizations.
SR13	The system must contain a search function, for TrachFlush ID, Patient ID, Treatment ID, and date.
SR14	The system should allow the user to exclude specific Treatment IDs from the statistics.
SR25	The system must have an automatic screen lock when the system is left unattended for 5 minutes.

Table 12.3: The table shows the requirements that need to be met by block 3.

To illustrate the implementation of the Screen Layout a pseudocode is displayed below.

```
Method open_new_tab_screen2(TM_ID_name):
```

```
    For each index from 0 to the last tab index:
```

```
        tab_text = Get the text of the tab at the current index
```

```
        If tab_text equals TM_ID_name:
```

```
            Select the tab at the current index
```

```
        Return
```

```
dashboard2 = Create a new Dashboard2 object with the TM_ID_name
```

```
Add dashboard2.tab2 to the notebook with text TM_ID_name
```

```
index = Get the index of the last tab
```

```
Select the tab at the new index
```

The method 'open new tab screen' handles the creation and navigation of tabs within the GUI. The function iterates over each tab in the interface, searching for a tab with text matching 'TM ID name'. If such a tab is found, it is selected, and the function returns. If no matching tab is found, a new 'Dashboard2' object is created with the provided 'TM ID name'. This object is added as a new tab

to the interface's notebook, and its index is determined. Finally, the newly added tab is selected, ensuring that it is visible to the user.

12.4 Block 4: Add New Treatment ID

A function has been created to allow users to add a new Treatment ID, which includes a Patient ID and a TrachFlush ID. Therefore the purpose of block 4 is to fulfill the requirement in table 12.4:

ID	Requirements
SR3	The system must contain a function to create a new Treatment ID containing a Patient ID and a TrachFlush ID.

Table 12.4: The table shows the requirements that need to be met by block 4.

To illustrate the implementation of the function create a Treatment ID a pseudocode is displayed below.

```
Method create_treatmentID():
    treatmentID = random 4-digit number
    sql_query = "SELECT * FROM Treatment WHERE TreatmentID = treatmentID"
    # check if treatment ID already exists
    result = execute SQL query
    If result is empty:
        Return treatmentID
```

The function that creates a Treatment ID specifies that when the user enters a Treatment ID, it must consist of a 4-digit number. It then checks if the generated Treatment ID exists in the database. If the result is empty, indicating that the Treatment ID does not exist, it closes the database connection and returns the Treatment ID.

12.5 Block 5: Calculate Events and Statistics

To effectively display data, various algorithms are implemented as described in the design of algorithms in section 11.3. The purpose of block 5 is to fulfill the requirement in table 12.5:

ID	Requirements
SR11	The system must detect changes in body position by identifying patterns in the signal.
SR12	The system must detect 'flush' by the TrachFlush.
SR17	The system must calculate when an event has occurred.
SR21	The system must detect continuous fluctuation in cuff pressure as cuff leakage.
SR22	The system must detect abnormal fluctuation by detecting deviations from expected signal behavior.

Table 12.5: The table shows the requirements that need to be met by Block 5.

To illustrate the implementation of the algorithm for flush event a pseudocode is displayed below.

Method `flush_event_change()`:

`previous_flush_event = None`

`first_flush_event = False`

For each entry `in` `DataTable`:

`current_time = entry['Time']`

`current_flush_event = entry['FlushEvent']`

If `not` `first_flush_event`:

`first_flush_event = True`

`previous_flush_event = current_flush_event`

`Continue`

If `current_flush_event != previous_flush_event`:

`Append current_time to flush_event_times`

`previous_flush_event = current_flush_event`

The method 'flush event change' outlines the logic for detecting changes in the FlushEvent and recording the times of these changes. It begins by initializing 'previous flush event' to 'None' and 'first flush event' to 'False'. As it iterates through each entry in the 'DataTable' in the database, it retrieves 'current time' and 'current flush event'. If the 'first flush event' is 'False', it sets it to 'True', saves the 'current flush event' as 'previous flush event', and skips further processing for that entry. If 'current flush event' differs from 'previous flush event', it appends 'current time' to 'flush event times' and updates 'previous flush event' to 'current flush event'.

To illustrate the implementation of the algorithms for detecting position change and cuff leakage pseudocodes are displayed below.

Method `check_over_under(cuff_pressure, current_setpressure)`:

```
difference_value = 1
If cuff_pressure > current_setpressure + difference_value:
    Return f"Cuff Pressure: {cuff_pressure} For set pressure: {current_setpressure}"
ElseIf cuff_pressure < current_setpressure - difference_value:
    Return f"Cuff Pressure: {cuff_pressure} For set pressure: {current_setpressure}"
Return None
```

Method `over_under_events()`:

```
For each (time, cuff) in zip(DataTable_Time, DataTable_cuffPressure):
    # Update set pressure if necessary
    If change_index < length(SetPressure_changes) and time >= SetPressure_changes[change_index][0]:
        current_setpressure = SetPressure_changes[change_index][1]
        Increment change_index
    # Check if the cuff pressure is over or under the set pressure
    event = check_over_under(cuff, current_setpressure)
    If event is not None:
        If cuff > current_setpressure - 1:
            Append event to over_events
            Append time to over_events_times
        ElseIf cuff < current_setpressure - 1:
            Append event to under_events
            Append time to under_events_times
```

The method 'check over under' compares the cuff pressure with the current set pressure. If the difference exceeds a specified threshold defined as 'difference value', it returns a message indicating whether the cuff pressure is more than one higher or lower than the set pressure. The method 'over under events' operates on pairs of data retrieved from the 'DataTable' from the columns 'Time' and 'CuffPressure'. For each pair, it updates the 'current set pressure' if there are changes in the set pressure. It then uses the 'check over under' function to determine if the cuff pressure is more than one over or under the set pressure. If a pressure difference is detected, the function categorizes the event as either an "over" or "under" event and appends the corresponding event message and time to the appropriate lists.

12.6 Block 6: Dashboard 1

The first dashboard the user encounters is Dashboard 1, which provides a statistical overview of events and the average of all cuff pressures. Therefore the purpose of block 6 is to fulfill the requirement in table 12.6:

ID	Requirements
SR8	The system must be able to give a statistical overview of cuff pressure and events occurring across all Treatment IDs.
SR19	The system must give an overview of all events defined in requirements SR11, SR12, SR21, and SR22 in the specified time interval.

Table 12.6: The table shows the requirements that need to be met by block 6.

To illustrate the implementation of the method to create a bar plot a pseudocode is displayed below.

Method `create_barplot(mean_values)`:

```
# Data to bar plot
treatment_ids = List of keys from mean_values
values = List of values from mean_values

# Create a bar plot
Create a new figure and axis for the bar plot
Create a bar plot with treatment_ids on the x-axis and values on the y-axis
y-axis label = 'Cuff Pressure'
x-axis label = 'Treatment ID'
Set plot title = 'Average Cuff Pressure'

# Set x-axis ticks and labels
x-axis ticks = range(len(treatment_ids))
x-axis tick labels = treatment_ids with 45-degree rotation and right alignment

# Integrate bar plot
Create a FigureCanvasTkAgg object with the bar plot figure and set master to frame_barplot
Draw the canvas_barplot
Pack the Tkinter widget with fill='both' and expand=True
```

The 'create barplot' method generates a bar plot displaying the mean cuff pressures. It achieves this by retrieving Treatment IDs and their corresponding mean values, which are calculated in the *SignalProcessingDashboard1* class. The bar plot is configured to have Treatment IDs along the x-axis and mean values along the y-axis, and it is then integrated into the layout.

12.7 Block 7: Dashboard 2

Upon opening a tab for a specific Treatment ID, the user is directed to Dashboard 2. The purpose of block 7 is to fulfill the requirement in table 12.7:

ID	Requirements
SR1	The system must include a graphical visualization of the cuff pressure over the time the patient has been intubated.
SR2	The system must include a graphical visualization of the airway pressure over the time the patient has been intubated.
SR6	The system must show the Patient ID and the TrachFlush ID when the Treatment ID is chosen.
SR15	The system must indicate graphically when an event has occurred.
SR16	The system must indicate graphically when an alarm has occurred.
SR18	The system must inform when an alarm from the TrachFlush has occurred.
SR20	The system must provide information on the duration the patient's cuff has been controlled by a TrachFlush.
SR24	The system must issue a warning message if the patient has not been connected to a TrachFlush for at least 1 hour.

Table 12.7: The table shows the requirements that need to be met by block 7.

To illustrate the implementation of the method to set an event list a pseudocode is displayed below.

Method `set_event_list()`:

```
Get all_events() from SignalProcessingScreen2
Clear the list_event
# Populate the listbox with events and set it
for event, _ in all_events:
    self.list_event.insert(tk.END, event)
self.list_event.pack(fill='both', expand=True)
```

The 'set event list' method is designed to update the event list displayed in Dashboard 2. It begins by retrieving all events using the 'all events' method from the *SignalProcessingDashboard2* class. Next, it clears the existing 'list event' to ensure no outdated events are displayed. The method then iterates through the retrieved events, inserting each event into the 'list event' list box. Finally, it ensures that the 'list event' list box is properly displayed by packing it to fill the available space and allowing it to expand as needed. This process ensures that the event list is always up-to-date and properly formatted for the user.

12.8 Implemented Activity Diagram

Associated with the implementation of CPATool, a new activity diagram has been developed, seen in figure 12.2. The activity diagram is a reassessment of the activity diagram in high level design visualized in figure 10.6. The activity diagram visualizes the flow of CPATool.

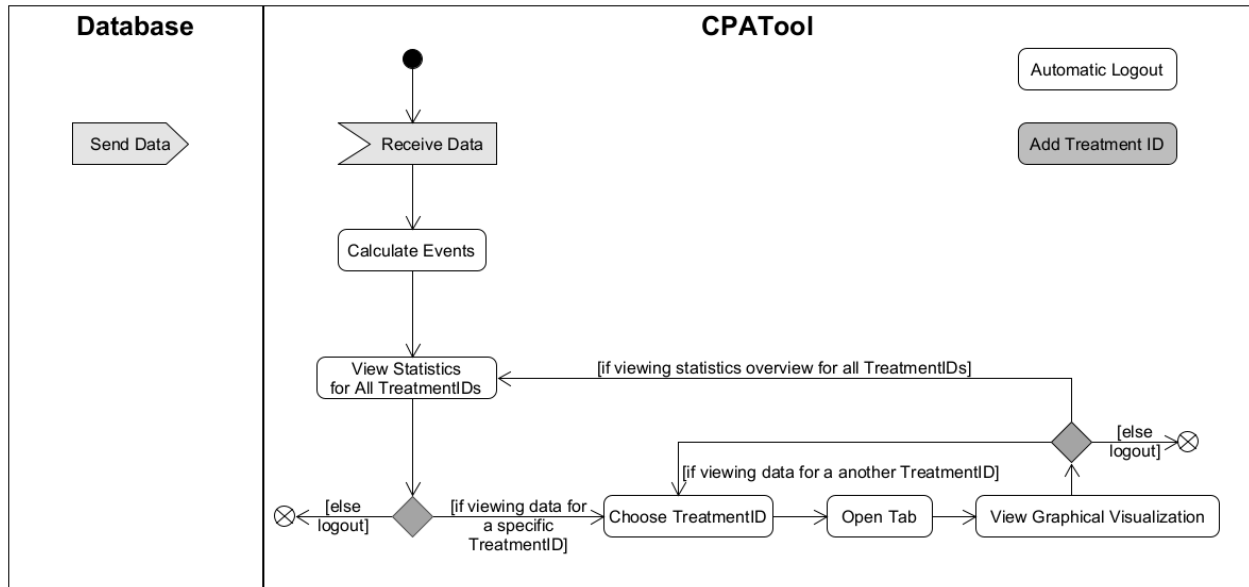


Figure 12.2: The activity diagram illustrates the implementation of CPATool.

Part V

Test

13. Software Testing

In the following chapter, the different methods used to test the implemented system will be described as well as a Requirement Traceability Matrix (RTM) which will show the result of the tests. During the validation phase of the V-model, software testing plays a crucial role in confirming whether the system functions according to its intended specifications. This includes making sure the software is error-free and meets both user and system requirements, as described in table 7.2 and 9.1.

13.1 Test Cases

A test case represents a structured protocol utilized in software testing to verify the functionality of specific parts of the system. The execution of a test case entails validating whether the observed outcomes align with the expected output. Each test case is characterized by a comprehensive array of parameters that are seen in figure 13.1.[74]

Module name:						
Test Case ID:						
Test case description:						
Environmental Information:						
Test ID	Test condition	Test steps	Test input	Test expected result	Actual result	Status

Figure 13.1: A visualization of the template used for the test cases.

Test cases can be delineated before, during, or after the software development process. It is known that the V-model prescribes that unit tests and integration tests are done continuously in the development process. However, in the context of this project, the specification of test cases occurs after the completion of software development. Depending on the testing methodology different types of functional test cases can be established. In this project, emphasis will be placed on executing unit- and integration tests to comprehensively evaluate the functionality and integrity of the developed software. [74]

13.2 Black Box Testing

Software testing can be performed by using the technique known as black box testing [75]. Black box testing is performed without any prior knowledge and access to the source code of the software and does not take into account, any inner workings of the software. The technique focuses only on the input and output of the software and is mainly used to test the functionality of the software as well as ensure that it meets the requirements. [76] Black box testing is usually performed by a tester with little to no knowledge of the internal code. However, in this project, it has been performed by the developers of the system, as it is the most fitting technique. All other aspects of black box testing, are done as intended. [76]

Within black box testing, functional tests are situated. Functional testing is an integral part of ensuring that the requirements of the system are met. Functional testing focuses on the actual usage of the system such as basic usability, mainline functions, accessibility, and error conditions. [77] This project will focus on the following three functional tests: Unit testing, integration testing, and system testing.

13.2.1 Unit Test

To verify functionality early in the development, unit testing is done. Unit testing involves the evaluation of individual units of a software system, which typically are functions. Subjecting each unit to various test cases with known inputs, it is possible to validate that the unit produces the expected outputs and behaves as intended under different conditions. [78, p. 232-234]

To conduct unit testing, the 'PyTest' library package was utilized within the PyCharm IDE. To ensure reproducibility and maintain an organized structure, a dedicated directory named 'Test' was created. Within this directory, all test files were systematically placed, facilitating easy navigation and management of tests.

Each class within the project was assigned its own test file. This modular approach ensures that all the functions of a class are comprehensively tested within a single, dedicated file. By isolating tests in this manner, it becomes simpler to identify and address issues specific to a particular class, thereby improving maintainability and debugging efficiency.

The testing framework employed the 'assertTrue' method from the unit test module to validate the functionality of individual functions. This method checks whether a given condition is true, and if not, the test fails. Each function was tested in isolation to ensure that all aspects of the system performed as expected under various conditions.

Appendix C.1 provides detailed results of these unit tests, showcasing the outcomes of testing each function individually. Here, it is observed that all functions succeed individually, indicating that each function operates as intended when tested in isolation.

13.2.2 Integration Test

Integration testing is done by testing two or more units. The purpose of integration testing is to test the interface between the components to detect any complications that may arise when these interact with each other. [78, p.234-237]

During integration testing, each block of the system undergoes separate testing, as each block comprises multiple functions that are integrated. This process allows for an assessment to determine whether the blocks, when considered as a whole, function as intended. In the following section, the results of the test cases for the seven blocks are described. Detailed test cases and performance results are provided in Appendix C.2.

Block 1: Database Communication

The database test results indicated that the system successfully established a connection to the database and fetched data from the correct tables. During the pilot tests, it was observed that the database received data every 500 ms in the correct columns of the right tables. The test of requirements SR23, DBR11, DBR12, DBR13, DBR14, and DBR15, failed because the functionality of saving to the database was not implemented. Table 13.1 lists the requirements tested by the database communication test case and shows the results for each requirement after testing.

BTC1	Test of Block 1: Database communitation	
ID	Requirements	Test results
SR4	The system must establish a connection to the database.	✓
SR5	The system must retrieve data from the database.	✓
SR23	The system must send calculated values derived from SR17 to the database.	×
DBR1	The database must save cuff pressure data	✓
DBR2	The database must save airway pressure data.	✓
DBR3	The database must save timestamps for all collected TrachFlush data.	✓
DBR4	The database must save TrachFlush ID.	✓
DBR5	The database must save Patient ID.	✓
DBR6	The database must save Treatment ID.	✓
DBR7	The database must save time when a Treatment ID is assigned	✓
DBR8	The database must save system codes from TrachFlush.	✓
DBR9	The database must save alarms from TrachFlush.	×
DBR10	The database must save "FlushAvgOut" from the TrachFlush.	✓
DBR11	The database must store personal logins which contain username and password.	✓
DBR12	The database must save detected changes in body position for a specific Treatment ID.	×
DBR13	The database must save detected flush performed by the TrachFlush device for a specific Treatment ID.	×
DBR14	The database must save detected abnormal fluctuation for a specific Treatment ID.	×
DBR15	The database must delete TrachFlush data after the data has been saved for 5 years.	×
DBR16	The database must save detected leakage of the cuff for a specific Treatment ID.	×
ADBR1	The database must receive data from the TrachFlush every 500 ms.	✓

Table 13.1: The table contains the requirements linked to block 1 along with their corresponding test results. The check mark indicates that the requirements have been met, while the cross indicates that the requirements have not been fulfilled.

Block 2: Screen Login

Testing the login screen requires a database connection to verify data from the 'Login' table. The test results indicate that CPATool correctly allows users to enter their username and password. By querying the 'Login' table in the database, the system determines whether the login credentials are valid. If the credentials do not match any existing records, the user is prompted with a warning message indicating that the login does not exist. Conversely, if the credentials are valid, the user is redirected to the front page. Table 13.2 lists the requirements tested by the login test case and shows the results for each requirement after testing.

BTC2	Test of Block 2: Screen Login	
ID	Requirements	Test results
ASR1	The system must require the user to log in with a personal login.	✓

Table 13.2: The table contains the requirements linked to block 2 along with their corresponding test results. The check mark indicates that the requirements have been met, while the cross indicates that the requirements have not been fulfilled.

Block 3: Screen Layout

Testing the screen layout requires a database connection to verify data from the 'DataTable' table. The test results indicate that CPATool features two distinct dashboards to differentiate between single and multiple treatments. This design choice facilitates user interaction: upon opening the system, users are greeted with a statistical overview of all Treatment IDs. The user can then opt to delve deeper into a specific Treatment ID for further analysis. Additionally, the system incorporates an automatic logout feature, which activates after 5 minutes of user inactivity.

However, it is noted that some requirements related to block 3 remain unfulfilled. This discrepancy arises from incomplete implementation, resulting in the non-fulfillment of requirements SR10, SR13, and SR14. Table 13.3 lists the requirements tested by the screen layout test case and shows the results for each requirement after testing.

BTC3	Test of Block 3: Screen Layout	
ID	Requirements	Test results
SR7	The system must be divided into two dashboards to distinguish one or multiple treatments.	✓
SR9	The system should enable the user to open a new frame with data for a selected Treatment ID.	✓
SR10	The system must allow the user to adjust the time interval from 2 minutes to the full-time treatment of the graphical visualizations.	×
SR13	The system must contain a search function, for TrachFlush ID, Patient ID, Treatment ID, and date.	×
SR14	The system should allow the user to exclude specific Treatment IDs from the statistics.	×
SR25	The system must have an automatic screen lock when the system is left unattended for 5 minutes.	✓

Table 13.3: The table contains the requirements linked to block 3 along with their corresponding test results. The check mark indicates that the requirements have been met, while the cross indicates that the requirements have not been fulfilled.

Block 4: Add New Treatment ID

Testing the Add New Treatment ID requires a database connection to verify data from the 'Treatment' table. The result of the test case was that the CPATool was able to generate a new Treatment ID when a Patient ID and a TrachFlush ID were given in input boxes in the window of adding a new

Treatment ID. Table 13.4 lists the requirements tested by the Add New Treatment ID test case and shows the results for each requirement after testing.

BTC4 Test of Block 4: Add New Treatment ID		
ID	Requirements	Test results
SR3	The system must contain a function to create a new Treatment ID containing a Patient ID and a TrachFlush ID.	✓

Table 13.4: The table contains the requirements linked to block 4 along with their corresponding test results. The check mark indicates that the requirements have been met, while the cross indicates that the requirements have not been fulfilled.

Block 5: Calculate Events and Statistics

To test the Calculate Events and Statistics block it requires a database connection to verify data from the 'DataTable' table. The test results reveal that CPATool effectively detects events such as position changes, flushes, and cuff leakage, while also providing calculations for event occurrence. However, it is noted that requirement SR22 remains unfulfilled due to the absence of implementation regarding abnormal fluctuations. In table 13.5 are listed the requirements tested by the Calculate Events and Statistics test case and shows the results for the requirement after the tests.

BTC5 Test of Block 5: Calculate Events and Statistics		
ID	Requirements	Test results
SR11	The system must detect changes in body position by identifying patterns in the signal.	✓
SR12	The system must detect flush by the TrachFlush device.	✓
SR17	The system must calculate when an event has occurred.	✓
SR21	The system must detect continuous fluctuation in cuff pressure as cuff leakage.	✓
SR22	The system must detect abnormal fluctuation by detecting deviations from expected signal behavior.	×

Table 13.5: The table contains the requirements linked to block 5 along with their corresponding test results. The check mark indicates that the requirements have been met, while the cross indicates that the requirements have not been fulfilled.

Block 6: Dashboard 1

To test Dashboard 1, it requires a database connection to verify data from the 'DataTable' table. The results of the test case were that the CPATool was able to create a pie chart and a bar chart to give a statistical overview of cuff pressure and events occurring across all Treatment IDs. Table 13.6 lists the requirements tested by the Dashboard 1 test case and shows the results for each requirement after testing.

BTC6 Test of Block 6: Dashboard 1		
ID	Requirements	Test results
SR8	The system must be able to give a statistical overview of cuff pressure and events occurring across all Treatment IDs.	✓
SR19	The system must give an overview of all events defined in requirements SR11, SR12, SR21, and SR22 in the specified time interval.	✓

Table 13.6: The table contains the requirements linked to block 6 along with their corresponding test results. The check mark indicates that the requirements have been met, while the cross indicates that the requirements have not been fulfilled.

Block 7: Dashboard 2

To test Dashboard 2, it requires a database connection to verify data from the 'DataTable' table. The results of the test case were that the CPATool was able to show a graphical visualization of both cuff pressure and airway pressure. It was also able to, in the header of Dashboard 2, show the Patient ID and TrachFlush ID corresponding to the chosen Treatment ID. However, it is noted that requirements SR15, SR16, SR18, and SR24 remain unfulfilled due to the lack of implementation. Table 13.7 lists the requirements tested by the Dashboard 2 test case and shows the results for each requirement after testing.

BTC7 Test of Block 7: Dashboard 2		
ID	Requirements	Test results
SR1	The system must include a graphical visualization of the cuff pressure over the time the patient has been intubated.	✓
SR2	The system must include a graphical visualization of the airway pressure over the time the patient has been intubated.	✓
SR6	The system must show the Patient ID and the TrachFlush ID when the Treatment ID is chosen.	✓
SR15	The system must indicate graphically when an event has occurred.	×
SR16	The system must indicate graphically when an alarm has occurred.	✓
SR18	The system must inform when an alarm from the TrachFlush device has occurred.	×
SR20	The system must provide information on the duration the patient's cuff has been controlled by a TrachFlush device.	✓
SR24	The system must issue a warning message if the patient has not been connected to a TrachFlush for at least 1 hour.	×

Table 13.7: The table contains the requirements linked to block 7 along with their corresponding test results. The check mark indicates that the requirements have been met, while the cross indicates that the requirements have not been fulfilled.

13.2.3 System Test

System testing involves the comprehensive examination of the entire system, where all individual components are brought together and assessed as a unified entity. This stage of testing focuses on verifying the compatibility and interoperability of these components to ensure seamless integration. Additionally, system testing evaluates the system's behavior. It aims to detect and prevent any

instances of undesirable data loss or unauthorized data modifications. This safeguards the integrity and reliability of the system's data-handling processes. [78, p.237-240]

In the following section, the results from the system test are presented in detail. This test was conducted by thoroughly running the system and systematically evaluating all the implemented components. Each aspect of the system was tested to ensure that it functions correctly and meets the specified requirements.

Login

The initial screen seen when opening the CPATool is the login screen, as seen in figure 13.2a. The user must log in by entering a username and password on the login screen to use the CPATools functionalities. The Screen Login utilizes the Model class *DatabaseCommunication* for connection to the database and the View class *ScreenLogin* for the login interface, as seen in figure 12.1.

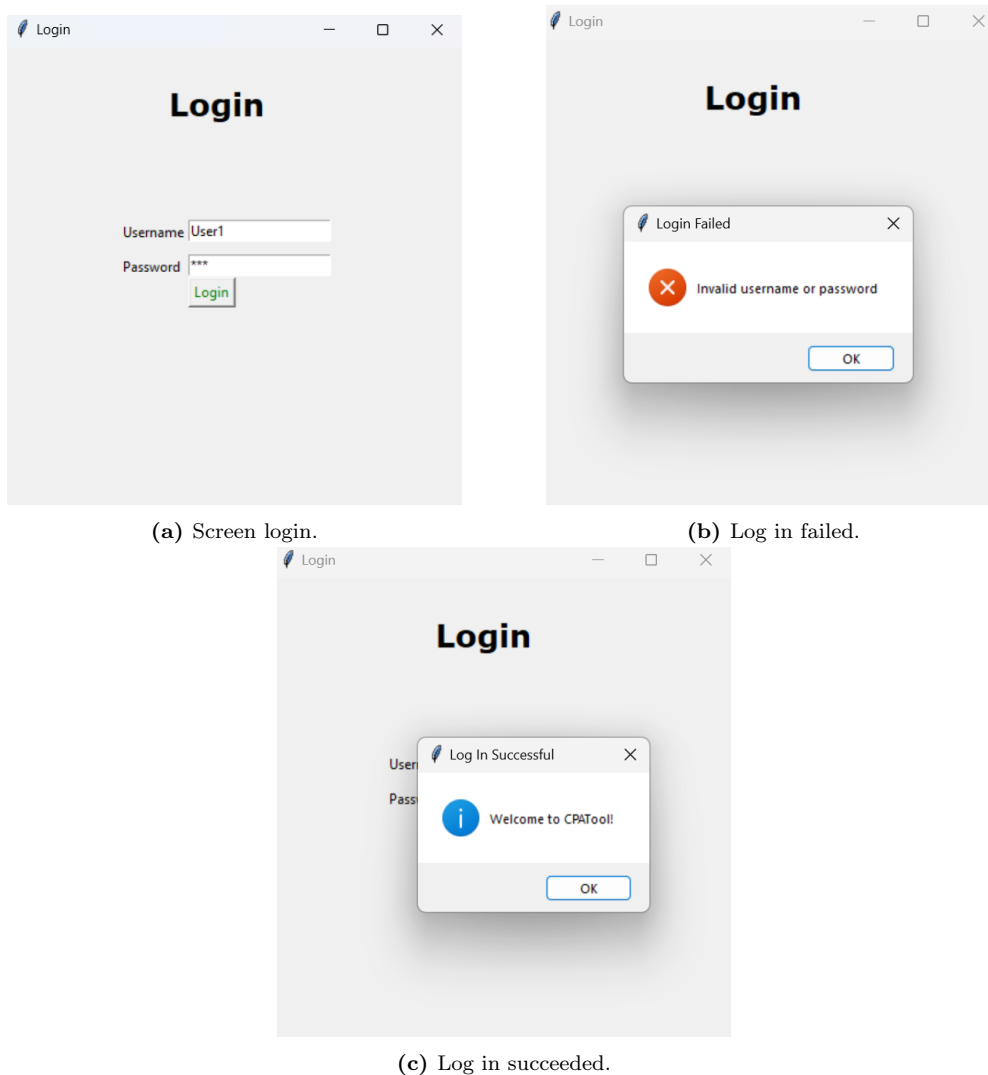


Figure 13.2: The login screen for CPATool contains a login screen where the user can enter a personal login, seen in figure 13.2a. If the personal login does not exist in the database a pop-up window appears with a warning, seen in figure 13.2b, and when the login succeeds a pop-up window with a welcome appears, seen in figure 13.2c.

After login, the user should be directed to Dashboard 1. Although this functionality is implemented, problems with the layout in Dashboard 1 occur after logging in. Due to these errors, the login function has been excluded from the rest of the system.

Dashboards

The Screen Layout, as seen in figure 13.3, acts as the foundational design element of CPATool. It consists of a header, a list with Treatment IDs placed on the left side which includes an 'Add New Treatment ID' button, and a large blank space reserved for the main content which either is Dashboard 1 or Dashboard 2. The dashboards will be opened as tabs at the top of the blank space. The Screen Layout utilizes the Model class *DatabaseCommunication* for the connection to the database and the View class *AddTreatmentID* for the purpose of adding a new Treatment ID, as seen in figure 12.1.

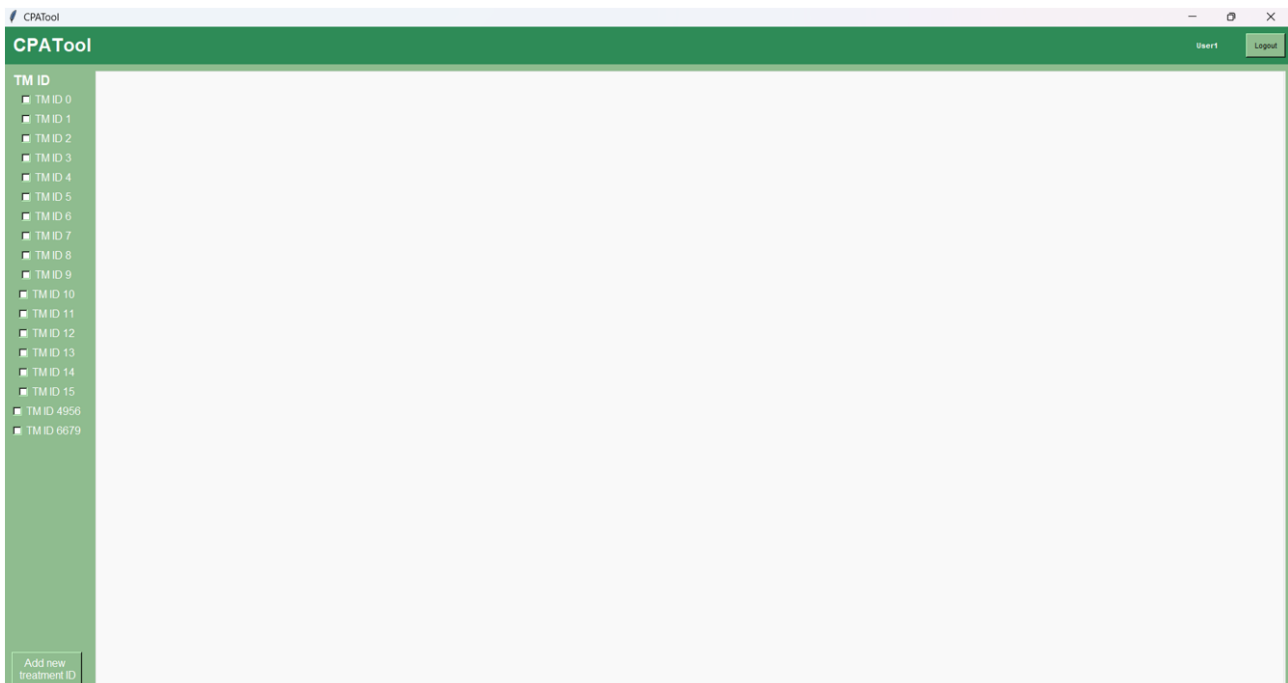


Figure 13.3: Visualization of the Screen Layout in CPATool.

Header

At the top of the Screen Layout, the header is presented which includes the system name, CPATool. In the right corner, the username is presented beside the logout button. By pressing the logout button the CPATool will close. Additionally, the system continuously monitors user activity. If the user is inactive for 5 minutes, the system will automatically trigger the logout process and close the system.

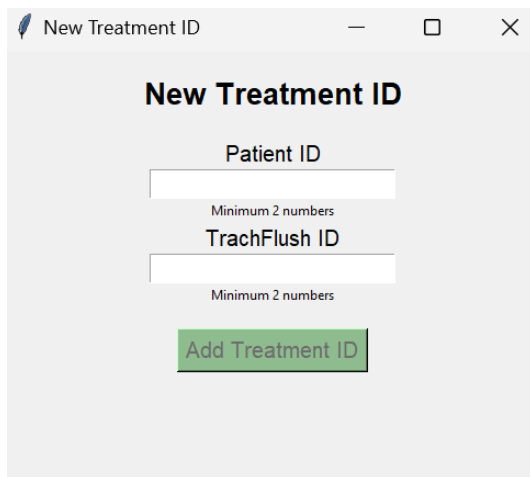
Treatment ID List

On the left side, a list of all Treatment IDs is pulled from the database through the Model class *DatabaseCommunication*, the list is shown in figure 13.3. By pressing a Treatment ID, Dashboard 2 will open with data belonging to the selected Treatment ID. The user has the opportunity to open

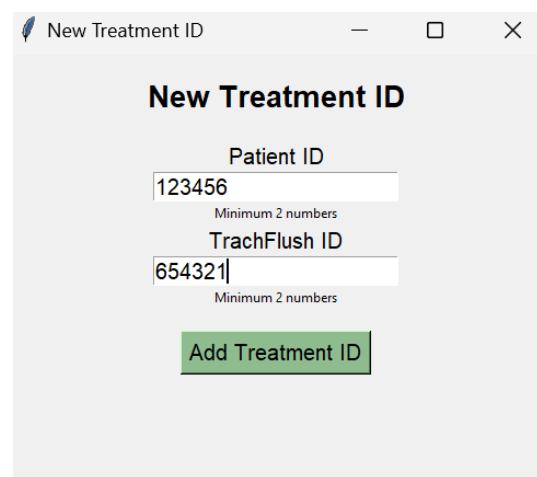
several Treatment IDs from the list which will be shown as tabs at the top of the Screen Layout. The main content in the tabs will be described in section 13.2.3. In connection with the Treatment ID, a checkbox is available, which is used for including and excluding Treatment IDs used in Dashboard 1 as described in section 13.2.3.

Add New Treatment ID

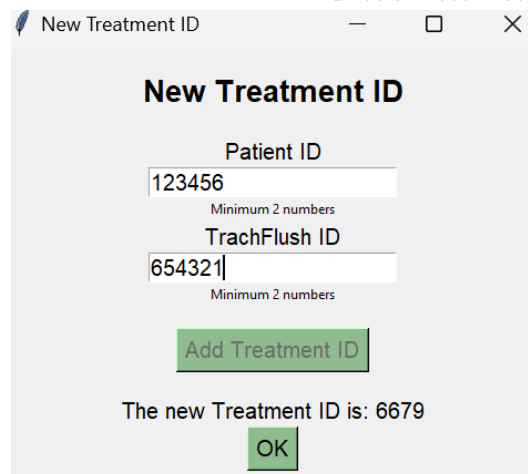
The lower left button below the list, shown in figure 13.3, presents the user with the opportunity to add a new Treatment ID. By pressing the button, the View class *AddTreatmentID* is used, which results in the 'New Treatment ID' screen appears. This is shown in figure 13.4a.



(a) Add Treatment ID start screen.



(b) Add Treatment ID when entering more than 2 numbers in both Patient ID and TrachFlush ID.



(c) Add treatment ID when the Add Treatment button is entered.

Figure 13.4: By pressing the 'Add new treatment ID' button, the 'New Treatment ID' screen appears as shown in 13.4a. Here the user can enter a Patient ID and a TrachFlush ID which must contain at least two numbers each, if the entry field is filled the 'Add new Treatment ID' button is set available 13.4b. When pressing the button, a Treatment ID is generated and the user can enter the OK button for closing the screen 13.4c.

To add a new Treatment ID, the user must enter a Patient ID and a TrachFlush ID as shown in figure 13.4a. When the user has entered a minimum of 2 numbers in each entry field the 'Add Treatment ID' button is set available as shown in figure 13.4b. By pressing the button, the user is given a Treatment

ID which is added to the database and must press 'OK' to close the screen as shown in figure 13.4c. Both the functionality and the interface belonging to the 'New Treatment ID' screen are defined in the View class *AddTreatmentID*.

Dashboard 1

Dashboard 1 acts as the statistical overview for all Treatment IDs. The dashboard is opened as a tab which is the first dashboard the user meets after a successful login. The dashboard is defined in the View class *Dashboard1* and the calculations for the statistical overview are defined in the Model class *SignalProcessingDashboard1*, as seen in the class diagram, figure 12.1. It consists of a line plot, pie chart, and bar chart containing various data from all Treatment IDs. Dashboard 1 is visualized in 13.5.

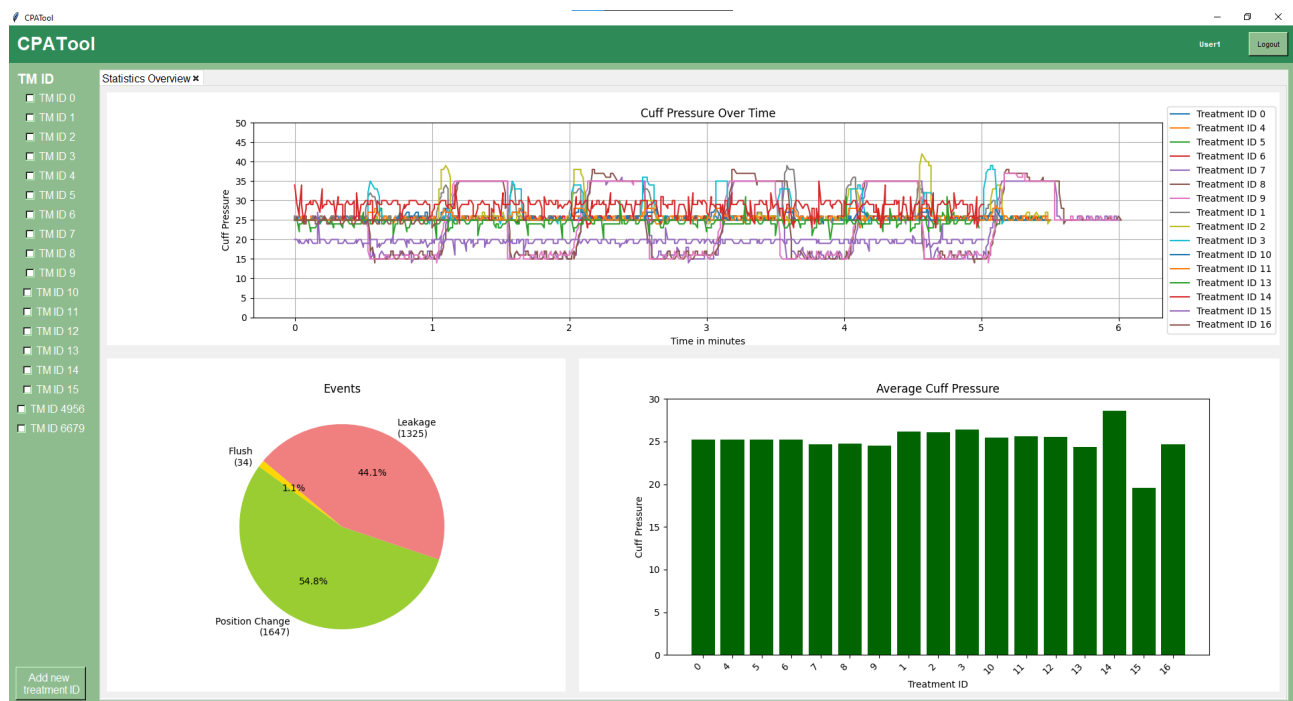


Figure 13.5: Dashboard 1 shows a statistical overview for Treatment IDs with a least 1 hour of available data.

In the Model class *SignalProcessingDashboard1*, the calculation for the pie chart and the bar chart are executed. The pie chart visualizes the number of events for the selected Treatment IDs. In the Model class *SignalProcessingDashboard1*, the number of events is calculated which is the number of flush, leakage, and position changes.

The bar chart visualizes the mean of the cuff pressure of the selected Treatment IDs. The mean of each Treatment ID is calculated and collected in the Model class *SignalprocessingDashboard1*.

The checkboxes linked to the Treatment IDs in the list on the left side of the Screen Layout can be used to select which Treatment IDs should be processed in the Model class *SignalprocessingDashboard1* and

displayed on the dashboard in the View class Dashboard 1. When the system opens, all Treatment IDs are selected by default.

In the top of the dashboard, a line plot for cuff pressure over time for all Treatment IDs is visualized. On the right side of the plot, the legend shows the included Treatment IDs and their corresponding color on the graph.

Dashboard 2

Dashboard 2 acts as the analysis for a specific Treatment ID which consists of a header, cuff pressure plot, airway pressure plot, event list, and alarm list. The Dashboard 2 is defined in the View class *Dashboard2*, the database communication and the calculations for the specific Treatment ID are defined in the Model classes *DatabaseCommunication* and *SignalProcessingDashboard2*, as seen in the class diagram, figure 12.1. Dashboard 2 opens as a tab containing data from the selected Treatment ID. The tabs will open by pressing the corresponding Treatment ID in the list. Additionally, it is possible to close the tabs. Dashboard 2 is visualized in 13.6.



Figure 13.6: Dashboard 2 shows details for the selected Treatment ID, on the figure Treatment ID 5 is selected, which Treatment ID is a pilot test for flush.

At the top of Dashboard 2, the header is presented which includes the Patient ID, treatment start, treatment duration, and TrachFlush ID. The header contains information about the selected Treatment ID, which has been obtained through the Model class *DatabaseCommunication*.

The upper plot visualizes cuff pressure over time for the selected Treatment ID. On the plot of cuff

pressure, several events are marked which is calculated in the Model class *SignalProcessingDashboard2*. The green horizontal stippled line indicates the manually set cuff pressure, as set on the TrachFlush device. The black vertical stippled lines indicate when a flush has been activated. The red dots indicate when the cuff pressure is more than 1 over the set pressure. Additionally, the blue dots indicate when the cuff pressure is more than 1 under the set pressure. Under the cuff pressure plot, the airway pressure plot is visualized with the airway pressure over time. This plot is directly defined in the View class *Dashboard2* since no calculations have been made on the data before it is visualized.

On the top right side of Dashboard 2, a list of events is shown with their corresponding time when they are detected. The detection of the events is calculated in the Model class *SignalProcessingDashboard2*.

Under the event list, there is allocated space for an alarm list, but because the alarm list function has not been implemented, nothing appears in the alarm list. The alarm list must, as well as the event list, show the corresponding time for the detected alarms in the Model class *SignalProcessingDashboard2*.

13.2.4 Acceptance Test

Traditionally, the final step before releasing software is acceptance testing. Acceptance testing ensures that the software meets the requirements and expectations set by stakeholders, typically end-users or clients in this case a chief physician. [79] As the software presented in this project is not ready for release, the acceptance test adds no significant value to the assessment of the system as a whole. It is therefore not included in this project. However, if the CPATool were ready for testing, it would as described, be utilized by a chief physician in the ICU. The test should be conducted with a patient in the ICU who is endotracheal intubated and connected to a TrachFlush. It should then be tested whether the user is able to add a Treatment ID corresponding to the patient and the Trachflush device. The TrachFlush must then send data to the database and the user must then observe if the system uses the data as expected and described in the system test.

13.3 Test Results

To compile the results of the unit tests, integration tests, and system test, an RTM has been created. In the matrices shown in tables 13.8, 13.9, 13.10, 13.11, the requirements are listed along with their corresponding MoSCoW prioritization. Additionally, each requirement is linked to a test case, indicating which test has been conducted to verify the requirement, along with whether the requirement has been met. These test cases can be found in appendix B. If an accepted requirement does not have an associated test case, it means the requirement has been tested in the system test.

ID	System Requirement	MoSCoW	Test Case ID	Status
SR1	The system must include a graphical visualization of the cuff pressure over the time the patient has been intubated.	Mo	BTC07.1	✓
SR2	The system must include a graphical visualization of the airway pressure over the time the patient has been intubated.	Mo	BTC07.2	✓
SR3	The system must contain a function to create a new Treatment ID containing a Patient ID and a TrachFlush ID.	Mo	BTC04	✓
SR4	The system must establish a connection to the database.	Mo	BTC01.1	✓
SR5	The system must retrieve data from the database.	Mo		✓
SR6	The system must show the Patient ID and the TrachFlush ID when the Treatment ID is chosen.	Mo	BTC07.3	✓
SR7	The system must be divided into two dashboards to distinguish one or multiple treatments.	Mo	BTC03.2, BTC03.3	✓
SR8	The system must be able to give a statistical overview of cuff pressure and events occurring across all Treatment IDs.	Mo	BTC06.1, BTC06.3	✓
SR9	The system should enable the user to open a new frame with data for a selected Treatment ID.	S	BTC03.2, BTC03.3	✓
SR10	The system must allow the user to adjust the time interval from 2 minutes to the full-time treatment of the graphical visualizations.	S		×
SR11	The system must detect changes in body position by identifying patterns in the signal.	S	BTC05.1	✓
SR12	The system must detect flush by the TrachFlush device.	S	BTC05.3	✓
SR13	The system must contain a search function, for TrachFlush ID, Patient ID, Treatment ID, and date.	S		×
SR14	The system should allow the user to exclude specific treatment IDs from the statistics.	S	BTC03.4	×
SR15	The system must indicate graphically when an event has occurred.	S	BTC07.6	✓
SR16	The system must indicate graphically when an alarm has occurred.	S	BTC07.7	×
SR17	The system must calculate when an event has occurred.	S	BTC05.1, BTC05.2, BTC05.3	✓
SR18	The system must inform when an alarm from the TrachFlush device has occurred.	Co	BTC07.9	×
SR19	The system must give an overview of all events defined in requirements SR11, SR12, SR21, and SR22 in the specified time interval.	Co	BTC05.1, BTC05.2, BTC05.3	✓
SR20	The system must provide information on the duration the patient's cuff has been controlled by a TrachFlush device.	Co	BTC07.4, BTC07.5	✓
SR21	The system must detect continuous fluctuation in cuff pressure as cuff leakage.	Co	BTC05.	✓
SR22	The system must detect abnormal fluctuation by detecting deviations from expected signal behavior.	Co		×
SR23	The system must send calculated values derived from SR16 to the database	Co		×
SR24	The system must issue a warning message if the patient has not been connected to a TrachFlush for at least 1 hour.	Mo	BTC07.11	×
SR25	The system must have an automatic screen lock when the system is left unattended for 5 minutes.	Mo	BTC03.6	✓

Table 13.8: The table represents the RTM for the system requirements. Fulfilled requirements are indicated with a check mark, while unfulfilled requirements are marked with a cross. If a requirement has not been implemented in the system, no corresponding test case is created, resulting in an empty cell.

ID	Additional System Requirement	MoSCoW	Test Case ID	Status
ASR1	The system must require the user to log in with a personal login.	Mo		×
ASR2	The system must be programmed in Python.	Mo		✓
ASR3	The system must delete all data related to the Treatment ID after a saving period of 5 years.	Co		×

Table 13.9: The table represents the RTM for the additional system requirements. Fulfilled requirements are indicated with a check mark, while unfulfilled requirements are marked with a cross. If a requirement has not been implemented in the system, no corresponding test case is created, resulting in an empty cell.

ID	Database Requirement	MoSCoW	Test Case ID	Status
DBR1	The database must save cuff pressure data.	Mo	BTC01.2	✓
DBR2	The database must save airway pressure data.	Mo	BTC01.3	✓
DBR3	The database must save timestamps for all collected TrachFlush data.	Mo	BTC01.7	✓
DBR4	The database must save TrachFlush ID.	Mo	BTC01.4	✓
DBR5	The database must save Patient ID.	Mo	BTC01.5	✓
DBR6	The database must save Treatment ID.	Mo	BTC01.6	✓
DBR7	The database must save time when a Treatment ID is assigned.	Mo	BTC01.7	✓
DBR8	The database must save system codes from TrachFlush.	S	BTC01.8	✓
DBR9	The database must save alarms from TrachFlush.	S		×
DBR10	The database must save "FlushAvgOut" from the TrachFlush.	S	BTC01.9	✓
DBR11	The database must store personal logins which contain username and password.	S	BTC01.10	✓
DBR12	The database must save detected changes in body position for a specific Treatment ID.	S		×
DBR13	The database must save detected flush performed by the TrachFlush device for a specific Treatment ID.	S		×
DBR14	The database must save detected abnormal fluctuation for a specific Treatment ID.	Co		×
DBR15	The database must delete TrachFlush data after the data has been saved for 5 years.	Co		×
DBR16	The database must save detected leakage of the cuff for a specific Treatment ID.	Co		×
DBR17	The database must be able to save data for 5 years.	Co		×

Table 13.10: The table represents the RTM for the database requirements. Fulfilled requirements are indicated with a check mark, while unfulfilled requirements are marked with a cross. If a requirement has not been implemented in the system, no corresponding test case is created, resulting in an empty cell.

ID	Additional Database Requirement	MoSCoW	Test Case ID	Status
ADBR1	The database must receive data from the TrachFlush every 500 ms.	S	BTC01.11	✓

Table 13.11: The table represents the RTM for the additional database requirements. Fulfilled requirements are indicated with a check mark, while unfulfilled requirements are marked with a cross. If a requirement has not been implemented in the system, no corresponding test case is created, resulting in an empty cell.

Part VI

Discussion and Conclusion

14. Discussion

The following chapter will outline a discussion of the result of this project. The discussion will present an overview of the main results of the test and a discussion of the methods used in this project. This is followed by an outline of the problem and implemented solution. Furthermore, the limitations of the system will be discussed, in terms of both any errors or inefficiencies in the programming as well as deliberate implementation exclusion in the software.

14.1 The Addressed Problem

During endotracheal intubation, it is assessed that absent measurements and external factors can affect the cuff pressure, leading to complications such as VAP. Furthermore, conventional measurement methods are used to regulate the cuff pressure but these methods are inaccurate according to the standard range of cuff pressure which must be between 20-30 cm H_2O . Additionally, the cuff pressure is not continuously regulated when using conventional methods. According to the study performed by Nilanjan Dey et al, at Operation and Intensive, Region Hospital Gødstrup[41], a proper seal is important to mitigate the risks of VAP. Therefore, continuous cuff pressure monitoring could be of significance.

14.2 Results

The project's objective, as stated in the problem statement, was to develop a system that could monitor cuff pressure over time. This would enable analysis to facilitate quality control in the ICU by paying attention to how much cuff pressure can change over time, which has the potential to cause complications such as VAP. As a result, the CPATool has been developed as an analysis tool and a tool to be used in daily practice in the ICU. The purpose of the CPATool is, as stated, to provide an overview of the different patients' cuff pressure and how it changes over time, while also giving insight into which external factors can affect the cuff pressure. The CPATool allows the chief physician to get a statistical overview of multiple selected Treatment IDs and to analyze the individual Treatment ID. The test results as seen in the tables: 13.8, 13.9, 13.10, 13.11, are obtained through black box testing, involving unit tests, integration tests, and system tests. The tables show that 17 out of 25 system requirements have been met during testing. Similarly, it shows that 1 out of 3 additional system requirements are fulfilled. The tests for the database requirements show that 10 out of 15 requirements are met. Additionally, all must-have requirements are met except for one, which indicates that the

most important features have been implemented, thereby demonstrating that the CPATool addresses the problem statement thoroughly.

The project's development process has utilized the V-model to provide an overview of the entire process. The V-model was used as a tool in the planning of what the project should include and in what order it should be done. The V-model has helped give structure, ensuring that all elements of the software development have been covered. Additionally, as a part of the development process, OOM has been used to provide an object-oriented approach to the analysis, design, and programming of the CPATool. In the project, the analysis and design process has been more prominent than the programming. It involved developing the analysis and design diagrams, use cases, activity diagrams, and class diagrams. These diagrams were seen as the most beneficial for providing an overview and showing the flow of the system. Using these diagrams, gave more structure to the process and a shared understanding of what the CPATool should include. The diagrams have contributed to providing insight into the iterative process the system has endured in terms of additions and exclusions.

14.3 Problem Solving

The CPATool supports the deficiency of an information software system for monitoring cuff pressure at the ICU. The CPATool facilitates the comparison of multiple Treatment IDs, thereby making workflow-changing decisions based on trends when comparing Treatment IDs. Additionally, the CPATool enables the user to see details and analyze specific Treatment IDs, which could be efficient in daily practice.

The CPATool has been developed to accommodate inaccurate cuff pressure measurements, and possibly prevent consequences such as VAP. Absent cuff pressure monitoring has been diminished by the CPATool's cuff pressure graph. The graph displays cuff pressure over time, in a given time frame, and gives chief physicians a simple way of identifying when the cuff pressure may deviate from the standard range. Furthermore, the statistical overview will assist in identifying possible patterns in fluctuations in cuff pressure, ensuring further quality control in the ICU.

Additionally, the CPATool enables the viewing of the external factors, flush, position changes, and cuff leakage, that may affect the cuff pressure. These events are all displayed beside the cuff pressure graph along with a timestamp, as well as on the statistical overview. This will assist the chief physician in identifying events that may lead to either higher or lower cuff pressure, and thereby potentially encourage implementation of relevant policies and protocols. For example, if position changes consistently lead to pressure spikes, adjustments in patient handling protocols can be made.

Lastly, the CPATool is developed with the TrachFlush device in mind. The intention has been to develop a system that emphasize the importance of continuous cuff pressure monitoring. Using the

TrachFlush device, the aim is further to make implementing a cuff pressure maintenance system an obvious choice for chief physicians. The combination of continuous monitoring with the CPATool as well as automatic adjustments done by the TrachFlush device, should help decrease the occurrence of cuff pressure residing outside the recommended range.

Although, CPATool addresses the main issues presented in the problem analysis, a few limitations are present, some of which will be elaborated and discussed in the following section.

14.3.1 Limitations of the system

The algorithms, used to identify events, originating from SR11 and SR21 in table 9.1, are based on the data gathered from the pilot studies in appendix B.3 and B.4. The pilot studies were performed first and foremost to simulate data, with which it would be possible to design the algorithms. The following sections will discuss the process of obtaining said data, how the data collection could be improved, and subsequently, how it would improve the design of the algorithms.

Position Change algorithm

In the pressure test in appendix B.3, the simulated trachea was exposed to pressure every 30 seconds for 5 seconds. The pressure should emulate the pressure that the trachea would be exposed to during a position change. The test visualizes how the cuff pressure changes and regulates back to the set cuff pressure which indicates the expected regulation of the cuff pressure during a position change.

Although usable data was obtained through the test, it could be possible to obtain even more realistic data, by altering the setup of the test. In the study by Chenelle et al. (2013) [80], an anatomically correct mannequin head, was attached to a test lung and then intubated. Once intubated, data was collected as the mannequin head was moved in both lateral and vertical directions. A similar setup would most likely prove to provide more true-to-reality data for the pilot studies in this project.

Additionally, the pilot studies were recorded using only one setting on the mechanical ventilator, which is not ideal for simulating realistic data. Since ventilators are typically adjusted to meet the individual needs of each patient, a single set of settings cannot represent a broader patient group.

The position change algorithm was designed based on the obtained data and detects when cuff pressure exceeds the set pressure by more than 1 cm H_2O . A simple enhancement for this algorithm would be to issue an event only the first time an over-pressure is detected, either until it returns to normal or for each breath. Currently, the user is informed of an event, for every data point above the set cuff pressure. However, if the cuff pressure warning is issued only once, there is a risk it might be forgotten if it appears as an old warning even though it is still an ongoing event. Alternatively, if the event

repeats for each breath, it may clutter the event list, potentially obscuring other important events.

Leakage algorithm

In the leakage test in appendix B.4, a small leakage was simulated with a 0.8 mm syringe, placed in the cuff right before the recording was started. The recording showed a clear difference in the pattern of the signal. The signal fluctuated a great deal more than in the baseline pilot study. This provided a good indication of which parameters an algorithm for detecting a cuff leakage should focus on. The study may, once again, not accurately represent how changes in cuff leakage would affect cuff pressure in real-world scenarios, although it was decided that the pilot study was the best way to emulate real-life data.

The TrachFlush device used for this project is a developmental device, meaning it can be connected to a computer to provide live data through a script developed by AW Technologies. Although it is possible to retrieve data from the TrachFlush device, not all types of data are currently accessible. Specifically, it is not possible to obtain any data related to the pump incorporated in the TrachFlush device. It would have been useful to see how much air the TrachFlush device pumps into the cuff over time. This information is expected to provide relevant insights into cuff leakage. If a leak occurs, the device will attempt to fill the cuff to the manually set cuff pressure, thereby compensating for the leakage. This would likely be a better parameter for developing the cuff leakage algorithm, as it is expected that there would not be multiple similar conditions causing high-frequency pumping over an extended period.

While the performed pilot studies may not have been a replica of how real-life data would look, the studies provided enough similar data, which was used for the development of the CPATool. It might also be relevant to test if other circumstances during endotracheal intubation could produce similar results to the current events. This could help refine the position change algorithm to detect only position changes and possibly develop algorithms for other potential conditions.

14.3.2 Unfulfilled requirements

While some parts of the CPATool may have provided opportunities for further development, others were intentionally left out of the implementation entirely. This is due to the need to prioritize the parts most important for the general usability of the system. As so, some requirements from this project have not been fulfilled. The most central of these will be discussed in the coming sections.

In CPATool, the user has a list of Treatment IDs beside the dashboards. The list consists solely of Treatment IDs, which may cause difficulties for users who want to analyze data belonging to a specific Patient ID or TrachFlush ID. To reduce the difficulties and obtain SR13 seen in table9.1, a search

function could be incorporated in CPATool. In the search function, the user has the opportunity to search for a Patient ID, TrachFlush ID, Treatment ID, or date. The user will subsequently be informed of the combination of IDs existing under that search. This will allow the user to analyze cuff pressure trends for a specific TrachFlush device which may lead to an investigation of a particular TrachFlush device. Furthermore, the search function allows the user to analyze data from a specific patient who may have been hospitalized multiple times with different TrachFlush IDs to identify trends in the course of hospitalization.

The requirement SR10, 9.1, calls for the option to adjust the time interval of the presented data. A way of implementing this could be by incorporating a calendar feature that restricts the display of events and graphs within a specified time window. This functionality proves advantageous for daily practice and in-depth analysis, empowering users to control the time frame for the viewed data. The calendar can be modeled after the prototype shown in figure 10.3. This addresses the issue in the graph when a treatment extends over multiple days, potentially overwhelming the user with excessive data that may not be useful.

15. Future perspective

To fully implement the CPATool in a medical setting, obtaining the CE marking is a crucial step, as mentioned in section 8.4. As the CPATool is categorized as class IIa, per MDR 2017/745, a notified body would have to be involved in the process of getting approved for the CE mark. To obtain the CE mark, one process that is important to consider is the clinical evaluation. One such evaluation will not only support CE marking, but will also potentially contribute with significant knowledge on cuff pressure management, and its effect on endotracheal intubated patients.

Clinical evaluation will contribute to both the regulatory and research domains. By assessing the safety and performance of the CPATool, these evaluations will reinforce the findings on poorly managed cuff pressure and its consequences, such as VAP linked to underinflated cuffs. The CPATool can facilitate research on VAP prevention by offering healthcare professionals detailed insight into factors affecting cuff pressure.

Additionally, clinical evaluation of the CPATool will likely improve its performance, as real-life data will improve the design of the software significantly. While pilot studies have been performed to simulate data for the software, it is difficult to emulate the exact behavior of an intubated patient. Real-life data will ensure that the algorithms designed to detect various events are more precise in their functionality, and ensure that the entire system functions as intended.

The CPATool also highlights the benefits of using cuff measurements and regulating technologies, as conventional measurement methods often fail to maintain correct cuff pressure, as elaborated in section 3.1. Although the CPATool currently supports only TrachFlush data, expanding its capabilities to monitor non-regulated cuff pressure would add substantial value. This enhancement would benefit both the CPATool and the broader research on TrachFlush device usage.

A function that could improve the CPATool would be the opportunity to update the dataset. The current system updates the dataset only upon initial loading when launched. For optimal daily operations, it is crucial for CPATool to access live data. This could be achieved through automatic periodic updates. Investigating the optimal time interval between updates to ensure timeliness and relevance while minimizing system strain is essential. An alternative approach could involve incorporating an 'Update' button, allowing users to refresh the system as needed, thereby optimizing system performance and reducing load. This solution eliminates the automatic updating, prompting the user to

update manually which can lead to accidental analysis of outdated data.

The CPATool could also be improved by indicating the status of each Treatment ID, whether active or inactive. This would clarify whether data is currently being added to a Treatment ID or if it has ended and no further data is expected. This can be achieved either by an algorithm determining the status based on data activity or by allowing users to manually mark treatments as completed.

If users manually mark Treatment IDs as completed, this feature could be integrated into the Treatment ID list or the header in Dashboard 2. If an algorithm is used, it could set a Treatment ID to inactive after a certain period without data. This feature would require an additional column in the Treatment ID table within the database, indicating the active status of each Treatment ID.

Additionally, a filtering option could be included in the Treatment ID list, enabling users to easily filter out inactive Treatment IDs. This would be useful in daily practice, focusing attention only on active Treatment IDs. Conversely, users could choose to view only completed Treatment IDs for comprehensive analysis.

By addressing these future requirements and improvements, the CPATool can achieve better integration, and utility in medical settings, ultimately contributing to regulatory compliance and enhanced patient care.

16. Conclusion

Based on the problem analysis, it has become apparent, that there is a significant lack of monitoring cuff pressure in endotracheal intubated patients. It is shown that cuff pressure often resides outside the recommended range of 20-30 cm H_2O , and a variety of complications may occur. Taking into account, the problem analysis and derived problem delineation, the following problem statement was formulated:

How can a system including a database be developed to monitor cuff pressure over time and thereby enable an analysis to facilitate quality control in the ICU related to maintaining cuff pressure?

In order to accommodate the problem statement a system, CPATool, has been developed. With the knowledge from the problem analysis, and in collaboration with a chief physician in the ICU and the developers of the TrachFlush device, AW Technologies, it is feasible to conclude that such a system will add significant value when monitoring cuff pressure regularly. Lastly, it is believed that the CPATool will help emphasize the need for further research on the subject of cuff pressure management in general, as well as its many complications if mismanaged.

Bibliography

- [1] RA Ahmed and TJ Boyer. “Endotracheal Tube.[Updated 2023 Jul 24]”. In: *StatPearls [Internet]. Treasure Island (FL): StatPearls Publishing* (2023). URL: <https://www.ncbi.nlm.nih.gov/books/NBK539747/> (visited on 02/10/2024).
- [2] Mary Lou Sole et al. “Evaluation of an intervention to maintain endotracheal tube cuff pressure within therapeutic range”. In: *American Journal of Critical Care* 20.2 (2011), pp. 109–118. DOI: 10.4037/ajcc2011661.
- [3] Laurent Papazian, Michael Klompas, and Charles-Edouard Luyt. “Ventilator-associated pneumonia in adults: a narrative review”. In: *Intensive care medicine* 46.5 (2020), pp. 888–906.
- [4] Cihan Semet. “The ongoing challenge of ventilator-associated pneumonia: epidemiology, prevention, and risk factors for mortality in a secondary care hospital intensive care unit”. In: *Infection Prevention in Practice* 5.4 (2023), p. 100320. ISSN: 2590-0889. DOI: <https://doi.org/10.1016/j.infpip.2023.100320>. URL: <https://www.sciencedirect.com/science/article/pii/S2590088923000537>.
- [5] Yun Wang et al. “National trends in patient safety for four common conditions, 2005–2011”. In: *New England Journal of Medicine* 370.4 (2014), pp. 341–351. DOI: 10.1056/NEJMsa1300991.
- [6] Steven M Koenig and Jonathon D Truwit. “Ventilator-associated pneumonia: diagnosis, treatment, and prevention”. In: *Clinical microbiology reviews* 19.4 (2006), pp. 637–657. DOI: 10.1128/CMR.00051-05. (Visited on 02/14/2024).
- [7] Jean-Louis Vincent and Jacques Creteur. “The critically ill patient”. In: *Critical Care Nephrology*. Elsevier, 2019, pp. 1–4. (Visited on 02/14/2024).
- [8] Gabriela Alvarado et al. “Principles of Mechanical Ventilation”. In: *Personalized Mechanical Ventilation : Improving Quality of Care*. Ed. by Jorge Hidalgo et al. Cham: Springer International Publishing, 2022, pp. 19–35. ISBN: 978-3-031-14138-6. DOI: 10.1007/978-3-031-14138-6_2. URL: https://doi.org/10.1007/978-3-031-14138-6_2.
- [9] Hooman Poor. “Respiratory Mechanics”. In: *Basics of Mechanical Ventilation*. Cham: Springer International Publishing, 2018, pp. 1–10. ISBN: 978-3-319-89981-7. DOI: 10.1007/978-3-319-89981-7_1. URL: https://doi.org/10.1007/978-3-319-89981-7_1.

- [10] Kristin Walter. “Mechanical Ventilation”. In: *JAMA* 326.14 (Oct. 2021), pp. 1452–1452. ISSN: 0098-7484. DOI: 10.1001/jama.2021.13084. URL: <https://doi.org/10.1001/jama.2021.13084> (visited on 02/14/2024).
- [11] Iordan Potchileev, Maksym Doroshenko, and Asif N. Mohammed. “Positive Pressure Ventilation”. In: (2023). URL: <https://www.ncbi.nlm.nih.gov/books/NBK560916/> (visited on 02/14/2024).
- [12] Anthony J Raimonde, Natalie Westhoven, and Ryan Winters. “Tracheostomy”. In: (2020). (Visited on 02/14/2024).
- [13] Joseph F. Dasta et al. “Mechanical Ventilation”. In: *DICP* 24.10 (1990), pp. 959–970. DOI: 10.1177/106002809002401011. (Visited on 02/13/2024).
- [14] Tracheostomy Education. *Endotracheal Intubation*. 2021. URL: <https://tracheostomyeducation.com/endotracheal-intubation/> (visited on 03/12/2024).
- [15] Jennifer Whitlock. “What is Intubation”. In: (2023). URL: <https://www.verywellhealth.com/what-is-intubation-and-why-is-it-done-3157102> (visited on 02/13/2024).
- [16] Karin.R Pedersen and Karin Kaasby. *Cuff-trykmåling hos voksne patienter på intensive afsnit*. 2024. URL: <https://pri.rn.dk/Sider/14947.aspx> (visited on 02/09/2024).
- [17] Anæstesiologisk afdeling and Aarhus Universitetshospital. *Klinisk retningslinje for trachealsugning af den voksne intuberede patient*. 2022. (Visited on 02/19/2023).
- [18] Hanne A Mouritsen, Karin R Pedersen, and Kirsten D Jensen. *Endotrakeal sugning hos voksen intuberet/trakeostomeret intensiv patient*. 2022. URL: <https://pri.rn.dk/Sider/13532.aspx> (visited on 02/19/2023).
- [19] Kelly M Moon et al. “Endotracheal Tube Cuff Pressures in the Operating Room of a Pediatric Hospital: A Quality Improvement Initiative”. In: (2022). DOI: 10.1097/pq9.0000000000000619. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9742117/>.
- [20] Janet T Crimlisk et al. “Artificial airways: A survey of cuff management practices”. In: *Heart and Lung* 25 (1996). URL: [https://www.heartandlung.org/article/S0147-9563\(96\)80033-1/pdf](https://www.heartandlung.org/article/S0147-9563(96)80033-1/pdf) (visited on 02/13/2024).
- [21] Scott L Stewart et al. “A Comparison of Endotracheal Tube Cuff Pressure Using Estimation Techniques and Direct Intracuff Measurement”. In: *AANA Journal* 71 (2003), p. 443. URL: <https://pubmed.ncbi.nlm.nih.gov/15098531/> (visited on 02/13/2024).
- [22] DA Harvie et al. “The minimal leak test technique for endotracheal cuff maintenance”. In: *Anaesth Intensive Care* (2016). DOI: 10.1177/0310057X1604400512. URL: <https://pubmed.ncbi.nlm.nih.gov/27608343/>.

- [23] Mary Lou Sole et al. "Assessment of endotracheal cuff pressure by continuous monitoring: a pilot study". In: *American Journal of Critical Care* 18.2 (2009), pp. 133–143.
- [24] Mueen Ullah Khan et al. "Measurement of endotracheal tube cuff pressure: Instrumental versus conventional method". In: *Saudi journal of anaesthesia* 10 (2016), p. 428. DOI: 10.4103/1658-354X.179113.
- [25] Mary Lou Sole et al. "Evaluation of an intervention to maintain endotracheal tube cuff pressure within therapeutic range". In: *American Journal of Critical Care* 20.2 (2011), pp. 109–118. DOI: 10.4037/ajcc2011661.
- [26] Mohammad El-Orbany and M Ramez Salem. "Endotracheal tube cuff leaks: causes, consequences, and management". In: *Anesthesia & Analgesia* 117.2 (2013), pp. 428–434. DOI: 10.1213/ANE.0b013e318292ee21.
- [27] Steven M Koenig and Jonathon D Truwit. "Ventilator-associated pneumonia: diagnosis, treatment, and prevention". In: *Clinical microbiology reviews* 19.4 (2006), pp. 637–657. DOI: 10.1128/CMR.00051-05.
- [28] Elisabeth Lund. "Forebyggelse af ventilatorassocieret pneumoni hos intuberede patienter". In: *Sygeplejersken* 8 (2007), pp. 38–45. URL: <https://dsr.dk/fag-og-udvikling/sygeplejersken/arkiv/sygeplejersken-2007-8/forebyggelse-af-ventilatorassocieret-pneumoni-hos-intuberede-patienter/> (visited on 02/12/2024).
- [29] Amir Jalali, Zohreh Maleki, and Mohammadreza Dinmohammadi. "The Effect of Different Body Positions on Endotracheal Tube Cuff Pressure in Patients under Mechanical Ventilation". In: *Journal of Caring Sciences* 11.1 (2022), p. 15. DOI: 10.34172/jcs.2022.03.
- [30] Christelle Lizy et al. "Cuff pressure of endotracheal tubes after changes in body position in critically ill patients treated with mechanical ventilation". In: *American Journal of Critical Care* 23.1 (2014), e1–e8. DOI: doi.org/10.4037/ajcc2014489.
- [31] Yamil Selman et al. "Accuracy of the minimal leak test for endotracheal cuff pressure monitoring". In: *The Laryngoscope* 130.7 (2020), pp. 1646–1650. DOI: 10.1002/lary.28328.
- [32] Moriz Ettore Klonner et al. "Disposable Airway Pressure Manometers for Endotracheal Tube Cuff Inflation". In: *Animals* 13.3 (2023), p. 475. DOI: 10.3390/ani13030475. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9913048/>.
- [33] Ambu. *Cuff Pressure Gauge*. URL: <https://www.ambuusa.com/airway-management-and-anaesthesia/laryngeal-masks/product/ambu-cuff-pressure-gauge> (visited on 02/14/2024).
- [34] Roghieh Nazari et al. "The changes of endotracheal tube cuff pressure during manual and intermittent controlling in intensive care units". In: *Iranian Journal of Nursing and Midwifery Research* 25.1 (2020), p. 71. DOI: 10.4103/ijnmr.IJNMR_55_19.

- [35] *IntelliCuff*. URL: https://www.hamilton-medical.com/en_DK/Products/IntelliCuff.html (visited on 02/08/2024).
- [36] *Puritan Bennett™ Cuff Pressure Manager*. URL: <https://www.medtronic.com/covidien/en-us/products/mechanical-ventilation/accessories/puritan-bennett-cuff-pressure-manager.html> (visited on 02/08/2024).
- [37] *TrachFlush*. URL: <https://www.trachflush.com/> (visited on 02/08/2024).
- [38] Medtronic. *Quick Reference Guide Puritan Bennett™ cuff pressure manager*. 2023-02. (Visited on 02/08/2024).
- [39] Luigi vivona et al. “In Vivo Evaluation of a New Endotracheal Tube Cuff Controller Promoting Tracheal Secretion Clearance: Preliminary Results”. In: *Respiratory Care* 67.Suppl 10 (2022). ISSN: 0020-1324. eprint: <https://rc.rcjournal.com/content>. URL: https://rc.rcjournal.com/content/67/Suppl_10/3776321.
- [40] Shahram Nasrolahzadeh et al. “Comparison of the effect of pressure control and volume control ventilation on endotracheal tube cuff pressure in patients undergoing general anesthesia and mechanical ventilation: a parallel randomized clinical trial”. In: *BMC anesthesiology* 23.1 (2023), p. 300. (Visited on 02/14/2024).
- [41] Nilanjan Dey, Strange GF, and Nielsen BH. *Usage of cuff pressure measurement on intubated patients in the intensive care unit*. A poster at Operation og Intensiv , Regionshospitalet Gødstrup.
- [42] JavaTpoint. *V-Model*. 2021. URL: <https://www.javatpoint.com/software-engineering-v-model> (visited on 12/05/2023).
- [43] Ganesh Regulwar et al. “Variations in V Model for Software Development”. In: (2021). URL: https://www.researchgate.net/publication/348488626_Variations_in_V_Model_for_Software_Development (visited on 03/25/2024).
- [44] Lars Mathiassen et al. *Object Oriented Analysis & Design*. Metodica ApS, 2018.
- [45] Kate Brush. “MoSCoW method”. In: *TechTarget, April* (2020). URL: <https://www.techtarget.com/searchsoftwarequality/definition/MoSCoW-method> (visited on 03/25/2024).
- [46] IBM. *Use-case diagrams*. september 2023. 2023. URL: <https://www.ibm.com/docs/en/rational-soft-arch/9.7.0?topic=diagrams-use-case> (visited on 02/04/2024).
- [47] IBM. *Include relationships*. 2021-03. 2023. URL: <https://www.ibm.com/docs/en/rational-soft-arch/9.6.1?topic=diagrams-include-relationships> (visited on 04/29/2024).
- [48] *Regulation (EU) 2017/745 of the European Parliament and of the Council of 5 April 2017 on medical devices, amending Directive 2001/83/EC, Regulation (EC) No 178/2002 and Regulation (EC) No 1223/2009 and repealing Council Directives 90/385/EEC and 93/42/EEC (Text with EEA relevance.)* May 2017. (Visited on 02/19/2024).

- [49] Johner-Institute. *Technical Documentation for Medical Devices*. URL: <https://www.johner-institute.com/articles/regulatory-affairs/and-more/technical-documentation/> (visited on 05/19/2024).
- [50] ISO. *Standards*. URL: <https://www.iso.org/standards.html> (visited on 04/30/2024).
- [51] B. Karthika and A. R. Vijayakumar. “ISO 13485: Medical Devices – Quality Management Systems, Requirements for Regulatory Purposes”, year=”2022”. In: (), pp. 19–29. DOI: 10.1007/978-3-030-91855-2_2. URL: https://doi.org/10.1007/978-3-030-91855-2_2.
- [52] Dansk standard. *DS/ISO 13485 Medicinsk udstyr*. URL: <https://www.ds.dk/da/om-standarder/ledelsesstandarder/iso-13485-medicinsk-udstyr> (visited on 05/21/2024).
- [53] Pratik M Patel and Vivek A Deshpande. “Application of plan-do-check-act cycle for quality and productivity improvement-A review”. In: *Int. J. Res. Appl. Sci. Eng. Technol* 5.1 (2017), pp. 197–201.
- [54] Thamizharasan Sampath et al. “ISO 14971 and ISO 24971: Medical Device Risk Management”. In: (2022), pp. 31–56. DOI: 10.1007/978-3-030-91855-2_3. URL: https://doi.org/10.1007/978-3-030-91855-2_3.
- [55] Dansk Standards Association. *Medicinsk udstyr – Vejledning i anvendelse af ISO 14971*. 2020-07. 2020.
- [56] Danish Standards Association. *Software for medicinsk udstyr - Livscyklusprocesser for software*. 2006-12. 2006.
- [57] Lægemiddelstyrelsen. *CE-mærkning*. 2023. URL: <https://laegemiddelstyrelsen.dk/da/udstyr/ce-maerkning/> (visited on 02/29/2024).
- [58] Naveen Agarwal. *ISO 14971: Harnessing Preliminary Hazard Analysis (PHA) To Develop Safer Medical Devices*. 2020. URL: <https://www.meddeviceonline.com/doc/iso-harnessing-preliminary-hazard-analysis-pha-to-develop-safer-medical-devices-0001> (visited on 04/18/2024).
- [59] Kyungmee O. Kim and Ming J. Zuo. “General model for the risk priority number in failure mode and effects analysis”. In: *Reliability Engineering & System Safety* 169 (2018), pp. 321–329. DOI: <https://doi.org/10.1016/j.ress.2017.09.010>.
- [60] AW Technologies. *Instructions for Use*. 2023-09-14. 2023. (Visited on 04/30/2024).
- [61] Sean M Hickey and Al O Giwa. “Mechanical ventilation”. In: (2019).
- [62] Datatilsynet. *Hvad er personoplysninger?* URL: <https://www.datatilsynet.dk/hvad-siger-reglerne/grundlaeggende-begreber/hvad-er-personoplysninger> (visited on 03/26/2024).

- [63] Datatilsynet. *Pseudonymisering og anonymisering*. URL: <https://www.datatilsynet.dk/hvad-siger-reglerne/vejledning/sikkerhed/katalog-over-foranstaltninger/pseudonymisering-og-anonymisering> (visited on 03/26/2024).
- [64] Coursera. *What Is Python Used For? A Beginner's Guide*. 2024. URL: <https://www.coursera.org/articles/what-is-python-used-for-a-beginners-guide-to-using-python> (visited on 04/12/2024).
- [65] Avi Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. 6th edition. McGraw-Hill Education, 2010.
- [66] John G. Webster and Amit J. Nimunkar. *Medical instrumentation*. 5. Edition. John Wilay & Sons, Inc, 2020.
- [67] Miriam Walker et al. "High-Fidelity or Low-Fidelity, Paper or Computer Choosing Attributes When Testing Web Prototypes". In: *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* 46 (Sept. 2002). DOI: 10.1177/154193120204600513.
- [68] IBM. *Activity diagrams*. 2023-09. 2023. URL: <https://www.ibm.com/docs/en/rational-soft-arch/9.7.0?topic=diagrams-activity> (visited on 04/18/2024).
- [69] IBM. *Class diagrams*. 2023-09. 2023. URL: <https://www.ibm.com/docs/en/rational-soft-arch/9.7.0?topic=diagrams-class> (visited on 04/19/2024).
- [70] Gandharba Swain. *Object-oriented analysis and design through unified modeling language*. Laxmi Publications, 2010.
- [71] tutorialspoint. *MVC Framework - Introduction*. URL: https://www.tutorialspoint.com/mvc-framework/mvc_framework_introduction.htm (visited on 04/15/2024).
- [72] tutorialspoint. *MVC Framework - Architecture*. URL: https://www.tutorialspoint.com/mvc-framework/mvc_framework_architecture.htm (visited on 05/19/2024).
- [73] Geeksforgeeks. *Class Diagram, Unified Modeling Language (UML)*. URL: <https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/> (visited on 05/23/2024).
- [74] *How to write Test Cases- Software Testing*. Accessed: 14-05-2024. 2023. URL: <https://www.geeksforgeeks.org/test-case/?ref=lbp>.
- [75] GeeksforGeeks. *What is Software Testing?* 2024. URL: <https://www.geeksforgeeks.org/software-testing-basics/> (visited on 05/17/2024).
- [76] GeeksforGeeks. *Black Box Testing – Software Engineering*. 2024. URL: <https://www.geeksforgeeks.org/software-engineering-black-box-testing/> (visited on 05/17/2024).
- [77] GeeksforGeeks. *Functional Testing – Software Testing*. 2023. URL: <https://www.geeksforgeeks.org/software-testing-functional-testing/> (visited on 05/17/2024).

- [78] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2015. ISBN: 0133943038.
- [79] *Acceptance Testing - Software Testing*. Accessed: 14-05-2024. 2023. URL: <https://www.geeksforgeeks.org/acceptance-testing-software-testing/>.
- [80] Christopher T Chenelle et al. “Evaluation of an Automated Endotracheal Tube Cuff Controller During Simulated Mechanical Ventilation”. In: *Respiratory Care* 60.2 (2015), pp. 183–190. ISSN: 0020-1324. DOI: 10.4187/respcare.03387. eprint: <https://rc.rcjournal.com/content/60/2/183.full.pdf>. URL: <https://rc.rcjournal.com/content/60/2/183>.

Part VII

Appendix

A. Risk Management Plan

A.1 References

- DS/EN ISO 14971:2019 Medical Devices - Application of Risk Management to Medical Devices
- ISO/TR 24791:2020 Medical Devices - Guidance on the Application of ISO 14971
- IEC 62304:2006 - Medical Device Software - Software Life Cycle Processes

A.2 Scope

The scope of the risk management plan is to outline the steps necessary to properly identify and process risks associated with CPATool. This is to ensure that safety is incorporated into the design of the system. Further details concerning the steps of the plan, will be provided in the document.

A.3 Methodology

The risk management process will adhere to the chosen sections of the Risk Management Process outlined in ISO 14971:2019. This is a 12-step plan that will be implemented, which can be seen in table A.1. The implementation of this plan will be recorded in the Risk Management Document. The table A.1 presents the headings of the plan alongside the corresponding sections in the Risk Management Document where each step is documented.

Step #	Steps in plan	Risk Management Document
1	Intended Use and Device Characteristics	
2	Identify Hazards	
3	Estimate the Risk Factor for Each Hazardous Situation	
4	Native Risk Evaluation	
5	Identify Risk Control Measures	
6	Implementation of Risk Reduction Measures	
7	Residual Risk Evaluation	
8	Risk Benefit Analysis	
9	Risks Arising from Risk Control Measures	
10	Completeness of Risk Control	
11	Overall Residual Risk Evaluation	
12	The Risk Management Review	

Table A.1: Overview of the 12-step plan

A.4 Assignment of Responsibilities and Authors

The responsibilities of performing the risk management activities fall to the developers of the CPATool, as they have the experience and technical knowledge necessary. Furthermore, the developers will document and review any and all risk management activities.

A.5 Risk Acceptability Criteria

Every hazard in the risk management review will be classified as either minor, moderate or major. Hazards classified as minor, will be accepted immediately, hazards classified as moderate will be accepted with justification and lastly, hazards classified as major will not be accepted, and further subjected to a Risk Benefit Analysis (See section A.6, step 8).

A.6 Detailed Descriptions of Steps in the Risk Management Plan

1. Intended Use and Device Characteristics

The intended use of the medical device will be documented in the risk management file, as per ISO 14971:2019 section 5.2. Furthermore, any reasonably foreseeable misuse will be documented as well.

As the CPATool is software, it will be classified according to IEC 62304:2006, section 4.3. Further, section 7, *Software Risk Management Process* will be followed in combination with the ISO 14971:2019 risk management process.

2. Identify Hazards

In section 8.7.1, the method for identifying hazards is described in accordance with the risk analysis. The hazards will be placed in an Excel file with the hazards associated with the medical device. The hazard column of the Risk Analysis is specific to CPATool.

3. Estimate the Risk Factor for Each Hazardous Situation

The ISO 14971:2019 requires the manufacturer to perform risk estimation. Based on the guidance of the application of ISO 24971:2020 is the risks expressed in terms that facilitate decision making on risk acceptability and the need for risk control, therefore the severity and probability scales are used for the risk classification. The scale contains three levels which are ranked from negligible to catastrophic and from improbable to frequent.

The severity level is ranked as:

- **S1 (Negligible)**: Any damage to the medical equipment, excessive waiting time, and inconvenience exists.
- **S2 (Moderate)**: Reversible or minor injury.
- **S3 (Catastrophic)**: Loss of function or structure or death.

The probability levels of the occurrence are ranked as:

- **O1 (Improbable)**: Unlikely to occur within 5 years of the medical device's lifespan.
- **O2 (Occasional)**: Can occur but not frequently, it is likely to happen a few times within 5 years of the medical device's lifespan.
- **O3 (Frequent)**: Likely to occur several times within 5 years of the medical device's lifespan.

To estimate each risk the Risk Priority Number (RPN) is calculated as the product of the severity (S) and the probability of occurrence (P). The RPN is defined as:

$$RPN = P \cdot S \quad (\text{A.1})$$

By integrating probability and severity a Risk Assessment Matrix is developed, seen in figure A.1. This table offers manufacturers a structured overview of risk levels. Utilizing color-coded indicators: red for high risk, yellow for moderate risk, and green for low risk - users gain a visual representation facilitating informed decision-making.

Probability of Occurrence	Severity		
	Negligible 1	Moderate 2	Catastrophic 3
Frequent 3	3	6	9
Occasional 2	2	4	6
Improbable 1	1	2	3

Figure A.1: Risk Assessment Matrix indicating the risk levels divided in color-coded indicators: red = high risk, yellow = moderate risk, and green = low risk.

4. Native Risk Evaluation

See section 6 in ISO 14971:2019. The risk acceptability criteria are defined through the use of ISO 24791:2020, Annex C. The risk acceptance criteria are seen in table A.2.

RPN Score	Classification	Risk Acceptability Category
$0 < RPN \leq 4$	Minor	Acceptable
$4 < RPN \leq 6$	Moderate	Acceptable with justification
$6 < RPN \leq 9$	Major	Unacceptable

Table A.2: The table describes the risk acceptability criteria with the RPN score, classification, and risk acceptability category.

5. Identify Risk Control Measures

See section 7.1 in ISO 14971:2019. Determining risk control measures that are appropriate for reducing risks to an acceptable level, will be done through the use of ISO 24791:2020, section 7.1 and IEC 62304:2006, section 7.2.1.

6. Implementation of Risk Reduction Measures

The implementation of risk control measures, as well as the effectiveness of said risk control measures, will be verified and documented as per ISO 14971:2019 section 7.2. Verification will be done through design documentation and testing of risk control measures. Traceability between identified risks and risk control measures will be documented as well.

7. Residual Risk Evaluation

See ISO 14971:2019 Section 7.3. The residual risk evaluation will follow the same procedure as step 4 Native risk evaluation, in the risk management plan. Residual risks will be deemed either acceptable or unacceptable. Unacceptable risks will be subject to further risk control, and if this is not possible, a benefit-risk analysis will be performed.

8. Risk Benefit Analysis

If risk control measures of the residual risks are not possible, a benefit-risk analysis will be performed. The benefit-risk analysis will determine, through a review of data and literature, if the benefits of the intended use outweigh the residual risks, as determined in step 7 Residual Risk Evaluation.

9. Risks Arising from Risk Control Measures

See ISO 14971:2019 Section 7.5. If new hazards or hazardous situations should arise, or risk control measures affect any existing hazards or hazardous situations, these risks will be reviewed once more according to steps 2-8 in the risk management plan.

10. Completeness of Risk Control

See ISO 14971:2019 Section 7.6. To ensure all risks have been considered and all risk control activities are complete, the risk control in its entirety will be reviewed.

11. Overall Residual Risk Evaluation

See ISO 14971:2019 Section 8. The overall residual risk will be shown in a risk matrix, in order to present a graphic visualization of the distribution of risks. In conjunction with this matrix, an overall residual risk evaluation will be determined, in order to deem the overall risks acceptable or

unacceptable. If the risks are deemed acceptable, users will be informed of significant risks in the accompanying documentation. If for any reason the overall residual risks are deemed unacceptable in relation to the benefits of the intended use, additional risk control measures will be implemented.

12. The Risk Management Review

See ISO 14971:2019 Section 9. The risk management review will be implemented, as by the steps in the risk management plan, and will be documented in the risk management file. The risk management review includes a review of the outcome of the risk management plan which includes: risk analysis, risk evaluation, and risk control. As part of the risk management review, the overall risk table, seen in figure A.2, has been created to show the different hazards that could arise from the use of the CPATool.

Risk Analysis						Native Risk Evaluation				Risk Control			
Hazard ID	Hazard Group	Hazard	Cause of Hazardous situation	Sequence of events	Hazard situation	Harm	Severity	Probability	RPN	Risk Reduction measures	Severity	Probability	Residual risk evaluation
H1	Data	Failed data access	No internet connection	1. User opens the program. 2. The connection to the internet can not be established.	The user has no data available for analysis.	Loss of medical device functionality.	S1	O2	Acc(2)	Not assessed.			
H2	Data	Failed data access.	No database connection.	1. User opens the program. 2. The connection to the database can not be established.	The user has no data available for analysis.	Loss of medical device functionality.	S1	O2	Acc(2)	Not assessed.			
H3	Data	Limited data.	Not enough data available.	1. User opens the program. 2. The user accesses a tab for a chosen TreatmentID 3. Inefficient amount of data available for the chosen TreatmentID for analysis	Misinterpretation of the analysis	Wrongful decisions. Deterioration of health.	S2	O3	Acc(6)	Adding requirement SR24: The system must issue a warning message if the patient has not been connected to a TrachFlush for at least 1 hour.	S2	O2	Acc(4) The hazards is now acceptable.
H4	Diagnostic information	Patient traceability	Lack of traceability.	1. User opens the program. 2. The user has the opportunity to add a new TrachFlush and patient connection. 3. The user wrongfully connect a patient to the wrong TrachFlush.	The user get TrachFlush data for another patient og no data for analysis.	Wrongful decisions. Deterioration of health.	S2	O1	Acc(2)				
H5	Data	Integrity.	The database get hacked.	1. The database get hacked 2. An unauthorized person get access to the database and manipulates the data.	Misinterpretation of the statistics because of the manipulation of the data.	Wrongful decisions Deterioration of health.	S2	O1	Acc(2)	Not assessed.			
H6	Delivery	Rate.	Too low sample rate.	1. Lack of the connection from the TrachFlush to the database.	Insufficient amount of data available.	Inability to make decisions based on the visualized data.	S2	2	Acc(2)	Not assessed.			
H7	Delivery	Quantity.	Too few data sources.	1. Data is only available from two or fewer TrachFlush devices.	The statistics over the events across multiple TreatmentIDs are not adequate.	Inability to make decisions based on the visualized statistics.	S1	O2	Acc(2)				
H8	Diagnostic information	Patient identity.	Logout has never occurred.	1. User opens the program. 2. The user never logs out after using the program.	The personal data is available for everyone who has access to the computer.	Personal data being exposed.	S2	O3	Acc(6)	Adding requirement SR25: The system must have an automatic screen lock, when the system is left unattended for 5 minutes.	S1	O3	Acc(3) The hazards is now acceptable.

Figure A.2: The outcome of the risk management plan which includes: risk analysis, risk evaluation, and risk control.

B. Pilot Studies

Before the start of the pilot studies, the respirator was prepared by adjusting several settings on the respirator. On the respirator, the positive end-expiratory pressure (PEEP) was set to 5 cm H_2O , the peak airway pressure (Ppeak) was set to 15 cm H_2O , the O_2 concentration was set to 21 %, and PC above PEEP was set to 10 cm H_2O .

Furthermore, a C# code developed by AW Technologies is used for the recording of the pilot studies. In the C# code several parameters are defined related to the TrachFlush device. The C# code is modified by connecting to a database to save the data with the relevant parameters of the pilot studies. The relevant parameters saved from the C# code are timestamp, airway pressure, cuff pressure, flush avg, and system code. These parameters are relevant for the development of the algorithms. In addition to the saved parameters from the C# code, the parameter Treatment ID is added to the C# code and database for indication of the specific treatment which manually should be changed in the code before recording. This is done so that each pilot study can be identified later on.

B.1 Baseline test

Purpose:

The purpose is to record a dataset containing a baseline, thus observing how the cuff pressure appears under normal conditions.

Materials:

- TrachFlush device with associated power cable
- Cuff
- Endotracheal tube
- Computer with C# recording code
- Simulated trachea
- Respirator with associated air tube from a compressor
- Micro-B to USB A cable
- Artificial lung
- Timer

Procedure:

Preparation before recording:

1. Prepare the relevant software to record and connect TrachFlush device.
 - a. Open the software program in Visual Studio and test whether the code can run and that a window opens where parameters from the TrachFlush device can be displayed.
 - b. The TrachFlush device is connected to power with the associated power cable.
 - c. USB cable connects to the TrachFlush device and computer.
 - d. In the window, press 'update' to find the com port used. Then select the port in the scroll bar to the left of 'update'. If the connection has been established correctly, a popup window will appear with a message about this. In the popup window, press 'OK'.
 - e. Click "Login" to access data from the TrachFlush device. By pressing 'get', the data will be visualized in the data box on the left side of the window.
 - f. To get periodic updates of the data, mark 'periodic', whereby the data is also saved in the database every half second.
2. Prepare the simulated trachea by inserting the endotracheal tube, which is done by placing the endotracheal tube with 'deflated' cuff in the trachea.
3. Connecting the simulated trachea with endotracheal tube to the artificial lung, as well as connecting the endotracheal tube to the air tube from the ventilator.
4. Prepare the respirator by connecting the air tube to the respirator and then first turning on the air and then the respirator itself.
5. The manual cuff pressure is set on the TrachFlush device to be 25 cm H_2O .

Setup:

For the recording of baseline, the setup visualized in figure B.1 is used.

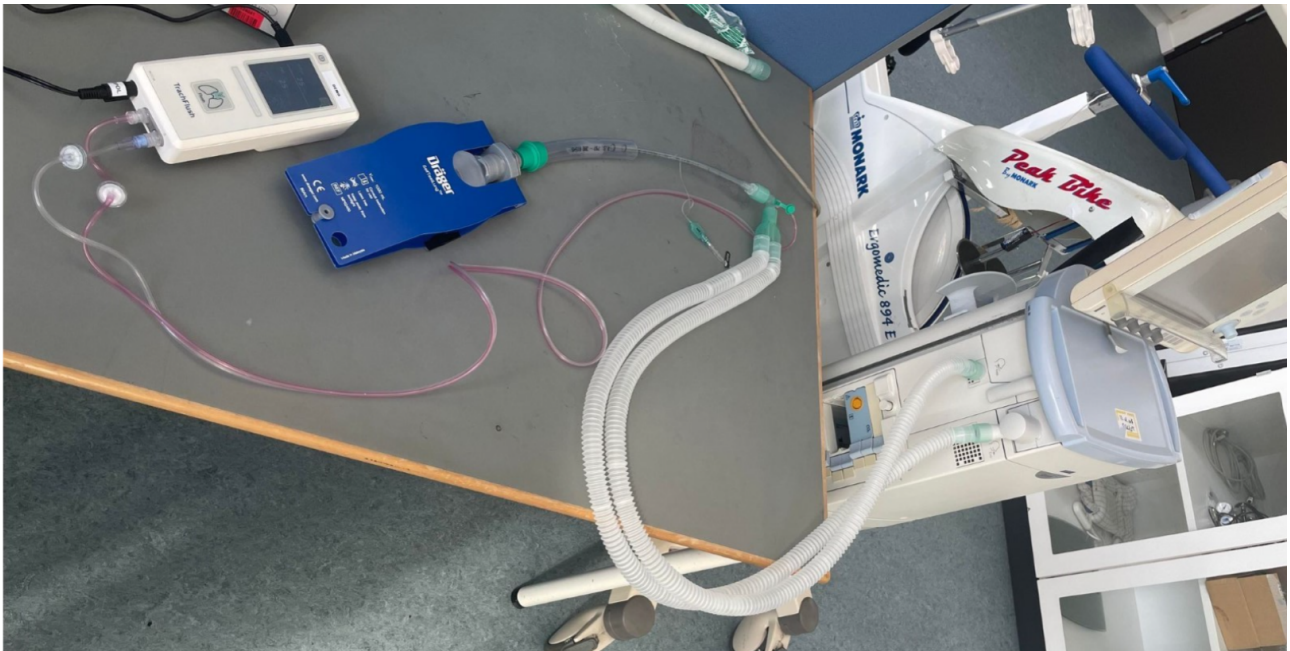


Figure B.1: The setup used to record the test.

Recording:

Starting point: The endotracheal tube lies freely on a table without disturbance or pressure.

1. The trachea is placed at the starting point on the table.
2. It is recorded for 5 minutes without any interactions.

Below is a chart of the recording procedure:

Time in minutes	Movement	Activity
0:00		The trachea lies at the starting point on the table. The recording begins.
5:00		Recording is stopped.

Results

In figure B.2 cuff pressure and airway pressure over time are visualized. It is observed that the measurement for the cuff pressure fluctuates between 26 cm H_2O and 24 cm H_2O , but it is mainly centered at 25 cm H_2O . Additionally, the airway pressure fluctuates from PEEP to Ppeak. Although it appears shifted by 1 cm H_2O , as PEEP is read at 4 cm H_2O instead of the set 5 cm H_2O , and Ppeak is read at 14 cm H_2O instead of the expected 15 cm H_2O .

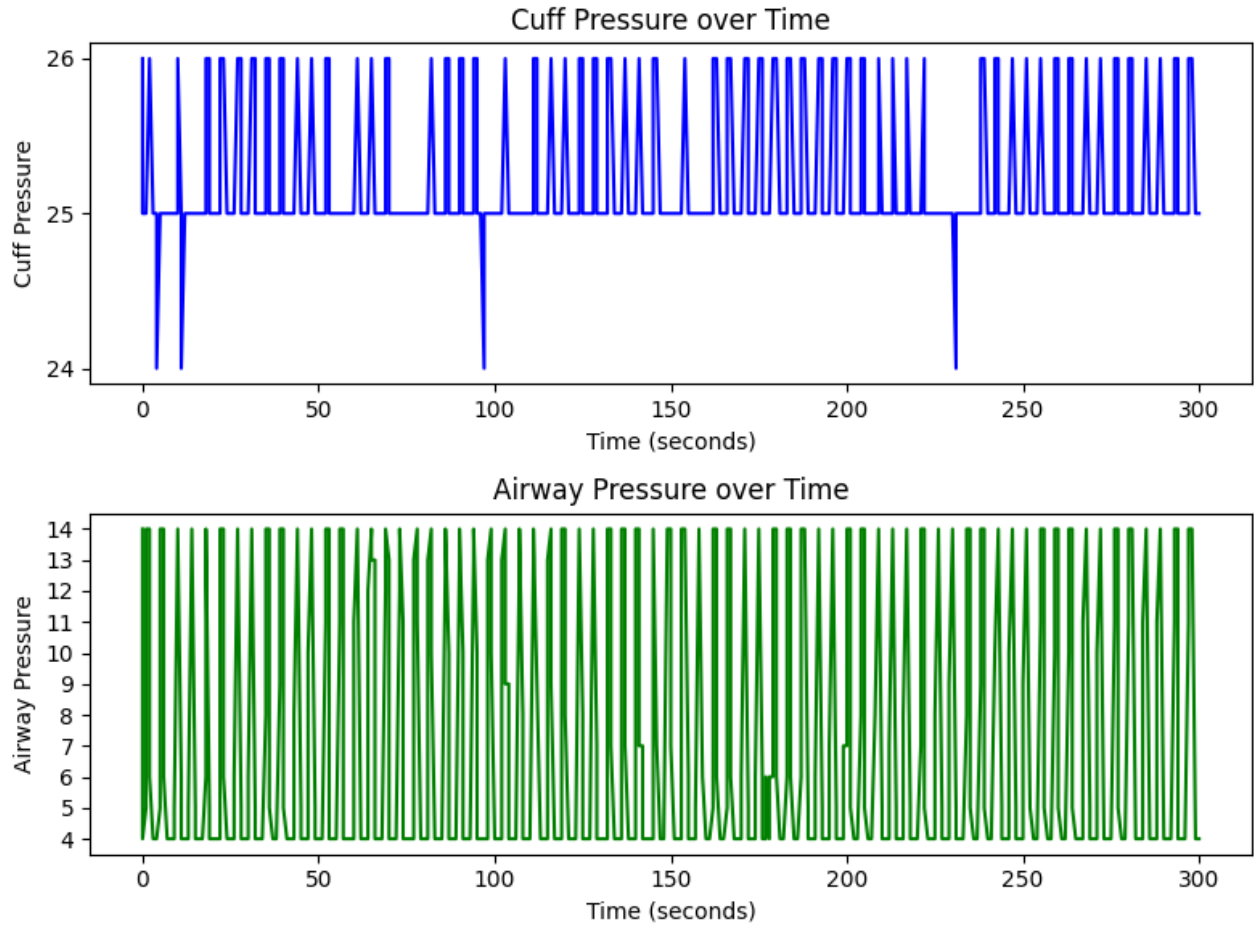


Figure B.2: The visualization of the cuff pressure and airway pressure over time under the baseline test.

Conclusion:

Based on the baseline test, it can be concluded that the airway pressure fluctuates between PEEP and Ppeak and the cuff pressure decreases to 24 cm H_2O and increases to 26 cm H_2O during the recording but is quickly regulated back to 25 cm H_2O .

B.2 Flush test

Purpose:

The purpose is to record datasets where the flush function is used thus observing how the live parameters are changed in connection with the flush function.

Materials:

- TrachFlush with associated power cable
- Cuff
- Endotracheal tube
- Computer with C# recording code

- Simulated trachea
- Respirator with associated air tube from a compressor
- Micro-B to USB A cable
- Artificial lung
- Timer

Procedure:

Preparation before recording:

1. Prepare the relevant software to record and connect TrachFlush device.
 - a. Open the software program in Visual Studio and test whether the code can run and that a window opens where parameters from TrachFlush device can be displayed.
 - b. TrachFlush device is connected to power with the associated power cable.
 - c. USB cable connects to TrachFlush device and computer.
 - d. In the window, press 'update' to find the com port used. Then select the port in the scroll bar to the left of 'update'. If the connection has been established correctly, a popup window will appear with a message about this. In the popup window, press 'OK'.
 - e. Click "Login" to access data from TrachFlush device. By pressing 'get', the data will be visualized in the data box on the left side of the window.
 - f. To get periodic updates of the data, mark 'periodic', whereby the data is also saved in the database every half second.
2. Prepare the simulated trachea by inserting the endotracheal tube, which is done by placing the endotracheal tube with 'deflated' cuff in the trachea.
3. Connecting the simulated trachea with endotracheal tube to the artificial lung, as well as connecting the endotracheal tube to the air tube from the ventilator.
4. Prepare the respirator by connecting the air tube to the respirator and then first turning on the air and then the respirator itself.
5. The manual pressure is set on the TrachFlush device to be 25 cm H_2O .

Setup:

There is used the same setup as the baseline test which can be found in appendix B.1

Recording:

Starting point: The endotracheal tube lies freely on a table without disturbance or pressure.

Movement: The flush function is activated.

1. The trachea is placed at the starting point on the table for 30 seconds before the first flush is activated.
2. The initial flush occurs after 30 seconds, followed by subsequent 'flushes' at 30-second intervals until 10 'flushes' have been completed.

3. After the last flush recording is stopped after 30 seconds.
4. This recording procedure is repeated until 3 data sets have been recorded.

Below is a chart of the recording procedure:

Time in minutes	Movement	Activity
0:00		The trachea lies at the starting point on the table. The recording begins.
0:30	1	Flush is performed.
1:00	2	Flush is performed.
1:30	3	Flush is performed.
2:00	4	Flush is performed.
2:30	5	Flush is performed.
3:00	6	Flush is performed.
3:30	7	Flush is performed.
4:00	8	Flush is performed.
4:30	9	Flush is performed.
5:00	10	Flush is performed.
5:30		Recording is stopped.

Results:

In figure B.3 the cuff pressure, airway pressure, and "FlushAvgOut" from the first recording are visualized. The utilized C# code pauses all measurements during a flush event. This means that there are no specific patterns in the cuff pressure related to the flush event, which further explains the pauses following each flush observed in the airway pressure graph. The pattern is not considered stable enough to develop an algorithm from, as future functions may also require pausing measurements. When the flush is activated, the parameter "FlushAvgOut" is changed which is visualized in the lower graph with a slight delay.

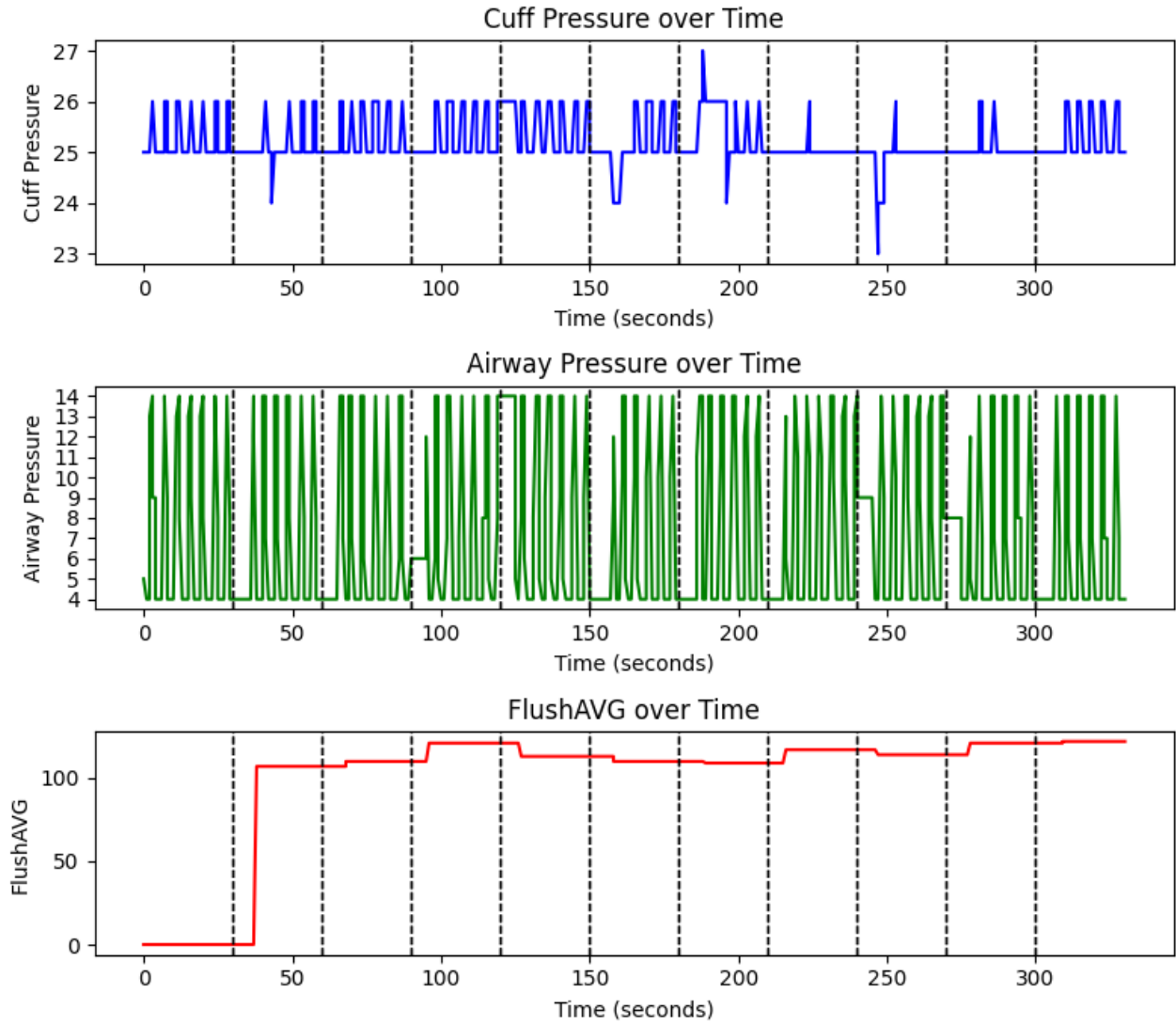


Figure B.3: The visualization of the cuff pressure, airway pressure, and "FlushAvgOut" over time under the flush test.

Conclusion:

Based on the flush test, it can be concluded that the live parameter "FlushAvgOut" is changed each time a flush is executed. This can be used as an indication that a flush has been performed.

B.3 Pressure test

Purpose:

The purpose is to record simulated pressure data on the trachea to indicate position change. Therefore, the focus lies on observing fluctuations in cuff pressure under controlled conditions.

Materials:

- TrachFlush device with associated power cable
- Cuff

- Endotracheal tube
- Computer with C# recording code
- Simulated trachea
- Respirator with associated air tube from a compressor
- Micro-B to USB A cable
- Artificial lung
- Rotary handle
- Timer
- Vise

Procedure:

Preparation before recording:

1. Prepare the relevant software to record and connect TrachFlush device.
 - a. Open the software program in Visual Studio and test whether the code can run and that a window opens where parameters from TrachFlush device can be displayed.
 - b. TrachFlush device is connected to power with the associated power cable.
 - c. USB cable connects to TrachFlush device and computer.
 - d. In the window, press 'update' to find the com port used. Then select the port in the scroll bar to the left of 'update'. If the connection has been established correctly, a popup window will appear with a message about this. In the popup window, press 'OK'.
 - e. Click "Login" to access data from TrachFlush device. By pressing 'get', the data will be visualized in the data box on the left side of the window.
 - f. To get periodic updates of the data, mark 'periodic', whereby the data is also saved in the database every half second.
2. Prepare the simulated trachea by inserting the endotracheal tube, which is done by placing the endotracheal tube with a 'deflated' cuff in the trachea.
3. Connecting the simulated trachea with the endotracheal tube to the artificial lung, as well as connecting the endotracheal tube to the air tube from the ventilator.
4. Prepare the respirator by connecting the air tube to the respirator and then first turning on the air and then the respirator itself.
5. The endotracheal tube in the trachea is placed in the vise so that the cuff on the endotracheal tube is positioned in the middle of the pressure plates of the vise. A thin string is angled around the endotracheal tube in the trachea and around the vise so that the endotracheal tube in the trachea stays in the same position even if the vise does not hold the endotracheal tube in the trachea in place. The cord must be wrapped loosely so it does not create pressure on the cuff.

Setup:

There is used the same setup as the baseline test, but added to the setup is a rotary handle as seen in figure B.4.



Figure B.4: Part of the Setup to the pressure test where a rotary handle is used to apply pressure on the cuff.

Recording:

Starting point: The endotracheal tube lies in the middle of the vise without any disturbance or pressure.

Movement: The pressure is defined as occurring when the rotary handle is adjusted so that the distance between the pressure plates of the vise decreases from 2.5 cm to 2 cm.

1. The trachea is placed at the starting point on the table for 30 seconds before the first regulation.
2. The first adjustment is made after 30 seconds where the rotary handle is adjusted to have a distance of 2 cm between the pressure plates.
3. After 5 seconds the pressure is released from the trachea.
4. The second adjustment is done after 25 seconds. This is repeated until ten pressure adjustments are done.
5. After the last pressure adjustment, an adjustment will be made back to the starting position.

6. The recording is stopped after 25 seconds in the start position.

7. This recording procedure is repeated until 3 data sets have been recorded.

Below is a chart of the recording procedure:

Time in minutes	Movement	Activity
0:00		The trachea lies at the starting point between the pressure plates of the vise. The recording begins.
0:30	1	The movement is performed and held for 5 seconds subsequently, it is placed at the starting point.
1:00	2	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
1:30	3	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
2:00	4	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
2:30	5	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
3:00	6	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
3:30	7	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
4:00	8	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
4:30	9	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
5:00	10	The movement is performed and held for 5 seconds subsequently it is placed at the starting point.
5:30		Recording is stopped.

Results:

In figure B.5, cuff pressure and airway pressure over time are visualized. The cuff pressure graph shows significant increases after each application of pressure to the cuff. The cuff pressure decreases when the endotracheal tube returns to its set cuff pressure. In the airway pressure graph, the applied pressure to the cuff does not affect the signal, as seen when compared to the airway signal in the baseline test in section B.1 in figure B.2.

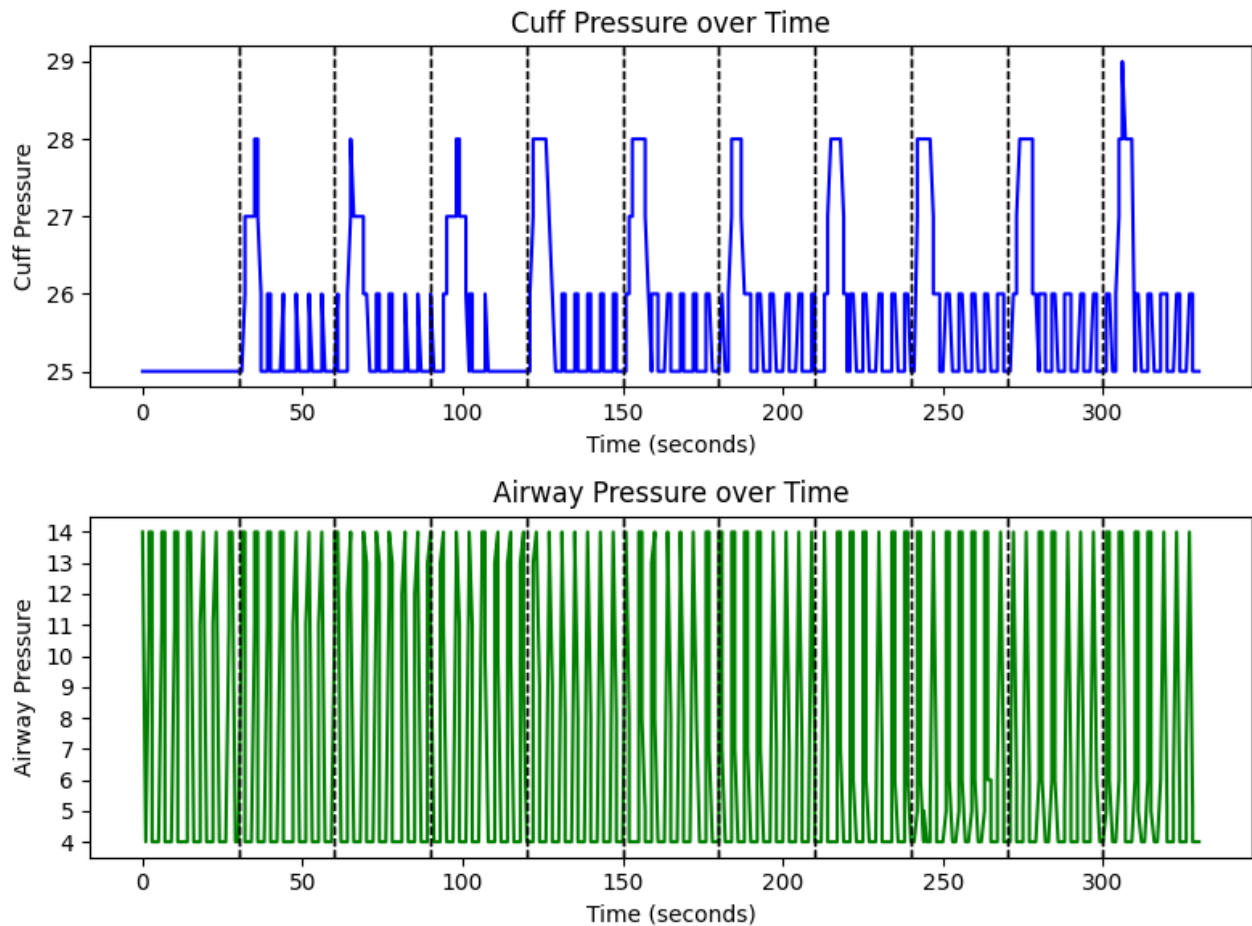


Figure B.5: The visualization of the cuff pressure and airway pressure over time under the pressure test.

Conclusion:

Based on the pressure test, it can be concluded that the cuff pressure increases when pressure is applied to the cuff. Upon release of the pressure, the TrachFlush device returns to the set value.

B.4 Cuff leakage test

Purpose

The purpose is to record a data set where leakage from the cuff is assumed to observe how the cuff pressure is regulated.

Materials:

- TrachFlush device with associated power cable
- Cuff
- Endotracheal tube
- Computer with C# recording code
- Simulated trachea

- Respirator with associated air tube from a compressor
- Micro-B to USB A cable
- Artificial lung
- Syringe needle 0.8 mm
- Timer

Procedure:

Preparation before recording:

1. Prepare the relevant software to record and connect TrachFlush device.
 - a. Open the software program in Visual Studio and test whether the code can run and that a window opens where parameters from TrachFlush device can be displayed.
 - b. TrachFlush device is connected to power with the associated power cable.
 - c. USB cable connects to TrachFlush device and computer.
 - d. In the window, press 'update' to find the com port used. Then select the port in the scroll bar to the left of 'update'. If the connection has been established correctly, a popup window will appear with a message about this. In the popup window, press 'OK'.
 - e. Click "Login" to access data from TrachFlush device. By pressing 'get', the data will be visualized in the data box on the left side of the window.
 - f. To get periodic updates of the data, mark 'periodic', whereby the data is also saved in the database every half second.
2. With a small needle, puncture the cuff to make a small hole.
3. Prepare the simulated trachea by inserting the endotracheal tube, which is done by placing the endotracheal tube with a 'deflated' cuff in the trachea.
4. Connecting the simulated trachea with the endotracheal tube to the artificial lung, as well as connecting the endotracheal tube to the mechanical ventilator tube.
5. Prepare the respirator by connecting the air tube to the respirator, firstly turning on the air and then the respirator itself.

Setup:

There is used the same setup as the baseline test which can be found in appendix B.1 There is additionally placed a small needle in the cuff to simulate the leakage as seen in figure B.6.

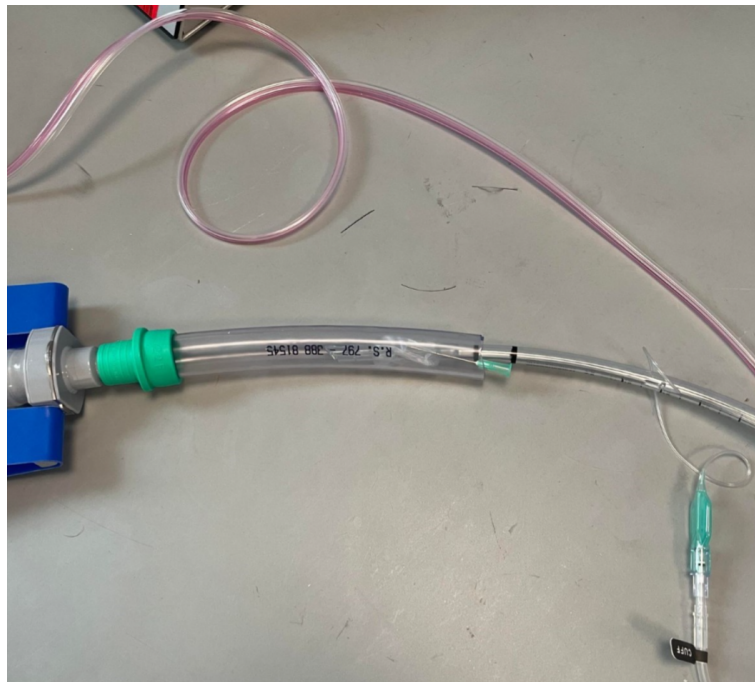


Figure B.6: The setup of the cuff leakage test where the needle is seen placed in the cuff.

Recording:

Starting point: The endotracheal tube lies freely on a table without disturbance or pressure.

1. The trachea is placed at the starting point on the table.
2. It is recorded for 5 minutes without any interactions.
3. This recording procedure is repeated until 3 data sets have been recorded.
 - a. The first dataset, manual pressure is set to be 25 cm H_2O on the TrachFlush device.
 - b. The second dataset, manual pressure is set to be 30 cm H_2O on the TrachFlush device.
 - c. The third dataset, manual pressure is set to be 20 cm H_2O on the TrachFlush device.

Below is a chart of the recording procedure:

Time in minutes	Movement	Activity
0:00		The trachea lies at the starting point on the table. The recording begins.
5:00		Recording is stopped.

Results:

In figure B.7 cuff pressure and airway pressure over time are visualized. The cuff pressure graph shows that the cuff pressure decreases further than the baseline test seen in section B.1 in figure B.2. Additionally, it appears that the TrachFlush device compensates for the falling pressure by setting

the cuff pressure too high, which then immediately falls again likely because of the leakage. In the airway pressure graph, no significant changes are observed.

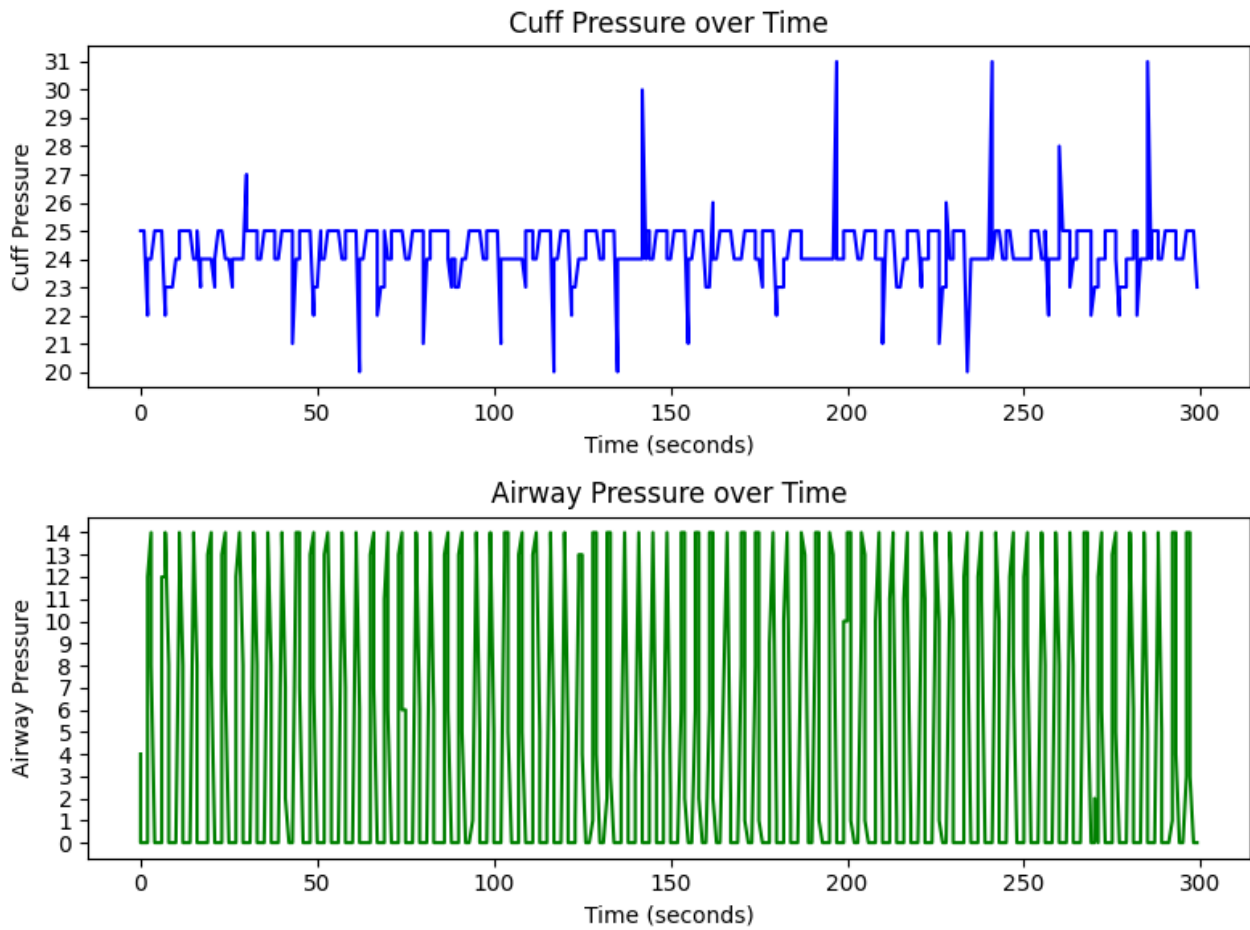


Figure B.7: The visualization of the cuff pressure and airway pressure over time under the leakage test.

Conclusion:

Based on the cuff leakage test, it can be concluded that the TrachFlush device tries to compensate for the loss of leaked air. Because of the leakage, the cuff pressure was fluctuating over and under the set cuff pressure.

C. Test Case

C.1 Unit Test Case

Block 1: Database Communication	✓ Test Results 1 ms ✓ test_DatabaseCommunication 1 ms ✓ TestDatabaseCommunication 1 ms ✓ test_db_connect 1 ms ✓ test_db_query 0 ms ✓ test_execute_query 0 ms
Block 2: Screen Login	✓ Test Results 1 ms ✓ test_ScreenLogin 1 ms ✓ TestScreenLogin 1 ms ✓ test_clear_entry_box 1 ms ✓ test_login 0 ms ✓ test_open_screenlayout 0 ms
Block 3: Screen Layout	✓ Test Results 0 ms ✓ test_ScreenLayout 0 ms ✓ TestScreenLayout 0 ms ✓ test_check_inactivity 0 ms ✓ test_configure_styles 0 ms ✓ test_logout 0 ms ✓ test_open_new_tab_screer 0 ms ✓ test_retrieve_treatment_id: 0 ms ✓ test_update_activity_time 0 ms ✓ test_update_tm_id_list 0 ms
Block 4: Add New Treatment ID	✓ Test Results 1 ms ✓ test_AddTreatmentID 1 ms ✓ TestAddTreatmentID 1 ms ✓ test_check_entry_length 1 ms ✓ test_create_treatment_id 0 ms ✓ test_destroy_add_treatmer 0 ms ✓ test_get_patient_id 0 ms ✓ test_get_tfid 0 ms ✓ test_get_treatment_id 0 ms ✓ test_send_patient_id_tfid_t 0 ms ✓ test_validate_int 0 ms

Block 5: Calculate Events and Statistics	✓ Test Results 2 ms ✓ test_SignalProcessingScreen1 2 ms ✓ TestSignalProcessingScreen1 2 ms ✓ test_calculate_mean 2 ms ✓ test_check_over_under 0 ms ✓ test_count_flush_events 0 ms ✓ test_over_under_events 0 ms ✓ Test Results 2 ms ✓ test_SignalProcessingScreen2 2 ms ✓ TestSignalProcessingScreen2 2 ms ✓ test_all_events_def 1 ms ✓ test_all_events_list 0 ms ✓ test_calculate_mean 0 ms ✓ test_check_over_under 0 ms ✓ test_combined_plot 0 ms ✓ test_count_flush_events 0 ms ✓ test_flush_event_change 1 ms ✓ test_get_data_table_time 0 ms ✓ test_over_under_events 0 ms ✓ test_show_alert 0 ms ✓ test_show_events 0 ms
Block 6: Dashboard 1	✓ Test Results 2 ms ✓ test_Screen1 2 ms ✓ TestScreen1 2 ms ✓ test_create_barplot 2 ms ✓ test_create_plechart 0 ms ✓ test_plot_data 0 ms ✓ test_synchronize_start_time 0 ms
Block 7: Dashboard 2	✓ Test Results 1 ms ✓ test_Screen2 1 ms ✓ TestScreen2 1 ms ✓ test_header_screen2 1 ms ✓ test_plot_airwaypressure 0 ms ✓ test_plot_cuffpressure 0 ms ✓ test_set_combined_plot 0 ms ✓ test_set_event_list 0 ms

C.2 Block Testing

Module Name:		Database Communication				
Test Case ID:		BTC01				
Test Case Description:		The integration between the database and the software				
Environmental information:		PyCharm Community, PhpMyAdmin, Microsoft Visual Studio				
Test ID	Test Condition	Test Steps	Test Input	Test Expected Result	Actual Result	Status
1	Verify that a connection between the system and the database is being established.	Open PyCharm Community and execute the 'DatabaseCommunication' class. Verify that the command window displays the message 'The database is connected' and confirmation	No input	The system displays the message 'The database is connected', indicating successful establishment of the connection between the system and the database	The system displays the message 'The database is connected', indicating successful establishment of the connection between the system and the database	Pass
2	Verify that the database saves cuff pressure data	Open PhpMyAdmin and select the 'DataTable'. Verify that the column 'CuffPressure' exist and contains cuff pressure data	No input	The 'DataTable' has the column 'CuffPressure' that consist of cuff pressure data	The 'DataTable' has the column 'CuffPressure' that consist of cuff pressure data	Pass
3	Verify that the database saves airway pressure data	Open PhpMyAdmin and select the 'DataTable'. Verify that the column 'AirwayPressure' exist and contains air pressure values	No input	The 'DataTable' has the column 'AirwayPressure' that consist of airway pressure data	The 'DataTable' has the column 'AirwayPressure' that consist of airway pressure data	Pass
4	Verify that the database saves TrachFlush ID	Open PhpMyAdmin and select the 'Treatment' table. Verify that the column 'TrachFlush ID' exist and contains different IDs	No input	The 'Treatment' table has the column 'TrachFlush ID' that consist of different IDs	The 'Treatment' table has the column 'TrachFlush ID' that consist of different IDs	Pass
5	Verify that the database saves Patient ID	Open PhpMyAdmin and select the 'Treatment' table. Verify that the column 'PatientID' exist and contains different IDs	No input	The 'Treatment' table has the column 'Patient ID' that consist of different IDs	The 'Treatment' table has the column 'Patient ID' that consist of different IDs	Pass
6	Verify that the database saves Treatment ID	Open PhpMyAdmin and select the 'Treatment' table. Verify that the column 'TreatmentID' exist and contains unique IDs	No input	The 'Treatment' table has the column 'Treatment ID' that consist of unique IDs	The 'Treatment' table has the column 'Treatment ID' that consist of unique IDs	Pass

7	Verify that the database saves the time a Treatment ID is assigned	Open PhpMyAdmin and select the 'Treatment' table. Verify that the 'Time' column accurately records the temporal information corresponding to the assignment of a Treatment ID	No input	The 'Treatment' table has the column 'Time' that consist of the temporal information about the assign Treatment ID	The 'Treatment' table has the column 'Time' that consist of the temporal information about the assign Treatment ID	Pass
8	Verify that the database saves system codes from the TrachFlush	Open PhpMyAdmin and select the 'DataTable'. Verify that the 'System code' column exists	No input	The 'Database' table has the column 'System code' that consist of values indicative of various codes	The 'Database' table has the column 'System code' that consist of values indicative of various codes	Pass
9	Verify that the database saves "FlushAvgOut" from the TrachFlush	Open PhpMyAdmin and select the 'DataTable'. Verify that the 'FlushEvent' column exist, where the TrachFlush data pertaining to "FlushAvgOut" is stored	No input	The 'Database' table has the column 'FlushEvent' contains data corresponding to "FlushAvgOut"	The 'Database' table has the column 'FlushEvent' contains data corresponding to "FlushAvgOut"	Pass
10	Verify that the database store personal login which contains a username and password	Open PhpMyAdmin and select the 'Login' table. Verify that the table consist of the columns 'Username' and 'Password'	No input	The 'Login' table contain distinct and unique record comprising both 'Username' and 'Password'	The 'Login' table contain distinct and unique record comprising both 'Username' and 'Password'	Pass
11	Verify that the database store TrachFlush data every 500 ms in the 'DataTable'	Open Microsoft Visual Studio and execute the 'Firm1' file. Confirm the connection of a TrachFlush device to the computer and that it is actively recording data. Open PhpMyAdmin and verify that the recorded data is consistently received in the database at intervals of 500 milliseconds.	No input	The database successfully receives data from the active TrachFlush device at 500 ms intervals	The database successfully receives data from the active TrachFlush device at 500 ms intervals	Pass
12	Verify that all collected data in the 'DataTable' is associated with a timestamp.	Open PhpMyAdmin and select the 'DataTable'. Verify that all rows as a timestamp in the 'Time' column	No input	All data in the 'DataTable' are associated with timestamp in the 'Time' column	All data in the 'DataTable' are associated with timestamp in the 'Time' column	Pass

Module Name:		Dashboard Login				
Test Case ID:		BTC02				
Test Case Description:		To verify the login function				
Environmental information:		PyCharm Community				
Test ID	Test Condition	Test Steps	Test Input	Test Expected Result	Actual Result	Status
1	Verify if the login window gets displayed	Open the system and see if a login window opens	No input	A login window is presented	A login window is presented	Pass
2	Verify if the login page presents a username input box	Open the system and see if there is a username input box	No input	There is a username input box	There is a username input box	Pass
3	Verify if the login page presents a password input box	Open the system and see if there is a password input box	No input	There is a password input box	There is a password input box	Pass
4	Verify if an error is reported by entering a non-existing username with an existing password by checking the database	Open the system and input the username and password. Followed by clicking the 'Login' button	Username:" More22" Password: "PW1"	A pop-up warning message appears stating that the login does not exist	A pop-up warning message appears stating that the login does not exist	Pass
5	Verify if an error is reported by entering an existing username with a non-existing password by checking the database	Open the system and input the username and password. Followed by clicking the 'Login' button	Username: "User1" Password: "Less22"	A pop-up warning message appears stating that the login does not exist	A pop-up warning message appears stating that the login does not exist	Pass

6	Verify if an error is reported by not entering a username and password	Open the system and input the username and password. Followed by clicking the 'Login' button	No input	A pop-up warning message appears stating that the login does not exist	A pop-up warning message appears stating that the login does not exist	Pass
7	Verify if entering a valid username and password, saved in the database, leads to the front page	Open the system and input the username and password. Followed by clicking the 'Login' button	Username: "User1" Password: "PW1"	The system directs into the front page	The system directs into the front page	Pass

Module Name:		Dashboard Layout				
Test Case ID:		BTC03				
Test Case Description:		To verify that the layout is displayed				
Environmental information:		PyCharm Community				
Test ID	Test Condition	Test Steps	Test Input	Test Expected Result	Actual Result	Status
1	Verify that a list with TMIDs is shown	Open and login to the system and see if a list with TMIDs is presented.	No input	A list with the existing TMIDs from the database is shown.	A list with the existing TMIDs from the database is shown.	Pass
2	Verify by clicking on a TMID in the list a tab with the same name appears on the bar.	Click on a TMID and see if a new tab opens on the bar with the same name	No input	A new tab opens with the selected TMID	A new tab opens with the selected TMID	Pass
3	Verify by clicking on a TMID in the list the system directs into the selected TMID tab	Click on a TMID and verify if a new tab containing the selected TMID opens	No input	A new tab opens with the selected TMID, and the system directs into that	A new tab opens with the selected TMID, and the system directs into that	Pass
4	Verify that selecting the checkbox for a specific TMID excludes it from the statistics on dashboard1	Click on a checkbox for a selected TMID on dashboard 1. Make sure that the checkmark disappears	No input	The selected TMID is excluded from the statistics on dashboard 1	The checkbox does not have any functionality	Fail
5	Verify that by clicking the 'Add New Treatment ID' button opens a pop-up window	Click on the button 'Add New Treatment ID'	No input	A pop-up window appears after clicking on the Add New Treatment ID" button	A pop-up window appears after clicking on the Add New Treatment ID" button	Pass

6	Verify that the system locks out automatically after 5 minutes of inactivity	The front page is opened. Nothing will be clicked on for 5 minutes	No input	The systems automatically lock out after 5 minutes of inactivity	The systems automatically lock out after 5 minutes of inactivity	Pass
7	Verify that the username is shown on the dashboard layout	See if the right username is placed on the dashboard layout	No input	The username is presented on the dashboard layout	The right username is not presented on the dashboard layout	Fail
8	Verify that the system exists by clicking on the "Log out" button	See if the system exists by click on the "Log out" button	No input	The system exists after clicking the "Log out" button	The system exists after clicking the "Log out" button	Pass

Module Name:		Add New Treatment ID				
Test Case ID:		BTC04				
Test Case Description:		To verify the Add New Treatment ID function				
Environmental information:		PyCharm Community				
Test ID	Test Condition	Test Steps	Test Input	Test Expected Result	Actual Result	Status
1	Verify that the "Add New Treatment ID" window presents a 'Patient ID' input box	Open 'Add New Treatment ID' and see if there is a 'Patient ID' input box	No input.	There's is a 'Patient ID' input box	There's is a TrachFlush ID 'input box	Pass
2	Verify that the "Add New Treatment ID" window presents a 'TrachFlush ID' input box	Open 'Add New Treatment ID' and see if there is a 'TrachFlush ID' input box	No input.	There's is a TrachFlush ID 'input box	There's is a TrachFlush ID 'input box	Pass
3	Verify that the 'Patient ID' input box does not accept letters as a valid input	Open 'Add New Treatment ID' and write letters as input in 'Patient ID' box	"ABCD"	The system does not accept the input	The system does not accept the input	Pass
4	Verify that the TrachFlush ID' input box does not accept letters As a valid input	Open 'Add New Treatment ID' and write letters as input in 'TrachFlush ID' box	"ABCD"	The system does not accept the input	The system does not accept the input	Pass
5	Verify that the system requires a minimum of two numbers as input in the 'Patient ID' input box	Open 'Add New Treatment ID' and write 1 number as input in 'Patient ID' box	"5"	The system does not accept the input	The system does not accept the input	Pass

6	Verify that the system requires a minimum of two numbers as input in the TrachFlush ID' input box	Open 'Add New Treatment ID' and write 1 number as input in 'TrachFlush ID' box	"5"	The system does not accept the input	The system does not accept the input	Pass
7	Verify that the 'Add Treatment ID' button is accessible only when the input requirements for both the 'Patient ID' and 'TrachFlush ID' boxes are met	Open 'Add New Treatment ID' and enter a minimum of 2 digits in both the 'Patient ID' and 'TrachFlush ID' boxes	Patient ID: "23" TrachFlush ID: "098"	The system allows pressing the 'Add Treatment ID' button	The system allows pressing the 'Add Treatment ID' button	Pass
8	Verify that the system generates a random Treatment ID after clicking on 'Add Treatment ID' button	Enter a valid 'Patient ID' and 'TrachFlush ID', then click on the 'Add Treatment ID' button. Check if a unique Treatment ID is generated	No input	The system generates a unique Treatment ID	The system generates a unique Treatment ID	Pass

Module Name:		Calculate Events and Statistics				
Test Case ID:		BTC05				
Test Case Description:		To verify calculations of events and statistics				
Environmental information:		PyCharm Community				
Test ID	Test Condition	Test Steps	Test Input	Test Expected Result	Actual Result	Status
1	Verify that the system detects when the cuff pressure is higher than 1 or more than the set pressure	Record the data for a new TMID where the manual set cuff pressure is 25 cm H ₂ O. Execute the code module 'SignalProcessing' to visualize the cuff pressure. Use the function 'count_over' to determine the number of instances where the cuff pressure has reached or exceeded 26 cm H ₂ O. Sum the occurrences and compare this value to the expected value detected by the function 'check_over_under' from the 'SignalProcessing1' class	Cuff pressure data	The system accurately detects and records the number of instances where the cuff pressure exceeds the manually set pressure by 1 cm H ₂ O or more. This detection is as precise as manually summing the number of these occurrences	The system accurately detects and records the number of instances where the cuff pressure exceeds the manually set pressure by 1 cm H ₂ O or more. This detection is as precise as manually summing the number of these occurrences	Pass

2	Verify that the system detects when the cuff pressure is lower than 1 or more than the set pressure	Record the data for a new TMID where the manual set cuff pressure is 25 cm H ₂ O. Execute the code module 'SignalProcessing' to visualize the cuff pressure. Use the function 'count_under' to determine the number of instances where the cuff pressure has been 24 cm H ₂ O or lower. Sum the occurrences and compare this value to the expected value detected by the function 'check_over_under' from the 'SignalProcessing1' class	Cuff pressure data	The system accurately detects and records the number of instances where the cuff pressure is lower than the manually set pressure by 1 cm H ₂ O or more. This detection is as precise as manually summing the number of these occurrences	The system accurately detects and records the number of instances where the cuff pressure is lower than the manually set pressure by 1 cm H ₂ O or more. This detection is as precise as manually summing the number of these occurrences	Pass
---	---	---	--------------------	--	--	------

3	Verify that the system detects when a flush has occurred	Record data for a new TMID where the manually set cuff pressure is 25 cm H ₂ O. Over a duration of 2 minutes, perform a flush every 30 seconds. Subsequently, access the database using phpMyAdmin, navigate to the 'DataTable', and locate the new TMID. Verify that three different values are recorded in the 'FlushEvent' column. Then, execute the function 'flush_event_change' in the 'Signalprocessing2' class and compare the number of detected flushes with the manually counted number of flushes	Cuff pressure data	The system accurately detects when a flush has occurred, matching the results obtained through manual detection	The system accurately detects when a flush has occurred, matching the results obtained through manual detection	Pass
---	--	--	--------------------	---	---	------

4	Verify that the system calculates the mean of all cuff pressure values	Access the database and export the 'DataTable' into an Excel file. From the Excel file, calculate the average of all TMIDs' 'CuffPressure'. Next, use the system to calculate the cuff pressure by utilizing the 'SignalProcessing1' class and the function 'calculate_mean'. Compare the average cuff pressure obtained from the Excel file with the average calculated by the system	Cuff pressure data	The system accurately calculates the average cuff pressure for all TMIDs compared to the manually calculated cuff pressure in Excel	The system accurately calculates the average cuff pressure for all TMIDs compared to the manually calculated cuff pressure in Excel	Pass
---	--	--	--------------------	---	---	------

Module Name:		Dashboard1				
Test Case ID:		BTC06				
Test Case Description:		To verify that dashboard1 functions				
Environmental information:		PyCharm Community				
Test ID	Test Condition	Test Steps	Test Input	Test Expected Result	Actual Result	Status
1	Verify that a graphical visualization of cuff pressure for all TMID 's is shown on dashboard1	Open the front page. Make sure that the graphical visualization of cuff pressure for all TMID 's are shown on the front page.	No input	The graphical visualization of cuff pressure for all TMID 's are shown	The graphical visualization of cuff pressure for all TMID 's are shown	Pass
2	Verify that dashboard1 shows a statistical overview of all mean cuff pressure for every TMID as a bar chart	Open the front page. Make sure that a bar chart of mean for every TMID is shown.	No input	The statistical overview of all mean cuff pressure for every TMID is shown as a bar chart	The statistical overview of all mean cuff pressure for every TMID is shown as a bar chart	Pass
3	Verify that dashboard1 shows a pie chart as a statistical overview of all events for all TMIDs	Open the front page. Make sure that a pie chart of all events is shown.	No input	A pie chart is shown as a statistical overview of all events for all TMIDs	A pie chart is shown as a statistical overview of all events for all TMIDs	Pass

Module Name:		Dashboard2				
Test Case ID:		BTC07				
Test Case Description:		To verify that dashboard2 functions				
Environmental information:		PyCharm Community				
Test ID	Test Condition	Test Steps	Test Input	Test Expected Result	Actual Result	Status
1	Verify that a graphical visualization of cuff pressure over time is shown for the selected TMID	Open the front page. Click on a specific TMID in the list. When in the new tab see if a cuff pressure visualization is shown	No input	The cuff pressure visualization for the specific TMID appears	The cuff pressure visualization for the specific TMID appears	Pass
2	Verify that a graphical visualization of airway pressure over time is shown for the selected TMID	Open the front page. Click on a specific TMID in the list. When in the new tab see if a cuff pressure visualization is shown	No input	The airway pressure visualization for the specific TMID appears	The airway pressure visualization for the specific TMID appears	Pass
3	Verify that the patient ID and TrachFlush ID is shown for the selected TMID	Go to a specific TMID tab. See if both the patient ID and TrachFlush ID are shown	No input	The patient ID and TrachFlush ID for the selected TMID appears	The patient ID and TrachFlush ID for the selected TMID appears	Pass

4	Verify that the duration that a specific TMID cuff has been controlled by a TrachFlush is shown	Go to a specific TMID tab. See if the duration of the time a TMID cuff has been controlled by a TrachFlush is presented	No input	The duration for the selected TMID is show	The duration for the selected TMID is show	Pass
5	Verify that the start of the treatment for the specific TMID is shown	Go to a specific TMID tab. See if the start of the treatment for a specific TMID is presented	No input	The start of the treatment for the selected TMID is show	The start of the treatment for the selected TMID is show	Pass
6	Verify that the events for the specific TMID are shown on the graph of the cuff pressure	Go to a specific TMID tab. See if the events are marked on the graphical visualization of the cuff pressure for the specific TMID	No input	The events are clearly marked on the graphical visualization of the cuff pressure for the specific TMID	The events are clearly marked on the graphical visualization of the cuff pressure for the specific TMID	Pass
7	Verify that the alarms for the specific TMID are shown on the graph of the cuff pressure.	Go to a specific TMID tab. See if the alarms are marked on the graphical visualization of the cuff pressure for the specific TMID	No input	The alarms are clearly marked on the graphical visualization of the cuff pressure for the specific TMID	The alarms are not marked on the graphical visualization of the cuff pressure for the specific TMID	Fail

8	Verify that a list which includes all events, and their start time is shown	Go to a specific TMID tab and verify if a list containing all event and their start time is displayed	No input	The event list is displayed, showing their start time	The event list is displayed, showing their start time	Pass
9	Verify that a list which includes all alarms, and their start time is shown	Go to a specific TMID tab and verify if a list containing all alarms and their start time is displayed	No input	The alarm list is displayed, showing their start time	The alarm list is empty	Fail
10	Verify that clicking on an event from the list highlights it on the cuff pressure graph for the specific TMID	Go to a specific TMID tab and verify if a list containing all events is displayed. Click on an event and verify that the event is highlighted on the cuff pressure graph	No input	The event that is clicked becomes highlighted on the cuff pressure graph	The event that is clicked does not become highlighted on the cuff pressure graph	Fail
11	Verify that a warning message is displayed if there is less than one hour of data	Go to a TMID tab where no data is available. Verify that a warning message is displayed indicating the absence of data	No input	A warning message is presented indicating the absence of data	A warning message is presented indicating the absence of data, which is not time-dependent, meaning it is displayed only when there is no data available	Fail